

**RANCANG BANGUN SISTEM PENDINGIN
OTOMATIS DENGAN KONTROLER PID
UNTUK EKSTRAKSI COLD BREW COFFEE**

TUGAS AKHIR

Diajukan sebagai salah satu syarat
untuk memperoleh gelar Sarjana Terapan (S.Tr)



**ANUGERAH ILAHI
NIM. 2211013017**

**PROGRAM STUDI DIV TEKNIK ELEKTRONIKA
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI PADANG
2026**

LEMBAR PENGESAHAN TUGAS AKHIR
TAHUN 2025/2026

RANCANG BANGUN SISTEM PENDINGIN OTOMATIS
DENGAN KONTROLER PID
UNTUK EKSTRAKSI COLD BREW COFFEE

Oleh:
Anugerah Ilahi
NIM. 2211013017

Program Studi DIV Teknik Elektronika
Jurusan Teknik Elektro
Politeknik Negeri Padang

Disetujui/disahkan oleh :
Pembimbing I Pembimbing II

Ir. Reza Nandika, S.T., M.Eng.
NIP. 19860709 202203 1 003

Anton Hidayat, S.T., M.T.
NIP. 19761025 200501 1 002

Ketua Jurusan Teknik Elektro

Koordinator Program Studi
DIV Teknik Elektronika

Ismail, S.S.T., M.T., Ph.D
NIP. 19731026 200312 1 001

Anton Hidayat, S.T., M.T.
NIP. 19761025 200501 1 002

HALAMAN PERSETUJUAN PENGUJI

RANCANG BANGUN SISTEM PENDINGIN OTOMATIS DENGAN KONTROLER PID UNTUK EKSTRAKSI COLD BREW COFFEE

Oleh:

Anugerah Ilahi
NIM. 2211013017

Tugas Akhir ini telah dipertanggungjawabkan di hadapan
Tim Penguji Tugas Akhir pada tanggal

Tim Penguji :

Ketua

Sekretaris

Nadia Alfitri, S.T., M.T.
NIP. 19760924 200312 2 042

Yul Antonisfia, S.T., M.T.
NIP. 19680726 199303 1 002

Anggota

Anggota

Tuti Anggraini, S.ST., M.T.
NIP. 19670930 199303 2 003

Ir. Reza Nandika, S.T., M.Eng.
NIP. 19860709 202203 1 003

PERNYATAAN BEBAS PLAGIARISME

Saya yang bertanda tangan di bawah ini:

Nama : Anugerah Ilahi
NIM : 2211013017
Tahun Tugas Akhir : 2026
Program Studi : DIV Teknik Elektronika
Jurusan : Teknik Elektro

Menyatakan bahwa Laporan Tugas Akhir ini adalah hasil karya sendiri dan bukan merupakan hasil plagiarisme dari karya orang lain dan belum pernah dikumpulkan oleh penulis lain untuk memperoleh gelar dari berbagai jenjang pendidikan di Perguruan Tinggi manapun. Semua sumber yang digunakan dalam penyusunan karya ini telah dicantumkan secara jelas sesuai dengan kaidah penulisan ilmiah yang berlaku.

Apabila dokumen ini di kemudian hari terbukti merupakan hasil plagiasi dari karya penulis lain dan/atau dengan sengaja mengajukan hasil karya penulis lain, maka saya sebagai penulis bersedia menerima sanksi akademik, pembatalan ijazah dan/atau sanksi hukum sesuai perundang-undangan yang berlaku.

Demikian pernyataan ini saya buat dengan sebenar-benarnya dan penuh tanggung jawab.

Padang, 23 Juli 2026

Anugerah Ilahi
NIM. 2211013017

SURAT PERNYATAAN PERSETUJUAN PUBLIKASI KARYA TULIS ILMIAH UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademika Prodi Sarjana Program Studi DIV Teknik Elektronika Jurusan Teknik Elektro Politeknik Negeri Padang, yang bertanda tangan di bawah ini saya:

Nama : Anugerah Ilahi
NIM : 2211013017
Tahun Tugas Akhir : 2026
Program Studi : DIV Teknik Elektronika
Jurusan : Teknik Elektro

Demi pengembangan ilmu pengetahuan, saya menyetujui untuk memberikan kepada Program Studi DIV Teknik Elektronika Jurusan Teknik Elektro Politeknik Negeri Padang, Hak Bebas Royalti Non-Eksklusif (Non-Exclusive Royalty-Free Right) atas Tugas Akhir saya yang berjudul “Rancang Bangun Sistem Pendingin Otomatis dengan Kontroler PID Untuk Ekstraksi Cold Brew Coffee” beserta perangkat yang ada (bila diperlukan). Dengan Hak Bebas Royalti Non-Eksklusif ini Program Studi DIV Teknik Elektronika Jurusan Teknik Elektro Politeknik Negeri Padang berhak menyimpan, mengalih media/memformat, mengelola dalam bentuk pangkalan data (database), mendistribusikan, dan menampilkan/ mempublikasikan Tugas Akhir saya di internet/media lain untuk kepentingan akademis tanpa perlu meminta izin dari saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta. Saya bersedia untuk menanggung secara pribadi tanpa melibatkan pihak Program Studi DIV Teknik Elektronika Jurusan Teknik Elektro Politeknik Negeri Padang, segala bentuk tuntutan hukum yang diambil atas pelanggaran hak dalam karya ilmiah saya ini. Demikian Surat Pernyataan ini saya buat dengan sebenarnya.

Padang, 23 Juli 2026

Anugerah Ilahi
NIM. 2211013017

ABSTRAK

Pengendalian suhu diperlukan pada sistem pendingin agar proses dapat berlangsung stabil dan terukur. Penelitian ini bertujuan merancang sistem pendingin otomatis dengan kontroler PID untuk menjaga suhu pada nilai *setpoint* yang ditentukan. Sistem menggunakan ESP32 sebagai pengendali utama, sensor DS18B20 sebagai pembaca suhu, Peltier TEC1-12706 sebagai aktuator pendingin, driver BTS7960 sebagai pengatur daya, RTC DS3231 sebagai acuan waktu, dan LCD 20×4 sebagai antarmuka tampilan. Sistem diterapkan pada proses ekstraksi *cold brew coffee* sebagai objek pengujian karena proses tersebut membutuhkan suhu rendah dalam durasi tertentu. Parameter PID ditentukan menggunakan metode Ziegler–Nichols berdasarkan respons *open-loop*. Hasil pengujian *open-loop* menghasilkan gain proses 0,0837 °C/PWM, waktu tunda 0,43 menit, dan konstanta waktu 6,47 menit. Parameter PID yang diterapkan adalah $K_p = 215,72$, $K_i = 4,181$, dan $K_d = 2782,79$. Pengujian *closed-loop* menunjukkan sistem mencapai daerah *setpoint* dalam 4,62 menit pada 15 °C, 19,33 menit pada 8 °C, dan 38,63 menit pada 5 °C. Nilai *overshoot* masing-masing sebesar 0,35 °C, 0,27 °C, dan 0,26 °C, sedangkan *steady-state error* sebesar 0,026 °C, 0,037 °C, dan 0,026 °C. Pengujian penerapan pada 5 °C selama 20 jam menunjukkan sistem PID menghasilkan rata-rata pH 5,33 dan TDS 1043,33 ppm, sedangkan kulkas biasa menghasilkan rata-rata pH 5,37 dan TDS 1001,00 ppm. Hasil pengujian menunjukkan bahwa sistem pendingin otomatis berbasis PID mampu mencapai dan mempertahankan suhu di sekitar *setpoint* dengan nilai *overshoot* dan *steady-state error* yang kecil.

Kata kunci: sistem pendingin otomatis, ESP32, DS18B20, Peltier, PID, Ziegler–Nichols.

ABSTRACT

Temperature control is required in cooling systems to maintain a stable and measurable process. This study aims to design an automatic cooling system with a PID controller to maintain temperature at a predetermined *setpoint*. The system uses an ESP32 as the main controller, a DS18B20 sensor for temperature measurement, a TEC1-12706 Peltier module as the cooling actuator, a BTS7960 driver for power regulation, an RTC DS3231 as the time reference, and a 20×4 LCD as the display interface. The system was applied to *cold brew coffee* extraction as the test object because the process requires low temperature for a specific duration. PID parameters were determined using the Ziegler–Nichols method based on the *open-loop* response. The *open-loop* test produced a process gain of 0.0837 °C/PWM, a delay time of 0.43 min, and a time constant of 6.47 min. The implemented PID parameters were $K_p = 215.72$, $K_i = 4.181$, and $K_d = 2782.79$. The *closed-loop* test showed that the system reached the *setpoint* region in 4.62 min at 15 °C, 19.33 min at 8 °C, and 38.63 min at 5 °C. The *overshoot* values were 0.35 °C, 0.27 °C, and 0.26 °C, while the *steady-state errors* were 0.026 °C, 0.037 °C, and 0.026 °C, respectively. The application test at 5 °C for 20 h showed that the PID cooling system produced an average pH of 5.33 and TDS of 1043.33 ppm, while the conventional refrigerator produced an average pH of 5.37 and TDS of 1001.00 ppm. The results show that the PID-based automatic cooling system can reach and maintain the temperature around the *setpoint* with low *overshoot* and *steady-state error*.

Keywords: automatic cooling system, ESP32, DS18B20, Peltier, PID, Ziegler–Nichols.

KATA PENGANTAR

Puji syukur penulis ucapkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan tugas akhir ini dapat diselesaikan. Laporan ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Terapan pada Program Studi DIV Teknik Elektronika, Jurusan Teknik Elektro, Politeknik Negeri Padang.

Laporan tugas akhir ini membahas “Rancang Bangun Sistem Pendingin Otomatis dengan Kontroler PID Untuk Ekstraksi Cold Brew Coffee.” Sistem dirancang untuk mengendalikan suhu menggunakan Peltier, sensor DS18B20, driver BTS7960, dan metode PID, serta mengatur durasi ekstraksi menggunakan RTC DS3231.

Laporan tugas akhir ini bertujuan sebagai persyaratan dalam kelulusan mahasiswa tingkat akhir agar dapat memperoleh gelar Sarjana Terapan Teknik (S.Tr.T). Selama menjalani penyelesaian tugas akhir dan penyusunan laporan ini, penulis telah banyak mendapatkan bimbingan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu, dalam kesempatan ini penulis menyampaikan terima kasih kepada :

1. Kepada orang tua dan keluarga penulis atas segala dukungan dan bantuannya, baik dalam segi finansial maupun semangat kepada penulis selama pengerjaan tugas akhir.
2. Bapak Ir. Revalin Herdianto, ST., M.Sc., Ph.D. selaku Direktur Politeknik Negeri Padang.
3. Bapak Ismail, SST., MT., Ph.D selaku Ketua Jurusan Teknik Elektro Politeknik Negeri Padang.
4. Anton Hidayat, S.T., M.T. selaku Ketua Program Studi D-IV Teknik Elektronika Industri Politeknik Negeri Padang.
5. Bapak Ir. Reza Nandika, S.T., M.Eng. selaku Dosen Pembimbing 1 yang telah banyak memberikan bimbingan dan pengarahan serta saran dalam penyusunan tugas akhir ini

6. Bapak Anton Hidayat, ST., MT. selaku dosen pembimbing 2 yang telah banyak memberikan bimbingan dan pengarahan serta saran dalam penyusunan tugas akhir ini.
7. Seluruh staf pengajar, staf teknisi, dan staf administrasi Jurusan Teknik Elektro.
8. Keluarga besar DIV Teknik Elektronika 2021 yang telah memberikan semangat dan membantu penulis dalam penyusunan tugas akhir ini.
9. Semua pihak yang telah membantu baik secara langsung maupun tidak langsung yang tidak dapat dituliskan satu per satu.

Padang, 23 Juli 2026

Anugerah Ilahi
NIM. 2211013017

DAFTAR ISI

LEMBAR PENGESAHAN TUGAS AKHIR TAHUN 2025/2026.....	i
HALAMAN PERSETUJUAN PENGUJI.....	ii
PERNYATAAN BEBAS PLAGIARISME	iii
SURAT PERNYATAAN PERSETUJUAN PUBLIKASI KARYA TULIS ILMIAH UNTUK KEPENTINGAN AKADEMIS.....	iv
ABSTRAK	v
<i>ABSTRACT</i>	vi
KATA PENGANTAR.....	vii
DAFTAR ISI	ix
DAFTAR GAMBAR	xii
DAFTAR TABEL.....	xiii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	4
1.3 Batasan masalah.....	4
1.4 Tujuan.....	5
1.5 Manfaat	6
1.6 Sistematika Penulisan.....	6
BAB II TINJAUAN PUSTAKA	9
2.1 Penelitian Terkait	9
2.2 Sistem Pendingin Termoelektrik.....	10
2.3 Sistem Kendali Suhu	14
2.4 Kontroler Proportional–Integral–Derivative (PID)	15
2.5 Metode Ziegler–Nichols	18
2.6 Pulse Width Modulation (PWM)	21
2.7 ESP32	23
2.8 Sensor Suhu DS18B20.....	26
2.9 Real Time Clock DS3231	28
2.10 Driver BTS7960.....	31
2.11 Liquid Crystal Display (LCD) 20×4 I ² C.....	35

2.12 <i>Push Button</i>	38
2.13 Buzzer	40
2.14 Sistem Pelepasan Panas dan Isolasi Termal	42
2.14.1 <i>Heatsink</i>	44
2.14.2 Kipas DC.....	45
2.14.3 <i>Thermal Paste</i>	47
2.15 Catu Daya	47
2.16 MATLAB.....	50
2.17 Arduino IDE	51
BAB III METODOLOGI PENELITIAN.....	52
3.1 Tahapan Penelitian.....	52
3.2 Perancangan Sistem.....	53
3.2.1 Prinsip Kerja Sistem	54
3.2.2 Pembagian Fungsi Komponen	54
3.3 Perancangan Perangkat Keras	55
3.3.1 Rangkaian Peltier dan Driver BTS7960	56
3.3.2 Rangkaian Antarmuka Pengguna.....	58
3.3.3 Rangkaian Sensor Suhu DS18B20	59
3.3.4 Rangkaian RTC DS3231	60
3.4 Perancangan Perangkat Lunak	62
3.4.1 Pembacaan Sensor dan Waktu	64
3.4.2 Logika Program dan Keluaran PWM	66
3.4.3 Tampilan LCD dan Indikator Buzzer	67
3.5 Perancangan Mekanik dan Sistem Pendingin	69
3.5.1 Desain Konstruksi Alat.....	70
3.5.2 Susunan Peltier, Pelat Dingin, <i>Heatsink</i> , dan Kipas	71
3.5.3 Posisi Sensor	72
3.6 Perancangan dan Penalaan Kontroler PID	73

3.6.1 Pengujian <i>Open-Loop</i> Sistem Pendingin	74
3.6.2 Penentuan Parameter K, L, dan T	75
3.6.3 Perhitungan Parameter PID Menggunakan Metode Ziegler–Nichols ..	77
BAB IV HASIL DAN PEMBAHASAN.....	80
4.1 Pengujian Awal dan Evaluasi Perbaikan Sistem Pendingin	80
4.1.1 Pengujian Kondisi Awal Sistem Pendingin	80
4.1.2 Pengujian Sistem Pendingin Setelah Perbaikan.....	83
4.1.3 Evaluasi Hasil Perbaikan Sistem Pendingin	84
4.2 Hasil Kalibrasi Sensor DS18B20.....	86
4.3 Hasil Pengujian RTC DS3231.....	91
4.4 Hasil Pengujian Driver BTS7960 dan Peltier	93
4.5 Hasil Pengujian <i>Open-Loop</i> Sistem Pendingin.....	95
4.6 Perhitungan Parameter PID dengan Metode Ziegler–Nichols	100
4.7 Hasil Pengujian <i>Closed-Loop</i> PID.....	102
4.7.1 Pengujian <i>Closed-Loop</i> PID <i>Setpoint</i> 15 °C	102
4.7.2 Pengujian <i>Closed-Loop</i> PID <i>Setpoint</i> 8 °C	105
4.7.3 Pengujian <i>Closed-Loop</i> PID <i>Setpoint</i> 5 °C	107
4.7.4 Perbandingan Hasil Pengujian <i>Closed-Loop</i> PID	110
4.8 Perbandingan Hasil Ekstraksi Menggunakan Sistem Pendingin Otomatis dan Kulkas Biasa	112
BAB V PENUTUP	115
5.1 Kesimpulan.....	115
5.2 Saran	116
DAFTAR PUSTAKA	117

DAFTAR GAMBAR

Gambar 2. 1 Peltier termoelektrik	11
Gambar 2. 2 Gambar Blok Diagram PID	16
Gambar 2. 3 Kurva reaksi proses untuk menentukan nilai L dan T[1], [3] .. Error! Bookmark not defined.	
Gambar 2. 4 Gambar ESP32.....	24
Gambar 2. 5 Gambar Sensor DS18B20	26
Gambar 2. 6 Gambar Modul RTC DS3231	29
Gambar 2. 7 Gambar Modul Driver BTS7960	32
Gambar 2. 8 Gambar LCD I2C	35
Gambar 2. 9 Gambar <i>Push Button</i>	38
Gambar 2. 10 Gambar Buzzer	41
Gambar 2. 11 Gambar Peltier Kit	43
Gambar 2. 12 Gambar <i>Heatsink</i>	45
Gambar 2. 13 Gambar Kipas DC.....	46
Gambar 2. 14 Gambar Power Supply	48
Gambar 3. 1 Diagram blok sistem pendingin ekstraksi <i>cold brew coffee</i>	53
Gambar 3. 2 Rangkaian Peltier dan driver BTS7960	57
Gambar 3. 3 Rangkaian antarmuka pengguna.....	58
Gambar 3. 4 Rangkaian sensor suhu DS18B20.....	60
Gambar 3. 5 Rangkaian RTC DS3231.....	61
Gambar 3. 6 Diagram alir perangkat lunak sistem	63
Gambar 3. 7 Desain mekanik sistem pendingin ekstraksi <i>cold brew coffee</i>	69
Gambar 3. 8 Susunan Peltier, pelat dingin, <i>heatsink</i> , dan kipas	71
Gambar 3. 9 Posisi sensor DS18B20 pada wadah ekstraksi.....	72
Gambar 3. 10 Penentuan nilai L dan T menggunakan metode garis singgung....	76

DAFTAR TABEL

Tabel 2. 1 Tabel Spesifikasi Peltier.....	11
Tabel 2. 2 Hubungan nilai PWM 8 bit terhadap <i>duty cycle</i>	22
Tabel 2. 3 Tabel Spesifikasi ESP32	24
Tabel 2. 4 Tabel Konfigurasi Sensor DS18B20 [13]	27
Tabel 2. 5 Tabel Spesifikasi Sensor DS18B20[13].....	27
Tabel 2. 6 Tabel Konfigurasi Modul RTC DS3231	29
Tabel 2. 7 Tabel Spesifikasi Utama RTC DS3231[14]	30
Tabel 2. 8 Tabel Konfigurasi Modul BTS7960	32
Tabel 2. 9 Tabel Spesifikasi Modul BTS7960	34
Tabel 2. 10 Tabel Konfigurasi LCD I2C	36
Tabel 2. 11 Tabel Spesifikasi LCD I2C	37
Tabel 2. 12 Tabel Konfigurasi <i>Push Button</i>	39
Tabel 2. 13 Tabel Logika <i>Push Button</i>	39
Tabel 2. 14 Tabel Spesifikasi <i>Push Button</i>	40
Tabel 2. 15 Tabel Konfigurasi Buzzer	41
Tabel 2. 16 Tabel Komponen Sistem Pelepasan Panas dan Isolasi Termal	43
Tabel 2. 17 Tabel Spesifikasi Kipas DC	46
Tabel 2. 18 Tabel Pembagian Sumber Tegangan	48
Tabel 2. 19 Spesifikasi catu daya yang digunakan	50
Tabel 3. 1 Fungsi komponen pada sistem.....	55
Tabel 3. 2 Hubungan rangkaian Peltier dan BTS7960	57
Tabel 3. 3 Hubungan rangkaian antarmuka pengguna	59
Tabel 3. 4 Hubungan sensor DS18B20 dengan ESP32	60
Tabel 3. 5 Hubungan RTC DS3231 dengan ESP32.....	61
Tabel 3. 6 Data masukan pada perangkat lunak sistem	65
Tabel 3. 7 Hubungan kondisi suhu terhadap keluaran PWM	67
Tabel 3. 8 Rancangan tampilan LCD 20×4	68
Tabel 3. 9 Komponen mekanik dan sistem pendingin.....	70
Tabel 3. 10 Parameter pengujian <i>open-loop</i> sistem pendingin.....	74

Tabel 4. 1 Kondisi Pengujian Awal Sistem Pendingin	80
Tabel 4. 2 Data Suhu terhadap Waktu pada Kondisi Awal	80
Tabel 4. 3 Kondisi Pengujian Sistem Pendingin Setelah Perbaikan.....	83
Tabel 4. 4 Data Suhu terhadap Waktu Setelah Perbaikan.....	83
Tabel 4. 5 Perbandingan Hasil Pengujian Sebelum dan Setelah Perbaikan	84
Tabel 4. 6 Data kalibrasi sensor DS18B20 terhadap suhu referensi.....	87
Tabel 4. 7 Hasil suhu DS18B20 setelah kalibrasi.....	90
Tabel 4. 8 Hasil pengujian RTC DS3231	92
Tabel 4. 9 Hasil pengujian driver BTS7960 dan Peltier	94
Tabel 4. 10 Ringkasan hasil pengujian <i>open-loop</i>	96
Tabel 4. 11 Parameter hasil kurva reaksi proses	97
Tabel 4. 12 Hasil perhitungan awal parameter PID	101
Tabel 4. 13 Hasil pengujian <i>closed-loop</i> PID <i>setpoint</i> 15 °C	104
Tabel 4. 14 Hasil pengujian <i>closed-loop</i> PID <i>setpoint</i> 8 °C	106
Tabel 4. 15 Hasil pengujian <i>closed-loop</i> PID <i>setpoint</i> 5 °C	109
Tabel 4. 16 Perbandingan hasil pengujian <i>closed-loop</i> PID	110
Tabel 4. 17 Perbandingan pH dan TDS hasil ekstraksi menggunakan sistem pendingin otomatis dan kulkas biasa	113
Tabel 4. 18 Rata-rata pH dan TDS hasil ekstraksi.....	113

BAB I

PENDAHULUAN

1.1 Latar Belakang

Suhu merupakan salah satu parameter fisik yang banyak dikendalikan pada sistem proses karena berpengaruh terhadap kestabilan kondisi kerja, laju perpindahan panas, dan hasil akhir suatu proses. Pada sistem yang melibatkan proses pendinginan, suhu perlu dijaga agar berada pada nilai yang telah ditentukan. Apabila suhu tidak dikendalikan, proses dapat berlangsung pada kondisi termal yang berubah-ubah sehingga hasil pengujian sulit diulang pada kondisi yang sama.

Pengendalian suhu dapat dilakukan menggunakan sistem kendali tertutup. Sistem kendali tertutup bekerja dengan membaca suhu aktual melalui sensor, membandingkan nilai tersebut dengan suhu target atau *setpoint*, kemudian memberikan aksi kendali kepada aktuator. Pada sistem pendingin, aksi kendali digunakan untuk mengatur besar daya pendinginan agar suhu aktual dapat mencapai dan dipertahankan di sekitar *setpoint*. Sistem kendali tertutup lebih sesuai dibandingkan pengoperasian daya tetap karena aktuator dapat menyesuaikan keluaran berdasarkan perubahan suhu aktual [1].

Salah satu aktuator pendingin yang dapat digunakan pada sistem berskala kecil adalah modul termoelektrik Peltier. Modul Peltier bekerja berdasarkan efek termoelektrik, yaitu perpindahan panas yang terjadi ketika modul dialiri arus listrik. Satu sisi modul menyerap panas dan menjadi dingin, sedangkan sisi lainnya melepaskan panas dan menjadi panas. Modul ini memiliki ukuran relatif ringkas, tidak menggunakan refrigeran, dan dapat dikendalikan melalui pengaturan daya listrik. Kemampuan pendinginan Peltier dipengaruhi oleh arus masukan, beban termal, suhu lingkungan, isolasi ruang pendingin, serta kemampuan pelepasan panas pada sisi panas modul [2].

Pengoperasian Peltier secara langsung dengan daya tetap belum mampu menyesuaikan kebutuhan pendinginan ketika suhu mendekati *setpoint*. Jika daya pendinginan terlalu besar, suhu dapat turun melewati target. Jika daya pendinginan terlalu kecil, sistem membutuhkan waktu lebih lama untuk mencapai suhu yang ditentukan. Kondisi tersebut menunjukkan bahwa sistem pendingin berbasis Peltier

memerlukan metode pengendalian agar keluaran aktuator dapat diatur sesuai dengan perubahan suhu aktual.

Kontroler *Proportional–Integral–Derivative* atau PID merupakan salah satu metode kendali yang banyak digunakan pada sistem kendali suhu. Kontroler PID bekerja berdasarkan tiga aksi kendali, yaitu aksi proporsional, integral, dan derivatif. Aksi proporsional memberikan respons berdasarkan besar kesalahan saat ini, aksi integral digunakan untuk mengurangi kesalahan keadaan tunak, sedangkan aksi derivatif merespons laju perubahan kesalahan. Kombinasi ketiga aksi tersebut dapat digunakan untuk memperbaiki respons sistem apabila parameter kontroler ditentukan sesuai dengan karakteristik sistem [1].

Penentuan parameter PID memerlukan data respons dinamis sistem. Salah satu metode yang dapat digunakan adalah metode Ziegler–Nichols kurva reaksi proses. Metode ini menentukan parameter PID berdasarkan hasil pengujian *open-loop* melalui nilai gain proses, waktu tunda, dan konstanta waktu. Pada sistem pendingin, pengujian *open-loop* dilakukan dengan memberikan perubahan nilai PWM kepada aktuator, kemudian mencatat perubahan suhu terhadap waktu. Data tersebut digunakan untuk menentukan parameter awal PID sebelum sistem diuji secara *closed-loop* [3], [1].

Sistem pendingin otomatis dapat diterapkan pada proses yang membutuhkan suhu rendah, salah satunya proses ekstraksi *cold brew coffee*. *Cold brew coffee* merupakan metode ekstraksi kopi menggunakan air bersuhu rendah dengan waktu perendaman yang relatif lama. Suhu, waktu ekstraksi, ukuran partikel, rasio kopi dan air, serta karakteristik bahan baku dapat memengaruhi jumlah senyawa terlarut dan sifat fisikokimia hasil ekstraksi. Penelitian sebelumnya menunjukkan bahwa suhu dan waktu ekstraksi berpengaruh terhadap nilai *Total Dissolved Solids* (TDS), *extraction yield*, pH, keasaman, dan karakteristik hasil seduhan *cold brew coffee* [4].

Cold brew coffee dipilih sebagai objek penerapan karena proses ekstraksinya berbeda dengan penyeduhan kopi panas. Proses *cold brew* dilakukan menggunakan air bersuhu rendah dengan waktu kontak antara kopi dan air yang lebih lama. Kondisi tersebut menyebabkan suhu dan durasi ekstraksi menjadi parameter penting yang perlu dijaga agar proses berlangsung lebih terukur. Perubahan suhu

selama proses ekstraksi dapat memengaruhi jumlah senyawa terlarut, nilai Total Dissolved Solids (TDS), pH, dan karakteristik fisikokimia hasil seduhan. Penelitian sebelumnya menunjukkan bahwa suhu, waktu ekstraksi, ukuran gilingan, dan rasio kopi terhadap air berpengaruh terhadap karakteristik hasil cold brew coffee [4].

Pemilihan *cold brew coffee* pada penelitian ini juga didasarkan pada kebutuhan proses yang memerlukan pengendalian suhu rendah dalam durasi panjang. Pendinginan menggunakan kulkas atau metode manual belum secara langsung mengatur suhu cairan ekstraksi berdasarkan nilai *setpoint*. Suhu di dalam ruang pendingin dapat berubah karena pengaruh suhu lingkungan, beban termal, dan sistem kerja alat pendingin. Kondisi tersebut membuat proses ekstraksi sulit dikendalikan secara presisi dan sulit diulang pada kondisi suhu yang sama.

Berdasarkan alasan tersebut, *cold brew coffee* digunakan sebagai objek penerapan sistem pendingin otomatis berbasis PID. Sistem ini dirancang untuk membaca suhu aktual, membandingkannya dengan *setpoint*, lalu mengatur daya pendinginan Peltier melalui sinyal PWM. Dengan pengendalian tersebut, suhu ekstraksi dapat dipertahankan lebih stabil selama proses berlangsung. Fokus utama penelitian tetap berada pada kinerja sistem pendingin otomatis, sedangkan *cold brew coffee* digunakan sebagai media penerapan yang membutuhkan suhu rendah dan durasi ekstraksi tertentu.

Pada proses manual, suhu ekstraksi *cold brew coffee* sulit dijaga secara konsisten karena dipengaruhi oleh suhu lingkungan, media pendingin, durasi proses, dan kondisi alat yang digunakan. Perbedaan suhu selama proses ekstraksi dapat menyebabkan kondisi pengujian antarpercobaan tidak sama. Kondisi tersebut membuat proses ekstraksi sulit diulang secara terukur, terutama ketika pengujian membutuhkan beberapa variasi suhu dan durasi.

Berdasarkan permasalahan tersebut, penelitian ini merancang sistem pendingin otomatis dengan kontroler PID untuk ekstraksi *cold brew coffee*. Sistem menggunakan ESP32 sebagai pengendali utama, sensor DS18B20 sebagai pembaca suhu, modul Peltier sebagai aktuator pendingin, driver BTS7960 sebagai pengatur daya, RTC DS3231 sebagai acuan waktu, serta LCD dan tombol sebagai antarmuka pengguna. Kontroler PID digunakan untuk mengatur nilai PWM yang diberikan

kepada driver BTS7960 sehingga daya pendinginan Peltier dapat disesuaikan dengan kondisi suhu aktual.

Penelitian ini berfokus pada perancangan dan pengujian kinerja sistem pendingin otomatis. Parameter utama yang dianalisis meliputi respons *open-loop*, perhitungan parameter PID menggunakan metode Ziegler–Nichols, waktu mencapai *setpoint*, *overshoot*, *steady-state error*, fluktuasi suhu, dan kemampuan sistem mempertahankan suhu pada beberapa nilai *setpoint*. Pengujian pH dan TDS digunakan sebagai data pendukung penerapan alat pada ekstraksi *cold brew coffee*, bukan sebagai fokus utama penilaian sistem.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana merancang dan membangun sistem pendingin otomatis menggunakan modul Peltier sebagai aktuator pendingin?
2. Bagaimana memperoleh model matematis sistem pendingin berdasarkan hubungan antara masukan PWM dan perubahan suhu melalui pengujian *open-loop*?
3. Bagaimana menentukan dan menerapkan parameter kontroler PID sesuai dengan karakteristik dinamis sistem pendingin?
4. Bagaimana kinerja sistem *closed-loop* dengan kontroler PID dalam mencapai dan mempertahankan suhu sesuai nilai *setpoint* ekstraksi *cold brew*?

1.3 Batasan masalah

Agar penelitian terarah, terukur, dan tidak meluas dari tujuan yang telah ditetapkan, ruang lingkup penelitian dibatasi sebagai berikut:

1. Penelitian dibatasi pada perancangan sistem pendingin otomatis untuk proses ekstraksi *cold brew coffee*.
2. Pengendalian suhu dilakukan menggunakan kontroler PID dengan parameter yang ditentukan melalui metode Ziegler–Nichols.
3. Identifikasi sistem dilakukan berdasarkan hubungan antara masukan PWM dan respons suhu pada pengujian *open-loop*.

4. Variabel yang dikendalikan dibatasi pada suhu *box* pendingin, sedangkan variabel manipulasi berupa nilai PWM aktuator pendingin.
5. Pengujian kinerja kontroler dibatasi pada kemampuan sistem mencapai dan mempertahankan suhu pada nilai *setpoint* dari 2–20 °C.
6. Analisis kinerja sistem meliputi waktu mencapai *setpoint*, kesalahan keadaan tunak, suhu minimum, rentang fluktuasi, dan kestabilan suhu.
7. Kondisi pengujian dilakukan pada volume cairan, konstruksi ruang pendingin, jumlah aktuator, dan kondisi lingkungan yang digunakan selama penelitian.
8. Penelitian tidak membahas optimasi metode kontrol selain PID maupun perbandingan dengan metode kontrol lainnya.

1.4 Tujuan

Penelitian ini bertujuan untuk merancang dan membangun sistem kendali suhu dan waktu pada proses ekstraksi *cold brew coffee* menggunakan metode *Proportional–Integral–Derivative* berbasis ESP32. Sistem dirancang untuk menurunkan dan mempertahankan suhu ekstraksi mendekati nilai *setpoint*, serta mengatur durasi proses secara otomatis agar kondisi ekstraksi lebih terukur dan konsisten.

Secara khusus, tujuan penelitian ini adalah sebagai berikut:

1. Merancang dan membangun sistem pendingin otomatis.
2. Menerapkan kontroler *Proportional–Integral–Derivative* (PID) untuk mengatur daya pendinginan menggunakan thermoelectric Peltier agar suhu ekstraksi dapat mencapai dan mempertahankan nilai *setpoint* yang telah ditentukan.
3. Mengetahui kinerja sistem pendingin berdasarkan waktu pencapaian *setpoint*, nilai *overshoot*, kesalahan keadaan tunak (*steady-state error*), dan kestabilan suhu selama proses ekstraksi.
4. Menerapkan pengaturan waktu ekstraksi secara otomatis sesuai dengan suhu dan durasi ekstraksi yang telah ditentukan.

1.5 Manfaat

1. Menyediakan solusi pendinginan otomatis untuk proses ekstraksi *cold brew coffee* agar suhu proses dapat dikendalikan secara lebih stabil dan terukur dibandingkan pendinginan manual.
2. Mengurangi ketergantungan terhadap pemantauan manual karena sistem dapat membaca suhu aktual, menghitung kesalahan suhu, dan mengatur daya pendinginan secara otomatis melalui keluaran PWM.
3. Meningkatkan konsistensi proses pengujian karena suhu dan durasi ekstraksi dapat diatur berdasarkan nilai *setpoint* yang telah ditentukan.
4. Menjadi media penerapan teknologi sistem kendali, sensor suhu, aktuator Peltier, elektronika daya, dan mikrokontroler dalam bentuk alat terapan yang sesuai dengan kompetensi Program Studi DIV Teknik Elektronika.
5. Menjadi dasar pengembangan produk unggulan Program Studi DIV Teknik Elektronika berupa sistem pendingin otomatis berskala kecil yang dapat dikembangkan untuk kebutuhan laboratorium, industri minuman skala kecil, dan penelitian berbasis pengendalian suhu rendah.
6. Menyediakan data kinerja sistem berupa respons *open-loop*, parameter PID, waktu mencapai *setpoint*, *overshoot*, *steady-state error*, fluktuasi suhu, sebagai acuan untuk pengembangan alat pendingin otomatis berikutnya.

1.6 Sistematika Penulisan

Sistematika penulisan laporan tugas akhir ini disusun untuk memberikan gambaran mengenai isi dan pembahasan pada setiap bab. Laporan tugas akhir terdiri atas lima bab utama yang disusun secara berurutan mulai dari pendahuluan hingga kesimpulan dan saran. Adapun sistematika penulisan laporan tugas akhir ini dijelaskan sebagai berikut.

BAB I Pendahuluan

Membahas dasar pelaksanaan penelitian yang meliputi latar belakang, rumusan masalah, batasan masalah, tujuan, manfaat, dan sistematika penulisan. Latar belakang menjelaskan permasalahan konsistensi suhu dan waktu dalam proses ekstraksi *cold brew coffee*, penggunaan Peltier sebagai aktuator pendingin, serta penerapan metode PID sebagai pengendali suhu. Rumusan masalah, batasan

masalah, dan tujuan digunakan untuk menentukan arah serta ruang lingkup penelitian agar pelaksanaannya tetap terukur.

BAB II Tinjauan Pustaka

Membahas teori dan penelitian terdahulu yang berkaitan dengan sistem yang dirancang. Pembahasan meliputi proses ekstraksi *cold brew coffee*, faktor-faktor yang memengaruhi hasil ekstraksi, penelitian terkait sistem pendingin Peltier, sistem kendali tertutup, metode PID, PWM, serta parameter respons sistem kendali. Pada bab ini juga dibahas prinsip kerja, spesifikasi, alasan pemilihan, dan fungsi komponen utama, yaitu ESP32, sensor DS18B20, RTC DS3231, modul Peltier, rangkaian penggerak daya, *heatsink*, kipas, LCD, *push button*, dan buzzer.

BAB III Metodologi

Menjelaskan tahapan pelaksanaan penelitian mulai dari studi literatur, penentuan spesifikasi, perancangan, pembuatan, hingga pengujian sistem. Bab ini mencakup perancangan konstruksi alat, perancangan perangkat keras, perancangan perangkat lunak, diagram blok, diagram alir, prinsip kerja sistem, dan perancangan sistem pendingin Peltier. Selain itu, bab ini menjelaskan metode pengujian karakteristik termal, proses identifikasi sistem, penentuan parameter K_p , K_i , dan K_d , prosedur *tuning* PID, serta teknik pengambilan dan pengolahan data.

BAB IV Hasil dan Pembahasan

Menyajikan data yang diperoleh dari proses pembuatan dan pengujian alat. Hasil pengujian meliputi kalibrasi sensor DS18B20, pengujian RTC DS3231, pengujian rangkaian penggerak, pengujian kemampuan pendinginan Peltier, pengujian respons sistem tanpa pengendali, serta hasil *tuning* dan pengujian pengendali PID. Data tersebut dianalisis berdasarkan waktu pencapaian *setpoint*, *overshoot*, *settling time*, *steady-state error*, fluktuasi suhu, ketepatan waktu ekstraksi, dan konsistensi hasil pengujian berulang.

BAB V Kesimpulan dan Saran

Memuat kesimpulan yang diperoleh berdasarkan hasil pengujian dan pembahasan. Kesimpulan disusun untuk menjawab rumusan masalah serta menunjukkan tingkat ketercapaian tujuan penelitian. Saran berisi rekomendasi perbaikan dan pengembangan sistem, seperti peningkatan kapasitas pendinginan, penambahan

pencatatan data, pemantauan berbasis Internet of Things, pengukuran konsumsi energi, dan pengujian kualitas hasil ekstraksi.

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terkait

Penelitian mengenai pengendalian suhu pada sistem pendingin termoelektrik telah dilakukan oleh Kherkhar *et al.* dengan menggunakan modul *Thermoelectric Cooler* (TEC), Arduino, dan kontroler PID. Sistem dirancang dalam bentuk kendali *closed-loop* dengan mengatur arus yang diberikan kepada modul termoelektrik berdasarkan selisih antara suhu aktual dan nilai *setpoint*. Pada pengujian dengan target suhu 5 °C, sistem mampu mempertahankan suhu dengan penyimpangan sekitar $\pm 0,1$ °C. Penelitian tersebut menunjukkan bahwa kontroler PID dapat digunakan untuk mengurangi kesalahan suhu dan menjaga kestabilan sistem pendingin berbasis efek Peltier [2].

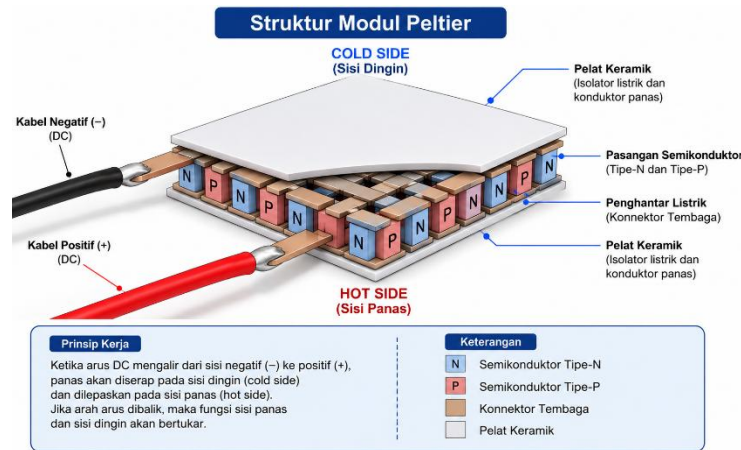
Penelitian lain dilakukan oleh Fitriyanto *et al.* mengenai penerapan kontroler PID untuk stabilisasi suhu alat pendingin termoelektrik. Sistem tersebut menggunakan Arduino Nano sebagai pengendali, sensor DS18B20 sebagai umpan balik suhu, dan modul Peltier yang dikendalikan melalui sinyal *Pulse Width Modulation* (PWM). Penalaan parameter PID dilakukan menggunakan metode *trial and error*. Hasil pengujian memperoleh parameter $K_p = 30$, $K_i = 0,6$, dan $K_d = 10$. Sistem mampu menurunkan suhu dari sekitar 29,2 °C menuju *setpoint* 12 °C dalam waktu kurang lebih 3 menit serta mempertahankan suhu dalam rentang ± 1 °C [5].

Selain sistem pendingin, pengaruh suhu terhadap proses ekstraksi *cold brew coffee* telah diteliti oleh Wang dan Lim. Penelitian tersebut menguji proses ekstraksi metode perendaman pada rentang suhu 4–37 °C, waktu ekstraksi 5 menit hingga 18 jam, serta beberapa variasi ukuran gilingan dan rasio kopi terhadap air. Hasil penelitian menunjukkan bahwa peningkatan suhu ekstraksi dapat meningkatkan laju dan hasil ekstraksi. Nilai *Total Dissolved Solids* (TDS) juga memiliki hubungan yang kuat dengan beberapa karakteristik fisikokimia kopi, seperti kandungan fenol, kafeina, dan keasaman. Hal tersebut menunjukkan bahwa suhu merupakan salah satu parameter penting yang harus dijaga selama proses ekstraksi *cold brew coffee* [6].

Berdasarkan penelitian terdahulu tersebut, kontroler PID telah terbukti dapat diterapkan untuk mengatur suhu sistem pendingin termoelektrik. Penelitian pengendalian suhu termoelektrik yang dibahas belum secara khusus diterapkan untuk mengatur beberapa variasi suhu dan durasi pada proses ekstraksi *cold brew coffee*. Selain itu, penelitian Fitriyanto *et al.* masih menggunakan metode *trial and error* dalam menentukan parameter PID, sedangkan penelitian Wang dan Lim berfokus pada karakteristik ekstraksi kopi tanpa merancang sistem pendingin otomatis. Tugas akhir ini merancang sistem pendingin otomatis berbasis termoelektrik Peltier dengan kontroler PID yang parameter kendalinya ditentukan menggunakan metode Ziegler–Nichols. Sistem dirancang untuk mencapai dan mempertahankan beberapa nilai *setpoint* sesuai kebutuhan ekstraksi serta menganalisis kinerja pengendalian berdasarkan waktu pencapaian *setpoint*, *overshoot*, kesalahan keadaan tunak, dan fluktuasi suhu. Pengukuran pH dan TDS digunakan sebagai pengujian pendukung untuk mengetahui hasil ekstraksi pada variasi suhu dan durasi yang diterapkan.

2.2 Sistem Pendingin Termoelektrik

Sistem pendingin berfungsi memindahkan kalor dari suatu media menuju lingkungan sehingga suhu media tersebut menurun. Tugas akhir ini menggunakan teknologi termoelektrik sebagai sumber pendinginan. Pendingin termoelektrik atau *Thermoelectric Cooler* (TEC) merupakan perangkat semikonduktor yang menghasilkan perbedaan suhu saat menerima aliran arus listrik. Fenomena tersebut dikenal sebagai efek Peltier. Satu sisi modul menyerap kalor dan menjadi dingin, sedangkan sisi lainnya melepaskan kalor dan menjadi panas [2]. Bentuk modul Peltier yang digunakan sebagai aktuator pendingin ditunjukkan pada Gambar 2.2.



Gambar 2. 1 Peltier termoelektrik

Modul termoelektrik tersusun atas pasangan semikonduktor tipe-P dan tipe-N. Pasangan semikonduktor tersebut dirangkai secara seri pada sisi listrik dan secara paralel pada sisi termal. Dua pelat keramik mengapit susunan semikonduktor sebagai isolator listrik sekaligus penghantar kalor. Arus searah yang mengalir melalui semikonduktor menyebabkan perpindahan kalor dari satu sisi menuju sisi lainnya. Pembalikan polaritas arus akan menukar posisi sisi dingin dan sisi panas.

Modul yang digunakan dalam rancangan ini adalah Peltier tipe TEC1-12706. Modul tersebut merupakan Peltier satu tingkat dengan ukuran sekitar 40 mm × 40 mm dan terdiri atas 127 pasangan termoelektrik. Spesifikasi utama TEC1-12706 berdasarkan lembar data Thermonamic disajikan pada Tabel 2.1.

Tabel 2. 1 Tabel Spesifikasi Peltier

No.	Parameter	Simbol	Nilai
1	Tipe modul	–	Single stage
2	Jumlah pasangan termoelektrik	–	127 pasangan
3	Dimensi modul	–	40 mm × 40 mm
4	Ketebalan modul	–	3,8 ± 0,1 mm
5	Perbedaan temperatur maksimum pada $T_h = 27^\circ\text{C}$	$\Delta T_{\max}$	70 °C
6	Tegangan maksimum pada $T_h = 27^\circ\text{C}$	$V_{\max}$	16,0 V
7	Arus maksimum pada $T_h = 27^\circ\text{C}$	$I_{\max}$	6,1 A

No.	Parameter	Simbol	Nilai
8	Kapasitas pendinginan maksimum pada $\Delta T = 0^{\circ}\text{C}$	Q_{cmax}	61,4 W
9	Resistansi listrik	R	2,0 Ω
10	Jenis sumber listrik	—	Arus searah atau DC

Berdasarkan Tabel 2.3, nilai ΔT_{max} sebesar 70°C tidak menunjukkan bahwa suhu *box* secara langsung diturunkan sebesar 70°C . Nilai tersebut diperoleh pada kondisi kapasitas pendinginan sisi dingin mendekati nol dan kondisi pengujian tertentu. Ketika Peltier menerima beban berupa cairan, wadah, panas lingkungan, dan hambatan termal, perbedaan temperatur yang dicapai akan lebih kecil. Karena itu, kemampuan pendinginan aktual harus ditentukan melalui pengujian langsung pada prototipe.

Sisi dingin modul Peltier ditempatkan pada bagian yang berhubungan dengan media pendingin. Sisi tersebut menyerap kalor dari media ekstraksi. Kalor kemudian berpindah menuju sisi panas modul. Sisi panas melepaskan kalor yang berasal dari media ekstraksi dan energi listrik yang digunakan oleh modul. Proses pelepasan kalor memerlukan *heat sink* dan kipas agar panas tidak terakumulasi. Peningkatan suhu pada sisi panas dapat menurunkan selisih suhu antara kedua sisi dan mengurangi kemampuan pendinginan modul [2].

Kemampuan pendinginan termoelektrik dipengaruhi oleh arus listrik, tegangan masukan, suhu lingkungan, beban termal, kualitas isolasi, dan kemampuan pembuangan panas. Sistem pendingin dengan isolasi yang kurang baik akan menerima kalor dari lingkungan secara terus-menerus. Kondisi tersebut meningkatkan beban kerja modul Peltier dan memperpanjang waktu penurunan suhu.

Kinerja pendingin termoelektrik dapat dinilai melalui nilai *Coefficient of Performance* (COP). Nilai COP menunjukkan perbandingan antara kalor yang diserap pada sisi dingin dan daya listrik yang digunakan oleh sistem. Persamaan COP ditunjukkan pada Persamaan (2.1) [2].

$$COP = \frac{Q_c}{P} \quad (2.1)$$

Keterangan:

COP = koefisien performa sistem pendingin

Q_c = kalor yang diserap pada sisi dingin (W)

P = daya listrik yang digunakan modul termoelektrik (W)

Pendingin termoelektrik memiliki beberapa kelebihan, seperti ukuran yang relatif ringkas, tidak menggunakan refrigeran, tidak memiliki komponen mekanis yang bergerak, dan dapat dikendalikan melalui perubahan arus listrik. Pendingin termoelektrik juga memiliki respons suhu yang relatif lambat. Kapasitas termal media, beban pendinginan, suhu lingkungan, dan sistem pembuangan panas memengaruhi kecepatan penurunan suhu.

Pengendalian *on-off* hanya mengatur aktuator dalam kondisi hidup atau mati. Metode tersebut dapat menimbulkan fluktuasi suhu di sekitar nilai *setpoint*. Modul Peltier masih menyerap kalor setelah sistem memutus daya karena terdapat energi termal yang tersimpan pada komponen pendingin. Pengaturan daya menggunakan sinyal *Pulse Width Modulation* (PWM) dapat mengurangi perubahan daya yang terlalu mendadak. Kontroler PID mengatur nilai PWM berdasarkan selisih antara suhu aktual dan suhu *setpoint*. Penelitian Kherkhar *et al.* menunjukkan bahwa kontroler PID dapat mengatur masukan sistem pendingin termoelektrik dan mempertahankan suhu mendekati nilai target [2].

Tugas akhir ini menggunakan empat modul Peltier sebagai aktuator pendinginan. Sisi dingin modul menyerap kalor dari ruang pendingin yang berisi media ekstraksi *cold brew coffee*. Sisi panas modul dilengkapi dengan *heat sink* dan kipas untuk membuang kalor ke lingkungan. ESP32 mengatur daya modul Peltier melalui driver BTS7960 menggunakan sinyal PWM. Kontroler PID menentukan nilai PWM berdasarkan selisih antara suhu aktual yang dibaca sensor DS18B20 dan nilai *setpoint*. Sistem tersebut dirancang untuk mencapai dan mempertahankan suhu ekstraksi sesuai dengan nilai yang telah ditentukan.

2.3 Sistem Kendali Suhu

Sistem kendali suhu merupakan sistem yang digunakan untuk mengatur suhu suatu media agar berada pada nilai yang telah ditentukan. Pada sistem pendingin otomatis, suhu yang dikendalikan disebut variabel proses, sedangkan nilai suhu yang ingin dicapai disebut *setpoint*. Sistem kendali bekerja dengan membaca suhu aktual, membandingkannya dengan *setpoint*, lalu memberikan aksi kendali kepada aktuator pendingin. Tujuan utama sistem kendali suhu adalah menjaga suhu tetap stabil meskipun terdapat perubahan beban termal atau pengaruh suhu lingkungan.

Sistem kendali dapat dibedakan menjadi sistem kendali terbuka dan sistem kendali tertutup. Sistem kendali terbuka memberikan keluaran tanpa memperhatikan hasil pengukuran dari sistem. Cara tersebut kurang sesuai untuk sistem pendingin yang memiliki perubahan suhu lambat dan dipengaruhi oleh kondisi lingkungan. Sistem kendali tertutup menggunakan umpan balik dari sensor untuk mengetahui kondisi aktual sistem. Nilai umpan balik tersebut digunakan oleh pengendali untuk menentukan besar aksi kendali yang harus diberikan kepada aktuator [1].

Pada tugas akhir ini, sistem kendali suhu menggunakan prinsip kendali tertutup. Sensor DS18B20 membaca suhu aktual pada media ekstraksi *cold brew coffee*. ESP32 menerima data suhu tersebut, lalu membandingkannya dengan nilai *setpoint* yang telah ditentukan. Selisih antara suhu aktual dan *setpoint* digunakan sebagai masukan bagi kontroler PID. Keluaran kontroler berupa nilai PWM yang dikirimkan ke driver BTS7960 untuk mengatur daya kerja modul Peltier.

Besarnya kesalahan suhu pada sistem pendingin dapat dinyatakan pada Persamaan (2.2).

$$e(t) = T_{aktual}(t) - T_{setpoint}(t) \quad (2.2)$$

Keterangan:

$e(t)$ = kesalahan suhu pada waktu tertentu (°C)

$T_{aktual}(t)$ = suhu aktual yang terbaca sensor (°C)

$T_{setpoint}(t)$ = suhu target yang ditentukan (°C)

Persamaan tersebut digunakan karena sistem yang dirancang bekerja sebagai sistem pendingin. Saat suhu aktual lebih tinggi daripada *setpoint*, nilai kesalahan

bernilai positif sehingga daya pendinginan perlu ditingkatkan. Saat suhu aktual mendekati *setpoint*, daya pendinginan dikurangi agar suhu tidak turun terlalu jauh. Saat suhu aktual lebih rendah daripada *setpoint*, keluaran pendinginan dikurangi atau dihentikan untuk mengurangi *overshoot*.

Karakteristik sistem pendingin termoelektrik cenderung memiliki respons lambat karena perubahan suhu dipengaruhi oleh kapasitas termal media, kemampuan pelepasan panas pada sisi panas Peltier, isolasi ruang pendingin, dan suhu lingkungan. Pengendalian secara langsung tanpa umpan balik dapat menyebabkan suhu tidak stabil atau melewati nilai *setpoint*. Sistem kendali tertutup digunakan agar aktuator dapat menyesuaikan daya pendinginan berdasarkan kondisi suhu aktual.

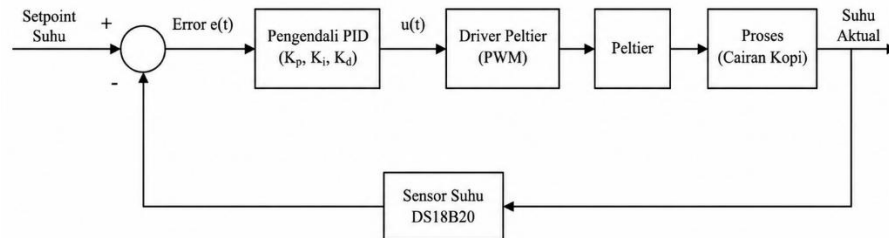
Pada tugas akhir ini, sistem kendali suhu diterapkan untuk menjaga suhu ekstraksi pada beberapa variasi *setpoint*, yaitu 5 °C, 8 °C, 12 °C, 15 °C, dan 20 °C. Kinerja sistem kendali dianalisis berdasarkan waktu pencapaian *setpoint*, *overshoot*, kesalahan keadaan tunak, dan fluktuasi suhu selama proses ekstraksi. Pengujian tersebut digunakan untuk mengetahui kemampuan sistem pendingin otomatis dalam mempertahankan suhu sesuai kebutuhan proses ekstraksi *cold brew coffee*.

2.4 Kontroler Proportional–Integral–Derivative (PID)

Pengendali *Proportional–Integral–Derivative* atau PID merupakan metode kendali *loop* tertutup yang menggunakan perbedaan antara nilai acuan dan nilai aktual sebagai dasar untuk menentukan keluaran pengendalian. Nilai aktual diperoleh dari sensor dan dikirimkan kembali kepada pengendali sebagai sinyal umpan balik. Selisih antara nilai acuan atau *setpoint* dan nilai aktual disebut *error*. Pengendali PID kemudian mengolah nilai *error* tersebut melalui komponen proporsional, integral, dan derivatif agar keluaran sistem mendekati nilai yang diinginkan [1].

Pua *et al.* menjelaskan bahwa pengendali PID terdiri atas tiga komponen utama, yaitu proporsional, integral, dan derivatif. Masing-masing komponen memiliki karakteristik yang berbeda dan saling melengkapi dalam membentuk respons sistem. Dalam perancangan pengendali PID, parameter K_p , K_i , dan K_d perlu

diatur agar respons keluaran sesuai dengan karakteristik sistem yang dikendalikan [10]. Konfigurasi pengendali PID pada sistem pendingin ekstraksi *cold brew coffee* ditunjukkan pada Gambar 2.2.



Gambar 2. 2 Gambar Blok Diagram PID

Aksi proporsional bekerja berdasarkan besar kesalahan suhu pada saat tertentu. Semakin besar selisih antara suhu aktual dan *setpoint*, semakin besar keluaran kendali yang diberikan kepada aktuator. Pada sistem pendingin, kondisi suhu aktual yang masih lebih tinggi dari *setpoint* menyebabkan nilai PWM meningkat sehingga daya pendinginan bertambah. Aksi proporsional dapat mempercepat respons sistem, tetapi nilai penguatan yang terlalu besar dapat menyebabkan suhu melewati *setpoint* dan menimbulkan osilasi.

Aksi integral bekerja berdasarkan akumulasi kesalahan terhadap waktu. Komponen integral digunakan untuk mengurangi kesalahan keadaan tunak atau *steady-state error*. Pada sistem pendingin, kesalahan kecil yang terjadi secara terus-menerus akan dijumlahkan oleh komponen integral. Hasil akumulasi tersebut digunakan untuk menambah atau mengurangi aksi kendali agar suhu dapat mendekati *setpoint*. Nilai integral yang terlalu besar dapat menyebabkan respons sistem menjadi lambat kembali stabil dan meningkatkan kemungkinan *overshoot*.

Aksi derivatif bekerja berdasarkan laju perubahan kesalahan. Komponen derivatif berfungsi memperkirakan arah perubahan suhu sehingga sistem dapat meredam perubahan yang terlalu cepat. Pada sistem pendingin, aksi derivatif membantu mengurangi kemungkinan suhu turun terlalu jauh melewati *setpoint*. Nilai derivatif yang sesuai dapat memperbaiki kestabilan sistem, sedangkan nilai derivatif yang terlalu besar dapat membuat keluaran kendali sensitif terhadap gangguan pembacaan sensor.

Bentuk umum persamaan kontroler PID ditunjukkan pada Persamaan (2.3).

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (2.3)$$

Keterangan:

$u(t)$ = keluaran kontroler

K_p = konstanta proporsional

K_i = konstanta integral

K_d = konstanta derivatif

$e(t)$ = kesalahan suhu pada waktu tertentu

$\int e(t) dt$ = akumulasi kesalahan terhadap waktu

$\frac{de(t)}{dt}$ = laju perubahan kesalahan terhadap waktu

Pada implementasi mikrokontroler, persamaan PID dihitung secara diskrit karena pembacaan sensor dilakukan dalam interval waktu tertentu. Bentuk diskrit kontroler PID dapat ditulis seperti Persamaan (2.4).

$$u(k) = K_p e(k) + K_i \sum e(k) \Delta t + K_d \frac{e(k) - e(k-1)}{\Delta t} \quad (2.4)$$

Keterangan:

$u(k)$ = keluaran kontroler pada pembacaan ke- k

$e(k)$ = kesalahan suhu pada pembacaan ke- k

$e(k-1)$ = kesalahan suhu pada pembacaan sebelumnya

Δt = interval waktu pembacaan sensor

Keluaran kontroler PID pada tugas akhir ini digunakan sebagai dasar penentuan nilai PWM. Nilai PWM tersebut dikirimkan oleh ESP32 ke driver BTS7960 untuk mengatur daya modul Peltier. Nilai PWM dibatasi pada rentang 0 sampai 255. Nilai 0 menunjukkan aktuator pendingin tidak bekerja, sedangkan nilai 255 menunjukkan aktuator bekerja pada daya maksimum.

Sistem pendingin pada tugas akhir ini menggunakan arah kesalahan $e(t) = T_{aktual}(t) - T_{setpoint}(t)$. Saat suhu aktual lebih tinggi dari *setpoint*, nilai kesalahan bernilai positif dan kontroler meningkatkan nilai PWM. Saat suhu aktual mendekati *setpoint*, kontroler menurunkan nilai PWM agar suhu tidak turun terlalu jauh. Saat

suhu aktual lebih rendah dari *setpoint*, keluaran pendinginan dikurangi sampai batas minimum untuk membantu sistem kembali menuju daerah kerja yang stabil.

Kontroler PID dipilih karena sistem pendingin Peltier memiliki respons yang lambat dan memerlukan pengaturan daya secara bertahap. Pengendalian *on-off* hanya memberi aksi hidup dan mati sehingga suhu dapat berfluktuasi lebih besar di sekitar *setpoint*. Kontroler PID mampu menyesuaikan daya pendinginan berdasarkan besar kesalahan, akumulasi kesalahan, dan laju perubahan kesalahan. Kinerja kontroler pada tugas akhir ini dianalisis melalui waktu pencapaian *setpoint*, *overshoot*, kesalahan keadaan tunak, dan fluktuasi suhu selama proses ekstraksi *cold brew coffee*.

2.5 Metode Ziegler–Nichols

Metode Ziegler–Nichols merupakan metode penalaan parameter kontroler yang digunakan untuk memperoleh nilai awal K_p , K_i , dan K_d berdasarkan respons sistem. Metode ini diperkenalkan oleh Ziegler dan Nichols sebagai pendekatan praktis untuk menentukan parameter kontroler dari hasil pengujian respons proses [3]. Pada tugas akhir ini, metode Ziegler–Nichols digunakan untuk menentukan parameter kontroler PID pada sistem pendingin termoelektrik Peltier. Penalaan dilakukan menggunakan pendekatan kurva reaksi proses dengan parameter waktu tunda (L) dan konstanta waktu (T).

Metode kurva reaksi proses dilakukan dengan memberikan perubahan input secara tiba-tiba pada sistem. Pada sistem pendingin ini, perubahan input diberikan dalam bentuk perubahan nilai PWM pada driver BTS7960. Perubahan PWM menyebabkan modul Peltier bekerja dan menghasilkan perubahan suhu pada media ekstraksi. Respons suhu terhadap perubahan PWM direkam untuk memperoleh karakteristik sistem.

Sistem pendingin termoelektrik memiliki respons yang relatif lambat. Perubahan suhu tidak langsung terjadi saat PWM diberikan karena proses pendinginan dipengaruhi oleh kapasitas termal media, perpindahan kalor pada ruang pendingin, kemampuan pembuangan panas pada sisi panas Peltier, dan suhu lingkungan. Karakteristik tersebut dapat diamati melalui kurva respons suhu terhadap waktu.

Pada metode Ziegler–Nichols kurva reaksi, respons sistem didekati dengan model orde satu yang memiliki waktu tunda atau *First Order Plus Dead Time* (FOPDT). Model FOPDT umum digunakan untuk merepresentasikan sistem proses yang memiliki gain, konstanta waktu, dan waktu tunda [1]. Bentuk umum model FOPDT ditunjukkan pada Persamaan (2.5).

$$G(s) = \frac{K e^{-Ls}}{Ts + 1} \quad (2.5)$$

Keterangan:

$G(s)$ = fungsi alih pendekatan sistem

K = gain proses

L = waktu tunda atau *delay time*

T = konstanta waktu atau *time constant*

s = operator Laplace

Gain proses (K) menunjukkan perbandingan antara perubahan keluaran dan perubahan masukan. Pada tugas akhir ini, keluaran sistem berupa suhu, sedangkan masukan sistem berupa nilai PWM. Gain proses dapat dihitung menggunakan Persamaan (2.6).

$$K = \frac{\Delta y}{\Delta u} \quad (2.6)$$

Keterangan:

Δy = perubahan keluaran sistem

Δu = perubahan masukan sistem

Pada sistem pendingin, suhu menurun saat nilai PWM dinaikkan. Kondisi tersebut dapat menghasilkan nilai gain bertanda negatif. Perhitungan parameter PID menggunakan nilai mutlak gain agar nilai parameter kontroler bernilai positif.

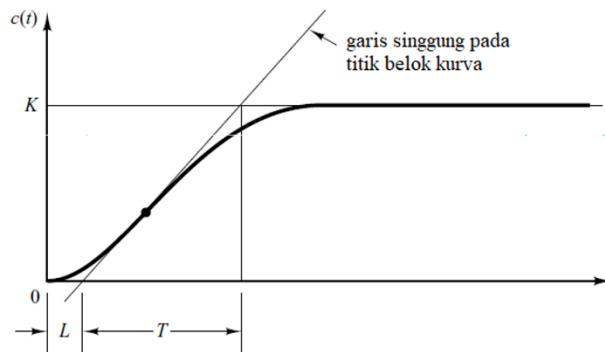
$$K = \left| \frac{\Delta T}{\Delta PWM} \right| \quad (2.7)$$

Keterangan:

ΔT = perubahan suhu dari kondisi awal menuju kondisi akhir

ΔPWM = perubahan nilai PWM yang diberikan ke driver

Nilai L dan T diperoleh dari kurva respons suhu terhadap waktu. Garis singgung dibuat pada bagian kurva yang memiliki kemiringan terbesar. Titik potong garis singgung dengan garis suhu awal menunjukkan waktu tunda (L). Selisih waktu antara titik potong garis suhu awal dan titik potong garis suhu akhir menunjukkan konstanta waktu (T). Ilustrasi penentuan nilai L dan T ditunjukkan pada Gambar 2.1.



Gambar 2. 3 Kurva reaksi proses untuk menentukan nilai L dan T [1], [3]

Tabel 2.1 Aturan penalaan Ziegler–Nichols metode kurva reaksi proses [3], [1]

Jenis Kontroler	K_p	T_i	T_d
P	$\frac{T}{KL}$	-	-
PI	$0,9 \frac{T}{KL}$	$3,33L$	-
PID	$1,2 \frac{T}{KL}$	$2L$	$0,5L$

Parameter T_i dan T_d dikonversi menjadi konstanta K_i dan K_d . Hubungan parameter tersebut ditunjukkan pada Persamaan (2.8) dan Persamaan (2.9).

$$K_i = \frac{K_p}{T_i} \quad (2.8)$$

$$K_d = K_p \times T_d \quad (2.9)$$

Pada tugas akhir ini, pengujian dilakukan dengan memberikan nilai PWM tertentu pada sistem pendingin. Sensor DS18B20 membaca perubahan suhu media ekstraksi selama proses pendinginan berlangsung. Data suhu terhadap waktu digunakan untuk membentuk kurva respons sistem. Kurva tersebut menjadi dasar untuk menentukan nilai K , L , dan T . Nilai tersebut digunakan untuk menghitung parameter kontroler PID berdasarkan aturan Ziegler–Nichols.

Parameter PID hasil penalaan diterapkan pada ESP32 untuk mengatur nilai PWM yang dikirimkan ke driver BTS7960. Driver BTS7960 mengatur daya kerja modul Peltier sesuai keluaran kontroler. Saat suhu aktual masih lebih tinggi dari *setpoint*, nilai PWM dinaikkan agar daya pendinginan bertambah. Saat suhu aktual mendekati *setpoint*, nilai PWM dikurangi agar penurunan suhu tidak melewati target secara berlebihan.

Metode Ziegler–Nichols berbasis T dan L digunakan karena parameter PID dapat ditentukan berdasarkan data respons sistem pendingin secara langsung. Hasil penalaan menjadi dasar pengujian kinerja sistem, meliputi waktu pencapaian *setpoint*, *overshoot*, kesalahan keadaan tunak, dan fluktuasi suhu selama proses ekstraksi *cold brew coffee*.

2.6 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) merupakan teknik pengaturan daya dengan mengubah lebar pulsa sinyal digital dalam satu periode tertentu. Sinyal PWM memiliki dua kondisi utama, yaitu logika tinggi dan logika rendah. Perbandingan antara waktu sinyal berada pada kondisi tinggi terhadap satu periode sinyal disebut *duty cycle*. Nilai *duty cycle* menentukan besar rata-rata daya yang diberikan kepada beban [11].

Pada sistem elektronika daya, PWM sering digunakan untuk mengatur kerja aktuator karena sinyal digital dari mikrokontroler dapat menghasilkan keluaran daya yang seolah-olah berubah secara bertahap. Mikrokontroler tidak mengubah tegangan keluaran secara analog, tetapi mengatur lama waktu sinyal berada pada kondisi aktif dan tidak aktif. Semakin besar nilai *duty cycle*, semakin besar rata-rata tegangan dan daya yang diterima oleh beban.

Hubungan antara *duty cycle*, waktu aktif, dan periode sinyal ditunjukkan pada Persamaan (2.10).

$$D = \frac{T_{on}}{T} \times 100\% \quad (2.10)$$

Keterangan:

D = *duty cycle* (%)

T_{on} = waktu sinyal aktif atau logika tinggi (s)

T = periode sinyal PWM (s)

Nilai tegangan rata-rata keluaran PWM dapat dinyatakan menggunakan Persamaan (2.11).

$$V_{rata-rata} = D \times V_{in} \quad (2.11)$$

Keterangan:

$V_{rata-rata}$ = tegangan rata-rata keluaran PWM (V)

D = *duty cycle* dalam bentuk desimal

V_{in} = tegangan masukan (V)

Pada sistem berbasis mikrokontroler, nilai PWM umumnya direpresentasikan dalam bentuk bilangan digital. Jika sistem menggunakan resolusi 8 bit, rentang nilai PWM berada dari 0 sampai 255. Nilai 0 menunjukkan *duty cycle* 0%, sedangkan nilai 255 menunjukkan *duty cycle* 100%. Hubungan nilai PWM 8 bit terhadap *duty cycle* ditunjukkan pada Tabel 2.2.

Tabel 2. 2 Hubungan nilai PWM 8 bit terhadap *duty cycle*

Nilai PWM	<i>Duty Cycle</i>	Kondisi Aktuator
0	0%	Tidak bekerja
64	25%	Daya rendah
128	50%	Daya sedang
191	75%	Daya tinggi
255	100%	Daya maksimum

Pada tugas akhir ini, PWM digunakan untuk mengatur daya kerja modul Peltier melalui driver BTS7960. ESP32 menghasilkan sinyal PWM sebagai keluaran kontroler PID. Driver BTS7960 menerima sinyal tersebut dan mengatur

besar daya yang diberikan kepada modul Peltier. Nilai PWM yang lebih besar membuat modul Peltier bekerja dengan daya pendinginan yang lebih tinggi. Nilai PWM yang lebih kecil menurunkan daya pendinginan sehingga suhu tidak turun terlalu jauh dari *setpoint*.

Penggunaan PWM pada sistem pendingin memberikan pengaturan daya yang lebih halus dibandingkan pengendalian *on-off*. Pengendalian *on-off* hanya memberi dua kondisi, yaitu aktif penuh atau mati. PWM memungkinkan aktuator menerima variasi daya sesuai kebutuhan sistem. Pada saat suhu aktual masih jauh di atas *setpoint*, kontroler PID menghasilkan nilai PWM tinggi. Pada saat suhu aktual mendekati *setpoint*, nilai PWM dikurangi untuk menjaga kestabilan suhu.

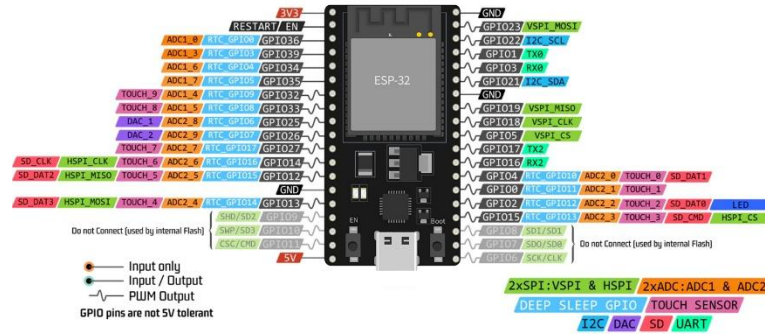
Pada sistem ini, nilai PWM dibatasi pada rentang 0 sampai 255. Batas tersebut digunakan agar keluaran kontroler PID tetap sesuai dengan kemampuan pembangkitan sinyal PWM pada mikrokontroler dan kebutuhan pengendalian driver. Pembatasan nilai PWM juga membantu mencegah keluaran kontroler melebihi rentang kerja aktuator.

2.7 ESP32

ESP32 merupakan mikrokontroler berbasis *System on Chip* yang mengintegrasikan prosesor, memori, antarmuka masukan dan keluaran, serta komunikasi nirkabel dalam satu perangkat. ESP32 dilengkapi prosesor 32 bit Xtensa LX6 dengan frekuensi kerja hingga 240 MHz, memori SRAM internal, GPIO, ADC, DAC, UART, SPI, I²C, serta perangkat pembangkit sinyal PWM. Fitur tersebut memungkinkan ESP32 digunakan untuk membaca sensor, menjalankan perhitungan pengendali PID, mengatur aktuator, dan mengelola antarmuka pengguna dalam satu sistem [12].

ESP32 berfungsi sebagai pusat pengendali sistem pendingin ekstraksi *cold brew coffee*. ESP32 membaca suhu aktual dari sensor DS18B20, menerima informasi waktu dari RTC DS3231, menghitung keluaran pengendali PID, dan menghasilkan sinyal PWM untuk driver BTS7960. Selain itu, ESP32 digunakan untuk mengendalikan LCD, membaca *push button*, dan mengaktifkan buzzer sesuai kondisi sistem.

Bentuk fisik ESP32 yang digunakan dalam penelitian ini ditunjukkan pada Gambar 2.6.



Gambar 2. 3 Gambar ESP32

Board ESP32 menyediakan pin masukan dan keluaran pada kedua sisi modul. Board pengembangan umumnya dilengkapi regulator tegangan, antarmuka USB ke serial, tombol *reset*, tombol *boot*, dan konektor USB untuk proses pemrograman. Bentuk dan jumlah pin berbeda sesuai tipe board yang digunakan.

ESP32 memiliki beberapa antarmuka komunikasi yang digunakan untuk menghubungkan komponen sistem. Sensor DS18B20 menggunakan komunikasi *I-Wire*, sedangkan RTC DS3231 dan LCD menggunakan komunikasi I²C. Sinyal kendali driver BTS7960 dihasilkan melalui GPIO yang mendukung PWM. Tombol dan buzzer dihubungkan melalui pin digital sesuai kebutuhan masukan dan keluaran.

Spesifikasi utama ESP32 yang berkaitan dengan penelitian ini disajikan pada Tabel 2.3.

Tabel 2. 3 Tabel Spesifikasi ESP32

No.	Parameter	Spesifikasi
1	Arsitektur prosesor	Xtensa LX6 32 bit
2	Jumlah inti prosesor	Hingga dua inti
3	Frekuensi maksimum	240 MHz
4	Memori ROM	448 KB
5	Memori SRAM	520 KB
6	Tegangan operasi chip	2,3–3,6 V
7	Tegangan operasi yang direkomendasikan	3,3 V

No.	Parameter	Spesifikasi
8	Komunikasi kabel	UART, SPI, I ² C, I ² S
9	Masukan dan keluaran analog	ADC dan DAC
10	Komunikasi nirkabel	Wi-Fi 2,4 GHz dan Bluetooth
11	Pembangkit PWM	Periferal LEDC
12	Timer	Timer internal dan <i>watchdog timer</i>

Berdasarkan Tabel 2.8, kemampuan pemrosesan ESP32 mencukupi untuk melakukan pembacaan sensor dan perhitungan PID secara periodik. Memori internal digunakan untuk menyimpan program, variabel pengendali, data waktu, dan status sistem. Ketersediaan beberapa antarmuka komunikasi juga memungkinkan sensor, RTC, LCD, dan perangkat lainnya diintegrasikan dalam satu mikrokontroler.

Tegangan operasi chip ESP32 berada pada rentang 2,3–3,6 V dengan tegangan yang direkomendasikan sebesar 3,3 V [12]. Hal tersebut berbeda dengan tegangan masukan board pengembangan yang menerima tegangan dari USB atau pin 5 V melalui regulator internal. Perangkat yang dihubungkan langsung ke GPIO harus menggunakan level logika yang sesuai dengan ESP32 agar tidak merusak pin mikrokontroler.

Sinyal PWM pada ESP32 dihasilkan menggunakan periferal *LED Control* atau LEDC. Periferal tersebut dikonfigurasi berdasarkan frekuensi dan resolusi *duty cycle* yang dibutuhkan oleh sistem [11]. Keluaran PWM digunakan sebagai sinyal kendali bagi driver BTS7960, sedangkan kebutuhan arus Peltier dipenuhi menggunakan sumber daya eksternal. ESP32 tidak dihubungkan secara langsung dengan Peltier karena arus keluaran GPIO tidak mencukupi untuk mengoperasikan beban tersebut.

Timer internal ESP32 digunakan untuk mengatur interval pembacaan DS18B20, perhitungan PID, pembaruan PWM, pembacaan tombol, dan pembaruan tampilan. Penggunaan timer internal memungkinkan proses tersebut dijalankan secara periodik tanpa menghentikan keseluruhan program. Sementara itu, RTC DS3231 tetap digunakan sebagai acuan waktu mulai, waktu selesai, dan durasi

ekstraksi karena informasi waktunya dipertahankan menggunakan baterai cadangan.

ESP32 dipilih karena mampu mengintegrasikan seluruh fungsi kendali yang dibutuhkan dalam penelitian ini. Kemampuan pemrosesan, antarmuka komunikasi, PWM, serta timer internal memungkinkan sistem suhu dan waktu dijalankan secara bersamaan. Meskipun ESP32 memiliki Wi-Fi dan Bluetooth, fitur komunikasi nirkabel tersebut tidak menjadi fokus utama penelitian karena sistem difokuskan pada pengendalian suhu dan waktu ekstraksi.

2.8 Sensor Suhu DS18B20

DS18B20 merupakan sensor suhu digital yang menggunakan komunikasi *1-Wire*. Sensor ini menghasilkan data temperatur dalam bentuk digital sehingga tidak memerlukan rangkaian konverter analog ke digital tambahan. DS18B20 memiliki rentang pengukuran $-55\text{ }^{\circ}\text{C}$ sampai $+125\text{ }^{\circ}\text{C}$, resolusi 9 bit hingga 12 bit, dan akurasi $\pm 0,5\text{ }^{\circ}\text{C}$ pada rentang $-10\text{ }^{\circ}\text{C}$ sampai $+85\text{ }^{\circ}\text{C}$. [13]. Bentuk Sensor DS18B20 ditunjukkan pada Gambar 2.4.



Gambar 2. 4 Gambar Sensor DS18B20

Sensor DS18B20 dalam kemasan TO-92 memiliki tiga pin, yaitu GND, DQ, dan V_{DD} . Setiap pin memiliki fungsi yang berbeda dalam proses pemberian daya dan komunikasi dengan ESP32. Konfigurasi pin sensor DS18B20 disajikan pada Tabel 2.4.

Tabel 2. 4 Tabel Konfigurasi Sensor DS18B20 [13]

No.	Nama pin	Fungsi
1	GND	Dihubungkan ke <i>ground</i> sistem
2	DQ	Jalur data komunikasi <i>1-Wire</i>
3	V_{DD}	Masukan catu daya sensor sebesar 3,0–5,5 V

Pin DQ digunakan sebagai jalur komunikasi antara DS18B20 dan ESP32. Jalur tersebut membutuhkan resistor *pull-up* sebesar 4,7 k Ω yang dihubungkan antara pin DQ dan sumber tegangan. Pin V_{DD} digunakan untuk memberikan catu daya eksternal kepada sensor, sedangkan pin GND dihubungkan dengan *ground* sistem [13].

DS18B20 dioperasikan menggunakan catu daya eksternal atau menggunakan mode *parasite power*. Digunakan catu daya eksternal agar proses konversi suhu dan komunikasi sensor lebih stabil. Sensor dioperasikan pada tegangan 3,3 V sehingga sesuai dengan level tegangan ESP32.

Sensor DS18B20 memiliki resolusi pengukuran yang diatur dari 9 bit hingga 12 bit. Resolusi menentukan perubahan suhu terkecil yang ditampilkan oleh sensor. Pada resolusi 12 bit, perubahan suhu terkecil yang dibaca adalah 0,0625°C, sedangkan waktu konversi maksimum mencapai 750 ms [13]. Spesifikasi utama sensor DS18B20 disajikan pada Tabel 2.5.

Tabel 2. 5 Tabel Spesifikasi Sensor DS18B20[13]

No.	Parameter	Spesifikasi
1	Jenis keluaran	Data suhu digital
2	Protokol komunikasi	<i>1-Wire</i>
3	Tegangan operasi	3,0–5,5 V
4	Rentang pengukuran	–55°C sampai 125°C
5	Akurasi	$\pm 0,5^\circ\text{C}$ pada -10°C sampai 85°C
6	Resolusi	9, 10, 11, atau 12 bit
7	Resolusi terkecil	0,0625°C pada 12 bit

No.	Parameter	Spesifikasi
8	Waktu konversi maksimum	750 ms pada 12 bit
9	Identitas perangkat	Kode serial 64 bit
10	Mode catu daya	Eksternal atau <i>parasite power</i>

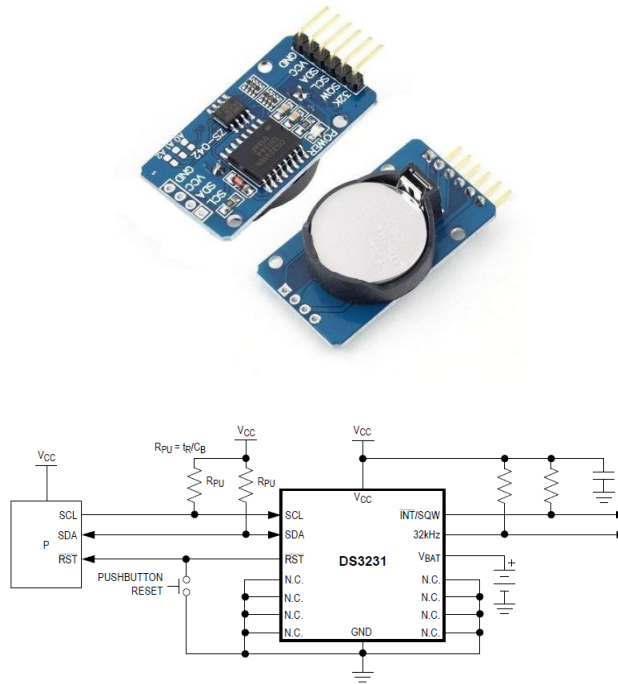
Rentang pengukuran DS18B20 mencakup suhu rendah yang digunakan dalam proses ekstraksi *cold brew coffee*. Resolusi 12 bit memungkinkan perubahan suhu diamati dengan lebih rinci, tetapi nilai resolusi tersebut tidak sama dengan akurasi sensor. Sensor tetap memiliki batas akurasi sebesar $\pm 0,5^{\circ}\text{C}$ pada rentang suhu penelitian [13].

DS18B20 digunakan sebagai sensor umpan balik pada sistem kendali PID. Sensor ditempatkan pada posisi yang mewakili suhu chamber. Posisi dan kedalaman sensor dibuat tetap pada setiap pengujian agar hasil pembacaan dibandingkan. Data suhu dari DS18B20 dibaca oleh ESP32 dan digunakan untuk menentukan besar keluaran PWM yang diberikan kepada driver BTS7960.

2.9 Real Time Clock DS3231

Real Time Clock DS3231 merupakan komponen pencatat waktu yang berkomunikasi dengan mikrokontroler melalui antarmuka I²C. DS3231 menyimpan informasi detik, menit, jam, hari, tanggal, bulan, dan tahun. Komponen ini dilengkapi *Temperature-Compensated Crystal Oscillator* (TCXO), yaitu osilator kristal yang dikompensasi terhadap perubahan temperatur untuk meningkatkan ketelitian pencatatan waktu [14]. DS3231 juga memiliki masukan baterai cadangan sehingga proses pencatatan waktu tetap berjalan ketika catu daya utama sistem terputus.

DS3231 digunakan untuk mencatat waktu mulai, menentukan waktu berakhir, dan menghitung durasi proses ekstraksi *cold brew coffee*. Penggunaan RTC membuat pencatatan waktu tidak hanya bergantung pada timer internal ESP32. Apabila ESP32 mengalami *reset*, informasi waktu aktual tetap dipertahankan oleh DS3231 selama baterai cadangan masih terpasang. Bentuk fisik modul RTC DS3231 yang digunakan ditunjukkan pada Gambar 2.5.



Gambar 2. 5 Gambar Modul RTC DS3231

Modul RTC DS3231 terdiri atas IC DS3231, baterai cadangan, dan terminal komunikasi dengan mikrokontroler. Baterai cadangan berfungsi mempertahankan pencatatan waktu ketika tegangan utama modul tidak tersedia. Pada sistem yang dirancang, komunikasi antara DS3231 dan ESP32 menggunakan terminal SDA dan SCL.

Terminal yang digunakan untuk menghubungkan modul RTC DS3231 dengan ESP32 disajikan pada Tabel 2.6.

Tabel 2. 6 Tabel Konfigurasi Modul RTC DS3231

No.	Terminal	Fungsi
1	VCC	Masukan catu daya modul
2	GND	Dihubungkan ke <i>ground</i> sistem
3	SDA	Jalur data komunikasi I ² C
4	SCL	Jalur <i>clock</i> komunikasi I ² C
5	SQW	Keluaran gelombang kotak atau sinyal interupsi alarm
6	32K	Keluaran sinyal frekuensi 32,768 kHz

Terminal SDA digunakan untuk mengirim dan menerima data waktu, sedangkan terminal SCL menyediakan sinyal sinkronisasi komunikasi I²C. Terminal SQW dan 32K tidak wajib digunakan apabila sistem hanya membutuhkan pembacaan waktu. Terminal utama yang digunakan adalah VCC, GND, SDA, dan SCL.

DS3231 menyediakan dua alarm waktu yang diprogram serta keluaran gelombang kotak. Alarm diatur berdasarkan kombinasi detik, menit, jam, hari, atau tanggal. Waktu selesai ekstraksi ditentukan melalui perbandingan antara waktu aktual RTC dengan waktu akhir yang telah dihitung oleh program ESP32. DS3231 berfungsi sebagai acuan waktu absolut, sedangkan timer internal ESP32 digunakan untuk mengatur interval pembacaan sensor, perhitungan PID, pembaruan PWM, dan tampilan.

Spesifikasi utama DS3231 disajikan pada Tabel 2.7.

Tabel 2. 7 Tabel Spesifikasi Utama RTC DS3231[14]

No.	Parameter	Spesifikasi
1	Jenis komunikasi	I ² C
2	Tegangan operasi IC	2,3–5,5 V
3	Informasi waktu	Detik, menit, jam, hari, tanggal, bulan, dan tahun
4	Format jam	12 jam atau 24 jam
5	Osilator	TCXO terintegrasi
6	Akurasi pada 0–40 °C	±2 ppm
7	Akurasi pada –40–85 °C	±3,5 ppm
8	Alarm	Dua alarm yang diprogram
9	Keluaran gelombang kotak	1 Hz, 1,024 kHz, 4,096 kHz, atau 8,192 kHz
10	Sumber daya cadangan	Baterai melalui terminal V_{BAT}

DS3231 memiliki tingkat penyimpangan waktu yang rendah karena menggunakan TCXO terintegrasi. Nilai akurasi ±2 ppm menunjukkan batas penyimpangan frekuensi osilator pada kondisi pengujian tertentu, bukan berarti

tidak terdapat kesalahan waktu sama sekali. Ketepatan durasi yang dihasilkan tetap diuji dengan membandingkan waktu RTC terhadap alat ukur waktu pembanding. Spesifikasi tersebut berasal dari lembar data resmi DS3231.

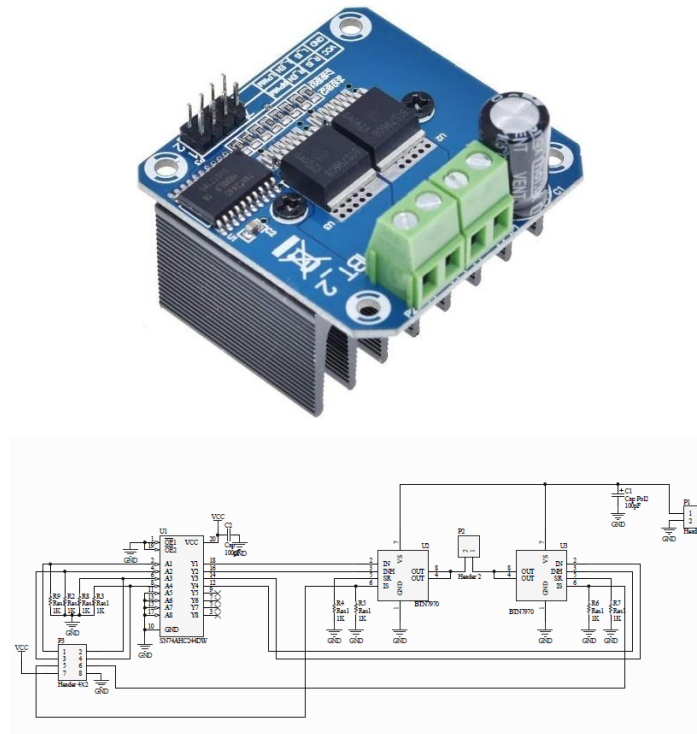
DS3231 dipilih karena mampu mempertahankan waktu ketika catu daya utama sistem terputus dan dihubungkan dengan ESP32 melalui dua jalur komunikasi I²C. Fungsi tersebut sesuai dengan kebutuhan proses ekstraksi yang berlangsung dalam waktu relatif lama. Penggunaan RTC juga memisahkan fungsi pencatatan waktu dari proses kendali suhu sehingga perhitungan PID tetap dijalankan secara periodik oleh timer internal ESP32.

2.10 Driver BTS7960

BTS7960 merupakan IC penggerak daya berupa *high-current half-bridge* yang mengintegrasikan MOSFET sisi atas, MOSFET sisi bawah, dan rangkaian pengendali dalam satu kemasan. IC ini menerima sinyal logika dari mikrokontroler dan mengendalikan beban DC yang membutuhkan arus lebih besar. BTS7960 dilengkapi masukan kendali, keluaran pendeteksi arus, pengaturan *slew rate*, serta perlindungan terhadap suhu berlebih, tegangan lebih, tegangan rendah, arus berlebih, dan hubung singkat [15].

Modul BTS7960 yang umum digunakan terdiri atas dua IC BTS7960. Setiap IC membentuk satu rangkaian *half-bridge*, sedangkan penggunaan dua IC menghasilkan konfigurasi *full H-bridge*. Konfigurasi tersebut memungkinkan arah arus pada beban diatur melalui dua masukan PWM. Arah arus Peltier dibuat tetap karena sistem hanya digunakan untuk melakukan pendinginan, sedangkan besar daya Peltier diatur menggunakan sinyal PWM.

Bentuk fisik modul driver BTS7960 yang digunakan ditunjukkan pada **Gambar 2.7**.



Gambar 2. 6 Gambar Modul Driver BTS7960

Modul BTS7960 memiliki terminal kendali berukuran kecil untuk dihubungkan dengan ESP32 serta terminal daya berukuran lebih besar untuk sumber tegangan dan beban. Terminal daya dibuat lebih besar karena arus yang melewati jalur Peltier lebih tinggi daripada arus pada rangkaian kendali. Modul juga dilengkapi *heatsink* untuk membantu melepaskan panas yang dihasilkan ketika driver bekerja.

Konfigurasi terminal modul BTS7960 yang digunakan dalam penelitian ini disajikan pada Tabel 2.8.

Tabel 2. 8 Tabel Konfigurasi Modul BTS7960

No.	Terminal	Fungsi
1	VCC	Catu daya rangkaian logika modul
2	GND	<i>Ground</i> rangkaian kendali
3	RPWM	Masukan PWM untuk arah arus pertama
4	LPWM	Masukan PWM untuk arah arus berlawanan
5	R_EN	Mengaktifkan rangkaian <i>half-bridge</i> kanan

No.	Terminal	Fungsi
6	L_EN	Mengaktifkan rangkaian <i>half-bridge</i> kiri
7	R_IS	Keluaran pendeteksi arus atau status sisi kanan
8	L_IS	Keluaran pendeteksi arus atau status sisi kiri
9	B+	Masukan positif sumber daya Peltier
10	B-	Masukan negatif sumber daya Peltier
11	M+	Terminal keluaran menuju salah satu kabel Peltier
12	M-	Terminal keluaran menuju kabel Peltier lainnya

Terminal RPWM dan LPWM digunakan untuk menentukan arah serta besar keluaran daya. Terminal R_EN dan L_EN harus berada pada kondisi aktif agar kedua bagian *half-bridge* bekerja. Terminal R_IS dan L_IS menyediakan sinyal yang berhubungan dengan arus beban dan kondisi gangguan, tetapi penggunaannya bergantung pada rancangan sistem.

Hanya salah satu terminal PWM yang menerima sinyal keluaran PID, sedangkan terminal PWM untuk arah berlawanan dipertahankan pada kondisi LOW. Konfigurasi tersebut menjaga polaritas Peltier tetap sama sehingga satu sisi selalu berfungsi sebagai sisi dingin dan sisi lainnya sebagai sisi panas. Pembalikan polaritas tidak digunakan karena sistem tidak dirancang untuk melakukan pemanasan.

Sinyal PWM dari ESP32 hanya berfungsi sebagai sinyal kendali dan tidak menjadi sumber daya utama Peltier. Daya Peltier diperoleh dari catu daya eksternal yang dihubungkan ke terminal B+ dan B-. Nilai *duty cycle* PWM menentukan lama driver menghubungkan tegangan sumber menuju Peltier dalam satu periode. BTS7960 menjadi penghubung antara keluaran kendali berdaya rendah dari ESP32 dan kebutuhan daya Peltier yang relatif besar.

Spesifikasi utama IC BTS7960 berdasarkan lembar data resmi Infineon disajikan pada Tabel 2.9.

Tabel 2. 9 Tabel Spesifikasi Modul BTS7960

No.	Parameter	Spesifikasi
1	Jenis rangkaian	<i>High-current half-bridge</i>
2	Tegangan operasi daya	5,5–27,5 V
3	Frekuensi PWM maksimum	25 kHz
4	Resistansi jalur tipikal	16 mΩ pada 25 °C
5	Batas arus tipikal	43 A
6	Masukan kendali	Logika TTL/CMOS
7	Tegangan maksimum masukan logika	5,3 V
8	Fitur diagnosis	<i>Current sense</i> dan status gangguan
9	Perlindungan	Tegangan rendah, tegangan lebih, arus lebih, suhu berlebih, dan hubung singkat
10	Mode nonaktif	Melalui terminal <i>inhibit</i>

BTS7960 menerima sinyal PWM hingga frekuensi 25 kHz dan memiliki masukan logika yang dikendalikan oleh mikrokontroler. Ambang logika HIGH pada masukan IN berada di bawah level keluaran 3,3 V ESP32, sehingga sinyal kendali ESP32 dikenali oleh IC. Lembar data BTS7960 menetapkan rentang tegangan operasi 5,5–27,5 V dan menyediakan berbagai fungsi proteksi internal.

Nilai 43 A pada lembar data merupakan tingkat pembatasan arus tipikal IC dan tidak boleh langsung dianggap sebagai arus kerja kontinu modul. Arus yang aman tetap dipengaruhi oleh suhu IC, kemampuan *heatsink*, ketebalan jalur PCB, kualitas terminal, ventilasi, dan lama pengoperasian. Lembar data juga menunjukkan bahwa batas arus yang dicapai berubah terhadap suhu dan kondisi operasi. Arus Peltier perlu diukur dan disesuaikan dengan kemampuan driver serta catu daya yang digunakan.

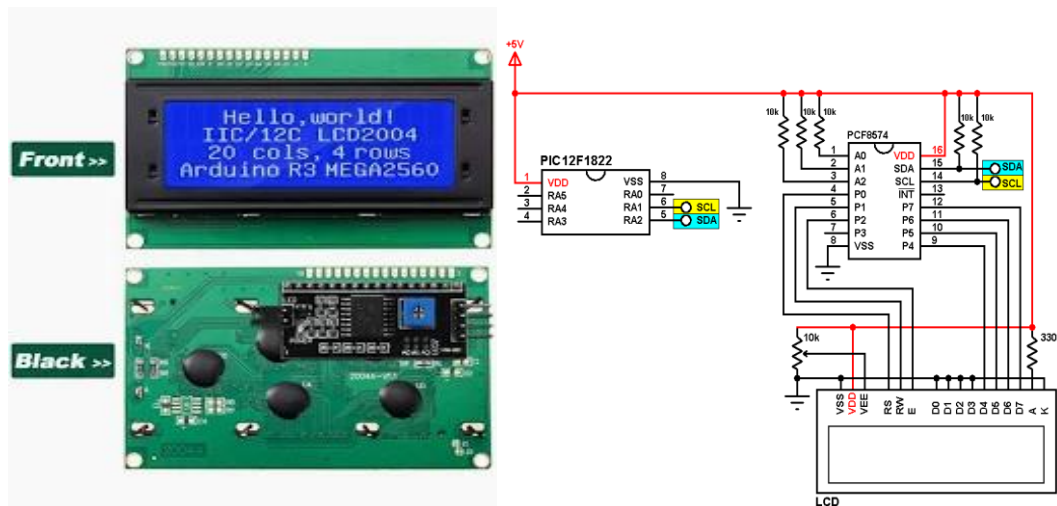
BTS7960 dipilih karena menerima sinyal PWM, mengendalikan beban DC berarus relatif besar, dan memiliki fungsi perlindungan internal. Pada sistem ini,

ESP32 menghitung keluaran PID berdasarkan suhu yang dibaca DS18B20. Nilai keluaran tersebut dikonversi menjadi *duty cycle* PWM dan diberikan kepada BTS7960. Driver kemudian mengatur daya dari catu daya eksternal menuju Peltier sehingga kemampuan pendinginan berubah mengikuti kondisi suhu aktual.

2.11 Liquid Crystal Display (LCD) 20×4 I²C

Liquid Crystal Display atau LCD merupakan perangkat keluaran yang digunakan untuk menampilkan informasi dalam bentuk karakter. LCD yang digunakan pada sistem ini adalah LCD karakter 20×4, yaitu layar yang mampu menampilkan 20 karakter pada setiap baris dengan jumlah empat baris. Setiap karakter dibentuk menggunakan matriks titik 5×8 dan layar dilengkapi dengan lampu latar agar informasi dibaca dengan lebih jelas [16].

LCD digunakan untuk menampilkan suhu aktual, nilai *setpoint*, waktu ekstraksi, keluaran PWM, serta status kerja sistem. Penggunaan empat baris memungkinkan beberapa parameter proses ditampilkan secara bersamaan tanpa terlalu sering mengganti halaman tampilan. Bentuk fisik LCD 20×4 dengan modul antarmuka I²C ditunjukkan pada Gambar 2.8.



Gambar 2. 7 Gambar LCD I2C

Perangkat tersebut terdiri atas layar LCD karakter dan modul antarmuka I²C yang dipasang pada bagian belakang layar. LCD pada dasarnya menggunakan antarmuka paralel yang mempunyai beberapa jalur data dan kendali. Penambahan

modul I²C mengurangi jumlah jalur komunikasi yang perlu dihubungkan ke ESP32 menjadi dua jalur utama, yaitu SDA dan SCL.

Modul antarmuka I²C umumnya menggunakan IC PCF8574 sebagai ekspander masukan dan keluaran. PCF8574 mengubah data serial dari bus I²C menjadi delapan keluaran paralel yang digunakan untuk mengendalikan pin data, pin kendali, dan lampu latar LCD. IC tersebut berkomunikasi melalui jalur *Serial Data* atau SDA dan *Serial Clock* atau SCL [17].

Konfigurasi terminal LCD 20×4 dengan antarmuka I²C disajikan pada Tabel 2.10.

Tabel 2. 10 Tabel Konfigurasi LCD I2C

No.	Terminal	Fungsi
1	GND	Dihubungkan ke <i>ground</i> sistem
2	VCC	Masukan catu daya modul LCD
3	SDA	Jalur pengiriman dan penerimaan data I ² C
4	SCL	Jalur sinyal <i>clock</i> komunikasi I ² C

Penggunaan antarmuka I²C membuat LCD hanya membutuhkan empat terminal untuk catu daya dan komunikasi. Jalur SDA dan SCL digunakan bersama dengan perangkat I²C lainnya, seperti RTC DS3231, selama setiap perangkat mempunyai alamat yang berbeda. Penggunaan bus yang sama membantu mengurangi jumlah GPIO ESP32 yang digunakan.

Alamat I²C modul LCD ditentukan oleh jenis IC ekspander dan kondisi terminal alamat A0, A1, serta A2. Alamat modul tidak selalu sama pada setiap perangkat. Alamat yang akan digunakan pada program perlu diperiksa menggunakan program pemindai I²C sebelum LCD diintegrasikan dengan komponen lainnya. PCF8574 menyediakan kombinasi alamat melalui tiga terminal pengaturan alamat tersebut [17].

Spesifikasi LCD yang berkaitan dengan penelitian disajikan pada Tabel 2.11.

Tabel 2. 11 Tabel Spesifikasi LCD I2C

No.	Parameter	Spesifikasi
1	Jenis tampilan	LCD karakter
2	Kapasitas tampilan	20 karakter $\times$ 4 baris
3	Matriks karakter	5 $\times$ 8 titik
4	Lampu latar	LED
5	Tegangan nominal LCD	5 V
6	Antarmuka mikrokontroler	I ² C
7	Jalur komunikasi	SDA dan SCL
8	IC ekspander I/O	PCF8574 atau sejenisnya
9	Tegangan operasi PCF8574	2,5–6 V
10	Pengaturan alamat	Terminal A0, A1, dan A2

LCD 20 $\times$ 4 menyediakan ruang tampilan yang mencukupi untuk menampilkan beberapa parameter sistem secara bersamaan. Modul PCF8574 bekerja pada tegangan 2,5–6 V dan menyediakan delapan jalur I/O melalui komunikasi I²C [17]. Sementara itu, modul LCD WH2004A menggunakan tegangan nominal 5 V dan mempunyai format tampilan 20 karakter $\times$ 4 baris [16].

Pada sistem yang dirancang, RTC DS3231 dan LCD menggunakan jalur I²C yang sama. ESP32 mengirimkan data tampilan ke LCD tanpa menghentikan proses pembacaan suhu maupun perhitungan PID. Informasi yang ditampilkan diperbarui secara periodik agar perubahan suhu, waktu, dan kondisi aktuator diamati selama ekstraksi berlangsung.

Level tegangan pada jalur SDA dan SCL perlu disesuaikan dengan level logika ESP32 sebesar 3,3 V. Apabila modul LCD diberi catu daya 5 V dan resistor *pull-up* I²C terhubung ke tegangan tersebut, diperlukan penyesuaian rangkaian, misalnya menggunakan *bidirectional logic level converter* atau memindahkan *pull-up* bus ke 3,3 V.

2.12 Push Button

Push button merupakan sakelar mekanis yang digunakan sebagai perangkat masukan untuk memberikan perintah kepada mikrokontroler. Komponen ini bekerja dengan mengubah kondisi kontak listrik ketika bagian tombol ditekan. Jenis yang digunakan pada sistem adalah *momentary normally open*, yaitu kontak berada dalam kondisi terbuka ketika tombol tidak ditekan dan terhubung selama tombol ditekan. Salah satu konfigurasi kontak yang umum digunakan pada *tactile switch* adalah *Single Pole Single Throw–Normally Open* atau SPST-NO [18]. Datasheet Omron B3F mencantumkan bentuk kontak SPST-NO dan waktu *bounce* maksimum 5 ms.

Push button digunakan sebagai antarmuka masukan untuk mengatur parameter dan menjalankan fungsi pada sistem pendingin. Perintah yang diberikan melalui tombol digunakan untuk memilih menu, menaikkan atau menurunkan nilai pengaturan, memulai proses ekstraksi, serta menghentikan proses ketika diperlukan. .

Bentuk fisik *push button* yang digunakan ditunjukkan pada Gambar 2.10.



Gambar 2. 8 Gambar *Push Button*

Push button memiliki bagian penekan dan terminal kontak yang dipasang pada rangkaian. Pada jenis *tactile switch* berkaki empat, dua terminal pada sisi yang sama umumnya telah terhubung secara internal. Ketika tombol ditekan, kedua kelompok terminal yang sebelumnya terpisah akan terhubung sehingga menghasilkan perubahan kondisi logika pada pin masukan ESP32.

Konfigurasi sambungan *push button* pada sistem disajikan pada Tabel 2.12.

Tabel 2. 12 Tabel Konfigurasi *Push Button*

No.	Terminal	Sambungan	Fungsi
1	Terminal pertama	GPIO ESP32	Mengirimkan perubahan logika ke ESP32
2	Terminal kedua	GND	Menghubungkan GPIO ke <i>ground</i> ketika tombol ditekan

Push button dihubungkan antara GPIO ESP32 dan GND. Pin GPIO dikonfigurasi menggunakan resistor *pull-up*, sehingga kondisi masukan bernilai HIGH ketika tombol tidak ditekan. Ketika tombol ditekan, GPIO terhubung dengan GND dan kondisi masukan berubah menjadi LOW. Konfigurasi tersebut dikenal sebagai masukan *active-low*.

Logika pembacaan tombol disajikan pada Tabel 2.13.

Tabel 2. 13 Tabel Logika *Push Button*

Kondisi tombol	Kondisi kontak	Logika GPIO	Perintah
Tidak ditekan	Terbuka	HIGH	Tidak aktif
Ditekan	Terhubung ke GND	LOW	Aktif

Program mendeteksi penekanan tombol ketika kondisi GPIO berubah dari HIGH menjadi LOW. Penggunaan logika *active-low* memungkinkan resistor *pull-up* internal pada ESP32 digunakan sehingga rangkaian tidak membutuhkan resistor tambahan pada setiap tombol. Kondisi awal GPIO harus ditetapkan dengan benar agar masukan tidak berada dalam keadaan mengambang.

Kontak mekanis *push button* tidak selalu langsung menghasilkan satu perubahan logika yang stabil. Ketika tombol ditekan atau dilepaskan, kontak mengalami perubahan HIGH dan LOW beberapa kali dalam waktu singkat yang disebut *contact bounce*. Datasheet B3F menunjukkan waktu *bounce* maksimum sebesar 5 ms pada seri tersebut [18]. Apabila kondisi ini tidak ditangani, satu kali penekanan terbaca sebagai beberapa perintah.

Pengaruh *contact bounce* ditangani melalui metode *software debounce*. Program hanya menerima kondisi tombol apabila logika masukan tetap stabil selama selang waktu tertentu. Metode tersebut mencegah perubahan menu atau nilai pengaturan terjadi beberapa kali akibat satu kali penekanan.

Karakteristik umum *tactile push button* seri B3F yang digunakan sebagai referensi disajikan pada Tabel 2.14.

Tabel 2. 14 Tabel Spesifikasi *Push Button*

No.	Parameter	Spesifikasi
1	Bentuk kontak	SPST-NO
2	Tegangan kerja beban resistif	3–24 VDC
3	Arus kerja beban resistif	1–50 mA
4	Resistansi kontak awal	Maksimum 100 mΩ
5	Waktu <i>bounce</i>	Maksimum 5 ms
6	Temperatur operasi	–25 °C hingga 70 °C
7	Jenis pengoperasian	<i>Momentary</i>
8	Kondisi normal	Kontak terbuka

Push button sesuai digunakan sebagai masukan logika karena hanya dilewati arus kecil dari rangkaian GPIO. Tombol tidak digunakan untuk memutus atau menghubungkan daya Peltier secara langsung. Proses pengendalian beban berarus besar tetap dilakukan melalui driver BTS7960, sedangkan *push button* hanya memberikan perintah kepada ESP32.

2.13 Buzzer

Buzzer merupakan komponen keluaran yang mengubah energi listrik menjadi energi bunyi. Berdasarkan prinsip kerjanya, buzzer menggunakan mekanisme elektromagnetik atau piezoelektrik. Buzzer piezoelektrik menghasilkan bunyi melalui getaran elemen piezoelektrik ketika menerima sinyal listrik dengan frekuensi tertentu. Komponen ini banyak digunakan sebagai indikator suara pada

peralatan elektronik karena memiliki ukuran kecil dan kebutuhan daya relatif rendah [19].

Buzzer digunakan sebagai indikator suara untuk memberikan informasi kepada pengguna. Buzzer diaktifkan ketika proses ekstraksi telah selesai, tombol tertentu ditekan, atau sistem mendeteksi kondisi yang memerlukan perhatian pengguna. Penggunaan indikator suara membantu pengguna mengetahui status sistem tanpa harus terus-menerus melihat LCD.

Bentuk fisik buzzer yang digunakan pada sistem ditunjukkan pada Gambar 2.11.



Gambar 2. 9 Gambar Buzzer

Buzzer memiliki lubang keluaran suara pada bagian atas dan terminal listrik pada bagian bawah. Lubang suara tidak boleh tertutup oleh konstruksi panel karena mengurangi intensitas bunyi yang terdengar. Posisi buzzer pada kotak alat perlu dibuat menghadap ke bagian luar agar pemberitahuan suara diterima pengguna dengan jelas.

Buzzer dua terminal memiliki terminal positif dan negatif. Konfigurasi terminal buzzer disajikan pada Tabel 2.15.

Tabel 2. 15 Tabel Konfigurasi Buzzer

No.	Terminal	Fungsi
1	Positif (+)	Menerima tegangan atau sinyal penggerak
2	Negatif (-)	Dihubungkan ke <i>ground</i> rangkaian

Polaritas terminal perlu diperhatikan terutama pada buzzer aktif yang telah dilengkapi rangkaian osilator internal. Terminal positif menerima sumber atau sinyal kendali, sedangkan terminal negatif dihubungkan ke *ground*. Pada buzzer berbentuk modul tiga terminal, terminal yang tersedia umumnya terdiri atas VCC, GND, dan terminal sinyal.

Buzzer dibedakan menjadi buzzer aktif dan buzzer pasif. Buzzer aktif telah memiliki rangkaian pembangkit frekuensi internal sehingga menghasilkan suara ketika diberi tegangan DC. Buzzer pasif tidak memiliki osilator internal sehingga membutuhkan sinyal pulsa atau PWM dengan frekuensi tertentu. Buzzer piezoelektrik seri PS dari TDK merupakan jenis tanpa rangkaian osilator dan harus digerakkan menggunakan sinyal bolak-balik atau pulsa, bukan tegangan DC secara terus-menerus [19].

Pola bunyi buzzer diatur melalui program ESP32. Perbedaan pola bunyi digunakan untuk membedakan pemberitahuan, misalnya bunyi singkat ketika tombol ditekan dan bunyi berulang ketika waktu ekstraksi telah selesai. Pengaturan pola dilakukan berdasarkan waktu aktif dan waktu tidak aktif buzzer tanpa menghentikan pembacaan sensor maupun perhitungan PID.

Buzzer tidak berhubungan langsung dengan proses pengendalian suhu. Komponen ini berfungsi sebagai perangkat antarmuka yang menyampaikan status proses kepada pengguna. Perintah pengaktifan buzzer diberikan oleh ESP32 berdasarkan data waktu dari RTC DS3231 dan kondisi sistem yang diproses oleh program.

2.14 Sistem Pelepasan Panas dan Isolasi Termal

Sistem pelepasan panas merupakan bagian penting dalam penggunaan *Thermoelectric Cooler* atau Peltier. Ketika Peltier dialiri arus listrik, panas yang diserap pada sisi dingin dipindahkan menuju sisi panas. Selain panas dari objek yang didinginkan, sisi panas juga menerima panas akibat konsumsi daya listrik Peltier. Panas pada sisi tersebut harus dilepaskan secara terus-menerus agar perbedaan temperatur antara sisi panas dan sisi dingin dipertahankan [2], [10].

Sistem pelepasan panas terdiri atas *heatsink*, kipas DC, dan *thermal paste*. Sementara itu, bagian ruang ekstraksi dilengkapi dengan isolasi termal untuk

mengurangi masuknya panas dari lingkungan. Susunan komponen pelepasan panas yang digunakan pada sistem ditunjukkan pada Gambar 2.12.



Gambar 2. 10 Gambar Peltier Kit

Berdasarkan Gambar 2.12, sisi panas Peltier dipasangkan dengan *heatsink*, sedangkan kipas ditempatkan agar aliran udara melewati sirip-sirip *heatsink*. Panas berpindah dari permukaan Peltier menuju *heatsink* secara konduksi. Panas tersebut kemudian dilepaskan dari permukaan *heatsink* menuju udara sekitar melalui konveksi paksa yang dihasilkan oleh kipas.

Fungsi setiap bagian pada sistem perpindahan panas disajikan pada Tabel 2.16.

Tabel 2. 16 Tabel Komponen Sistem Pelepasan Panas dan Isolasi Termal

No.	Komponen	Fungsi
1	Pelat dingin	Menghantarkan panas dari wadah atau cairan menuju sisi dingin Peltier
2	Peltier	Memindahkan panas dari sisi dingin menuju sisi panas
3	<i>Thermal paste</i>	Mengurangi hambatan termal pada bidang kontak antarkomponen

No.	Komponen	Fungsi
4	<i>Heatsink</i>	Memperluas permukaan pelepasan panas pada sisi panas Peltier
5	Kipas DC	Mengalirkan udara melalui <i>heatsink</i> agar pelepasan panas meningkat
6	Isolasi termal	Mengurangi perpindahan panas dari lingkungan menuju ruang ekstraksi

Berdasarkan Tabel 2.16, setiap komponen memiliki fungsi yang saling berkaitan. Pelat dingin harus memiliki kontak yang baik dengan sisi dingin Peltier agar panas dari ruang ekstraksi diserap. Sementara itu, *heatsink* dan kipas harus mampu membuang panas yang dipindahkan oleh Peltier agar temperatur sisi panas tidak terus meningkat.

2.14.1 *Heatsink*

Heatsink merupakan komponen penghantar panas yang digunakan untuk memperluas luas permukaan pelepasan panas. Komponen ini umumnya dibuat dari aluminium karena memiliki konduktivitas termal yang baik, massa relatif ringan, dan mudah dibentuk. Bentuk sirip pada *heatsink* memberikan luas permukaan yang lebih besar dibandingkan permukaan datar sehingga perpindahan panas menuju udara berlangsung lebih efektif. Bentuk dari *Heatsink* yang digunakan pada sistem ditunjukkan pada Gambar 2.13.



Gambar 2. 11 Gambar *Heatsink*

Dalam sistem Peltier, ukuran *heatsink* harus disesuaikan dengan panas yang dihasilkan pada sisi panas. Penggunaan *heatsink* yang terlalu kecil menyebabkan temperatur sisi panas meningkat. Kondisi tersebut mengurangi kemampuan sisi dingin dalam menyerap panas dan menyebabkan suhu target sulit dicapai. Penelitian pendingin termoelektrik sebelumnya juga menggunakan *heatsink* sebagai bagian utama sistem pembuangan panas Peltier [2], [10].

2.14.2 Kipas DC

Kipas DC digunakan untuk menghasilkan aliran udara pada permukaan *heatsink*. Aliran udara tersebut mempercepat perpindahan panas dari sirip *heatsink* menuju lingkungan. Tanpa kipas, pelepasan panas hanya mengandalkan konveksi alami sehingga kemampuan pembuangan panas menjadi lebih rendah. Bentuk dari kipas DC yang digunakan pada sistem ditunjukkan pada Gambar 2.13.



Gambar 2. 12 Gambar Kipas DC

Kipas sisi panas dioperasikan selama Peltier bekerja. Kipas tidak dikendalikan menggunakan keluaran PID karena keluaran PID secara khusus digunakan untuk mengatur daya Peltier. Pengoperasian kipas secara terus-menerus bertujuan menjaga suhu *heatsink* agar tidak meningkat secara berlebihan ketika *duty cycle* Peltier berubah.

Spesifikasi kipas yang digunakan disajikan pada Tabel 2.17 berdasarkan tulisan pada label kipas.

Tabel 2. 17 Tabel Spesifikasi Kipas DC

No.	Parameter	Spesifikasi
1	Tegangan	12 V
2	Arus	0,42 A
3	Daya	5 watt
4	Ukuran kipas	9 cm × 9 cm
6	Fungsi	Pendingin <i>heatsink</i> sisi panas Peltier

Nilai daya kipas dihitung menggunakan **Persamaan (2.5)**.

$$P_f = V_f \times I_f \quad (2.5)$$

Keterangan:

P_f = daya listrik kipas dalam watt

V_f = tegangan kerja kipas dalam volt

I_f = arus kerja kipas dalam ampere

2.14.3 Thermal Paste

Permukaan Peltier, pelat dingin, dan *heatsink* tidak sepenuhnya rata apabila dilihat secara mikroskopis. Celah kecil yang terdapat di antara permukaan terisi oleh udara yang memiliki kemampuan penghantar panas rendah. *Thermal paste* digunakan untuk mengisi celah tersebut agar kontak termal antarkomponen menjadi lebih baik.

Thermal paste diaplikasikan dalam lapisan tipis dan merata. Penggunaan yang terlalu sedikit menyebabkan masih terdapat celah udara, sedangkan penggunaan terlalu tebal meningkatkan jarak perpindahan panas. Selain itu, Peltier perlu dijepit dengan tekanan yang merata agar pelat keramik tidak mengalami kerusakan.

2.15 Catu Daya

Catu daya merupakan bagian yang menyediakan energi listrik bagi seluruh komponen pada sistem. Sistem pendingin ini mempunyai dua kelompok beban, yaitu rangkaian kendali berdaya rendah dan rangkaian aktuator berdaya lebih besar. Rangkaian kendali terdiri atas ESP32, DS18B20, RTC DS3231, LCD, *push button*, dan buzzer. Sementara itu, rangkaian daya terdiri atas driver BTS7960, Peltier, dan kipas pendingin.

Peltier tidak memperoleh daya secara langsung dari pin ESP32 karena membutuhkan tegangan dan arus yang jauh lebih besar daripada kemampuan keluaran mikrokontroler. Peltier memperoleh daya dari catu daya eksternal melalui driver BTS7960, sedangkan ESP32 hanya memberikan sinyal PWM sebagai perintah kendali. Pembagian tersebut sesuai dengan batasan rancangan bahwa Peltier dikendalikan melalui rangkaian penggerak dengan sumber tegangan eksternal.

Sistem pendingin termoelektrik pada penelitian Pua *et al.* juga menggunakan catu daya 12 VDC untuk menyuplai Peltier dan perangkat pendingin lainnya [10].

Besar arus catu daya harus disesuaikan dengan jumlah Peltier, kipas, serta karakteristik beban aktual. Bentuk fisik catu daya yang digunakan ditunjukkan pada Gambar 2.14.



Gambar 2. 13 Gambar Power Supply

Berdasarkan Gambar 2.14, catu daya menyediakan terminal masukan listrik dan terminal keluaran DC. Terminal keluaran positif dan negatif dihubungkan ke rangkaian daya melalui kabel yang sesuai dengan arus beban. Nilai tegangan keluaran harus disesuaikan dengan tegangan kerja Peltier, kipas, dan driver BTS7960 agar seluruh komponen beroperasi tanpa melampaui batas spesifikasinya.

Pembagian sumber tegangan pada sistem disajikan pada Tabel 2.18.

Tabel 2. 18 Tabel Pembagian Sumber Tegangan

No.	Bagian sistem	Sumber daya	Keterangan
1	Peltier TEC1-12706	Catu daya eksternal DC	Disalurkan melalui BTS7960
2	Driver BTS7960 bagian daya	Catu daya eksternal DC	Terminal B+ dan B-
3	Kipas DC	Catu daya eksternal DC	Tegangan mengikuti spesifikasi kipas

No.	Bagian sistem	Sumber daya	Keterangan
4	ESP32	USB atau catu daya teratur	Menggunakan regulator pada board
5	DS18B20	ESP32	Menggunakan tegangan logika sistem
6	RTC DS3231	ESP32	Dilengkapi baterai cadangan
7	LCD I ² C	Jalur catu daya kendali	Disesuaikan dengan modul LCD
8	Buzzer	Jalur catu daya kendali	Sesuai tipe buzzer

Berdasarkan Tabel 2.19, jalur daya Peltier dipisahkan secara fungsi dari jalur daya rangkaian kendali. Pemisahan tersebut bertujuan agar perubahan arus pada Peltier tidak langsung dibebankan pada regulator ESP32. Meskipun menggunakan jalur catu daya yang berbeda, GND ESP32 dan GND bagian logika BTS7960 perlu dihubungkan sebagai referensi tegangan yang sama agar sinyal PWM dikenali dengan benar.

Daya listrik yang digunakan oleh suatu beban DC dihitung menggunakan **Persamaan (2.6)**.

$$P = V \times I \quad (2.6)$$

Keterangan:

P = daya listrik dalam watt

V = tegangan beban dalam volt

I = arus beban dalam ampere

Persamaan tersebut juga digunakan pada penelitian Pua *et al.* untuk menghitung daya masukan sistem termoelektrik berdasarkan tegangan dan arus yang diukur [10].

Apabila terdapat beberapa beban yang menggunakan satu catu daya, kebutuhan arus total diperkirakan menggunakan **Persamaan (2.7)**.

$$I_{\text{total}} = I_{\text{Peltier}} + I_{\text{kipas}} + I_{\text{beban lainnya}} \quad (2.7)$$

Keterangan:

I_{total} = total kebutuhan arus sistem

I_{Peltier} = arus yang digunakan Peltier

I_{kipas} = arus yang digunakan kipas

$I_{\text{beban lainnya}}$ = arus beban lain pada sumber yang sama

Kapasitas arus catu daya harus lebih besar daripada kebutuhan arus total sistem. Penentuan kapasitas tidak hanya didasarkan pada arus rata-rata ketika PID bekerja, tetapi juga harus mempertimbangkan kondisi *duty cycle* maksimum. Ketika PWM mencapai 100%, Peltier memperoleh tegangan sumber secara penuh sehingga catu daya dan driver harus mampu menyalurkan arus tertinggi yang terjadi.

Spesifikasi catu daya yang digunakan disajikan pada Tabel 2.19 berdasarkan label perangkat dan hasil pengukuran.

Tabel 2. 19 Spesifikasi catu daya yang digunakan

No.	Parameter	Spesifikasi
1	Jenis catu daya	Power supply 12 V 50 A
2	Tegangan masukan	220 VAC
3	Tegangan keluaran	12 VDC
4	Arus keluaran maksimum	50 A
5	Daya keluaran maksimum	600 W
6	Beban yang disuplai	Peltier, BTS7960, dan kipas

Pada sistem ini, catu daya berfungsi menjaga ketersediaan energi bagi Peltier ketika keluaran PID berubah. Driver BTS7960 menerima daya dari catu daya eksternal dan mengatur penyalurannya menuju Peltier berdasarkan *duty cycle* PWM dari ESP32. Kestabilan tegangan catu daya akan memengaruhi respons pendinginan dan hasil pengujian identifikasi sistem.

2.16 MATLAB

MATLAB merupakan perangkat lunak komputasi numerik yang digunakan untuk pengolahan data, pemodelan sistem, simulasi, dan perancangan sistem

kendali. MATLAB digunakan sebagai perangkat bantu dalam proses identifikasi karakteristik sistem pendingin dan penentuan parameter pengendali PID.

MATLAB digunakan sebagai perangkat bantu untuk mengolah data pengujian, membuat grafik respons sistem, dan memvisualisasikan hasil pengujian. Penentuan parameter PID dilakukan menggunakan metode Ziegler–Nichols berdasarkan nilai K, L, dan T yang diperoleh dari kurva reaksi proses [3], [1].

MATLAB tidak digunakan sebagai pengendali ketika alat beroperasi. Nilai parameter PID yang diperoleh dimasukkan ke dalam program ESP32, kemudian diuji secara langsung pada prototipe.

2.17 Arduino IDE

Arduino IDE merupakan perangkat lunak yang digunakan untuk menulis, memeriksa, mengompilasi, dan mengunggah program ke mikrokontroler. Perangkat lunak ini menyediakan *code editor*, *Board Manager*, *Library Manager*, serta *Serial Monitor* yang digunakan untuk mengamati data dari perangkat secara langsung.

Arduino IDE digunakan untuk mengembangkan program ESP32 yang mencakup pembacaan sensor DS18B20, pengambilan data waktu dari RTC DS3231, perhitungan pengendali PID, pembangkitan PWM, pengaturan LCD, pembacaan *push button*, dan pengaktifan buzzer. Dukungan pemrograman ESP32 pada Arduino IDE diperoleh melalui pemasangan paket Arduino-ESP32 pada *Board Manager*.

Program terlebih dahulu diverifikasi untuk mendeteksi kesalahan penulisan dan memastikan seluruh pustaka dikompilasi. Setelah proses verifikasi berhasil, program diunggah ke ESP32 melalui koneksi USB. *Serial Monitor* digunakan untuk mengamati temperatur, nilai *setpoint*, keluaran PID, *duty cycle* PWM, serta data waktu selama tahap pengujian.

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

Penelitian ini dilakukan melalui beberapa tahapan yang tersusun secara sistematis, mulai dari studi literatur hingga pengujian kinerja sistem. Studi literatur dilakukan untuk memperoleh dasar mengenai proses ekstraksi *cold brew coffee*, sistem pendingin termoelektrik, pengendali PID, identifikasi sistem, serta karakteristik komponen yang digunakan. Berdasarkan hasil studi tersebut, ditentukan spesifikasi dan kebutuhan sistem yang akan dirancang.

Tahap berikutnya adalah perancangan perangkat keras dan perangkat lunak. Perangkat keras meliputi ESP32, sensor DS18B20, RTC DS3231, driver BTS7960, Peltier, sistem pelepasan panas, LCD, *push button*, buzzer, dan catu daya. Perangkat lunak dirancang untuk membaca suhu, mencatat waktu ekstraksi, menghitung keluaran PID, menghasilkan PWM, serta menampilkan kondisi sistem kepada pengguna.

Setelah perakitan selesai, setiap komponen diuji secara terpisah untuk memastikan bekerja sesuai fungsinya. Pengujian meliputi pembacaan sensor suhu, komunikasi RTC dan LCD, pembangkitan PWM, pengoperasian driver BTS7960, serta kemampuan Peltier dalam menurunkan suhu. Apabila ditemukan kesalahan, dilakukan pemeriksaan dan perbaikan sebelum sistem diuji secara keseluruhan.

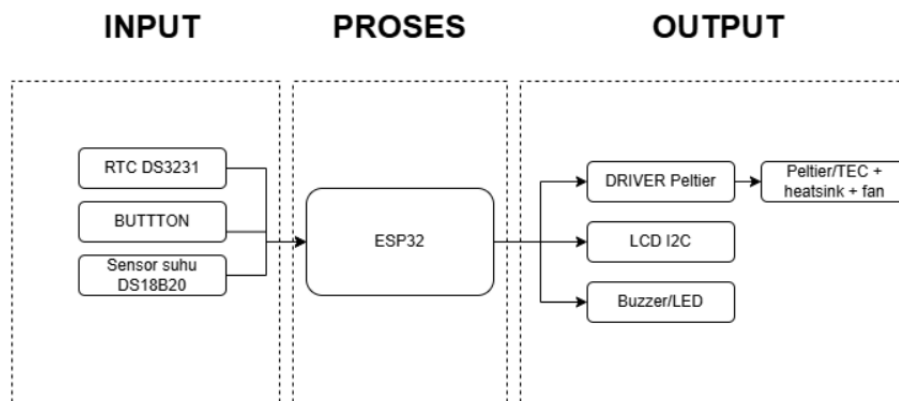
Pengujian sistem pendingin diawali dengan pengujian *open-loop*. Pada tahap ini, Peltier dijalankan menggunakan nilai PWM tertentu tanpa pengendali PID. Data waktu, nilai PWM, dan temperatur dicatat untuk memperoleh karakteristik respons sistem. Data tersebut kemudian diolah untuk memperoleh kurva respons sistem pendingin. Kurva respons digunakan untuk menentukan gain proses (K), waktu tunda (L), dan konstanta waktu (T).

Parameter K, L, dan T selanjutnya digunakan untuk menghitung nilai K_p, K_i, dan K_d menggunakan metode Ziegler–Nichols [3], [1]. Kemudian dimasukkan ke dalam program ESP32 dan diuji pada sistem sebenarnya. Kinerja pengendali dievaluasi berdasarkan waktu pencapaian *setpoint*, *overshoot*, galat keadaan tunak, dan kestabilan temperatur selama proses ekstraksi.

3.2 Perancangan Sistem

Perancangan sistem dilakukan untuk menentukan hubungan antara masukan, proses, dan keluaran pada alat pendingin ekstraksi *cold brew coffee*. Sistem menerima masukan berupa temperatur cairan dari sensor DS18B20, waktu dari RTC DS3231, serta perintah pengguna melalui *push button*. Seluruh data tersebut diproses oleh ESP32 untuk mengatur pendinginan, durasi ekstraksi, tampilan informasi, dan indikator suara.

Diagram blok sistem secara keseluruhan ditunjukkan pada Gambar 3.1.



Gambar 3. 1 Diagram blok sistem pendingin ekstraksi *cold brew coffee*

Berdasarkan Gambar 3.1, sensor DS18B20 mengukur temperatur ruangan ekstraksi dan mengirimkan data tersebut kepada ESP32. Temperatur aktual dibandingkan dengan nilai *setpoint* untuk menghasilkan nilai error. Nilai error kemudian diproses menggunakan algoritma PID sehingga diperoleh keluaran kendali dalam bentuk *duty cycle* PWM.

Sinyal PWM dari ESP32 diberikan kepada driver BTS7960. Driver berfungsi mengatur penyaluran daya dari catu daya eksternal menuju Peltier. Peltier menyerap panas dari bagian ruang ekstraksi melalui sisi dingin dan membuang panas melalui sisi panas yang dilengkapi *heatsink* serta kipas.

RTC DS3231 memberikan informasi waktu aktual kepada ESP32. Data waktu digunakan untuk mencatat waktu mulai, menghitung durasi ekstraksi, dan menentukan waktu selesai proses. Timer internal ESP32 digunakan untuk mengatur interval pembacaan sensor dan perhitungan PID agar proses kendali tetap berjalan secara periodik.

Push button digunakan untuk memberikan perintah berupa pengaturan *setpoint*, pengaturan durasi, memulai proses, dan menghentikan sistem. LCD menampilkan temperatur aktual, temperatur target, durasi ekstraksi, dan status kerja alat. Buzzer memberikan indikator suara ketika tombol ditekan, terjadi kondisi kesalahan, atau proses ekstraksi telah selesai.

3.2.1 Prinsip Kerja Sistem

Sistem mulai bekerja setelah pengguna mengatur nilai *setpoint* temperatur dan durasi ekstraksi melalui *push button*. Nilai pengaturan ditampilkan pada LCD agar diperiksa sebelum proses dijalankan. Setelah tombol mulai ditekan, ESP32 mencatat waktu awal dari RTC DS3231 dan mengaktifkan sistem pendingin.

Sensor DS18B20 membaca temperatur cairan secara periodik. Apabila temperatur aktual masih berada di atas *setpoint*, ESP32 menghitung keluaran PID dan mengubahnya menjadi *duty cycle* PWM. Semakin besar kebutuhan pendinginan, semakin besar nilai PWM yang diberikan kepada BTS7960.

Driver BTS7960 kemudian menyalurkan daya menuju Peltier sesuai nilai PWM. Sisi dingin Peltier menyerap panas dari ruang ekstraksi, sedangkan sisi panas melepaskan panas melalui *heatsink* dan kipas. Ketika temperatur mendekati *setpoint*, keluaran PID dikurangi untuk mencegah suhu turun terlalu jauh.

Selama proses berlangsung, ESP32 terus memperbarui tampilan temperatur dan waktu pada LCD. RTC DS3231 digunakan untuk membandingkan waktu aktual dengan waktu selesai yang telah ditentukan. Ketika durasi ekstraksi terpenuhi, ESP32 menghentikan keluaran PWM, menonaktifkan Peltier, dan mengaktifkan buzzer sebagai pemberitahuan bahwa proses telah selesai.

3.2.2 Pembagian Fungsi Komponen

Pembagian fungsi setiap komponen dalam sistem disajikan pada Tabel 3.1.

Tabel 3. 1 Fungsi komponen pada sistem

No.	Komponen	Fungsi
1	ESP32	Memproses data, menjalankan PID, menghasilkan PWM, dan mengatur seluruh sistem
2	DS18B20	Mengukur temperatur cairan ekstraksi
3	RTC DS3231	Menyediakan data waktu dan durasi proses
4	BTS7960	Mengatur penyaluran daya menuju Peltier
5	Peltier TEC1-12706	Memindahkan panas dari ruang ekstraksi menuju sisi panas
6	<i>Heatsink</i>	Melepaskan panas dari sisi panas Peltier
7	Kipas DC	Meningkatkan aliran udara pada <i>heatsink</i>
8	LCD 20×4	Menampilkan temperatur, waktu, dan status sistem
9	<i>Push button</i>	Memberikan perintah dan pengaturan kepada ESP32
10	Buzzer	Memberikan indikator suara
11	Catu daya	Menyediakan energi listrik bagi rangkaian kendali dan aktuator
12	Isolasi termal	Mengurangi panas dari lingkungan yang masuk ke ruang ekstraksi

Berdasarkan Tabel 3.1, ESP32 menjadi pusat pengolahan data dan pengendalian sistem. DS18B20 dan RTC DS3231 berfungsi sebagai perangkat masukan, sedangkan LCD dan buzzer berfungsi sebagai perangkat keluaran informasi. BTS7960 dan Peltier menjadi bagian utama dalam proses pengaturan temperatur.

3.3 Perancangan Perangkat Keras

Perancangan perangkat keras dilakukan dengan mengintegrasikan bagian masukan, pengendali, aktuator, antarmuka pengguna, dan catu daya. Bagian masukan terdiri atas sensor suhu DS18B20, RTC DS3231, dan *push button*. Data

masuk diproses oleh ESP32 untuk menentukan keluaran pengendali PID, mengatur durasi ekstraksi, serta memperbarui informasi pada LCD.

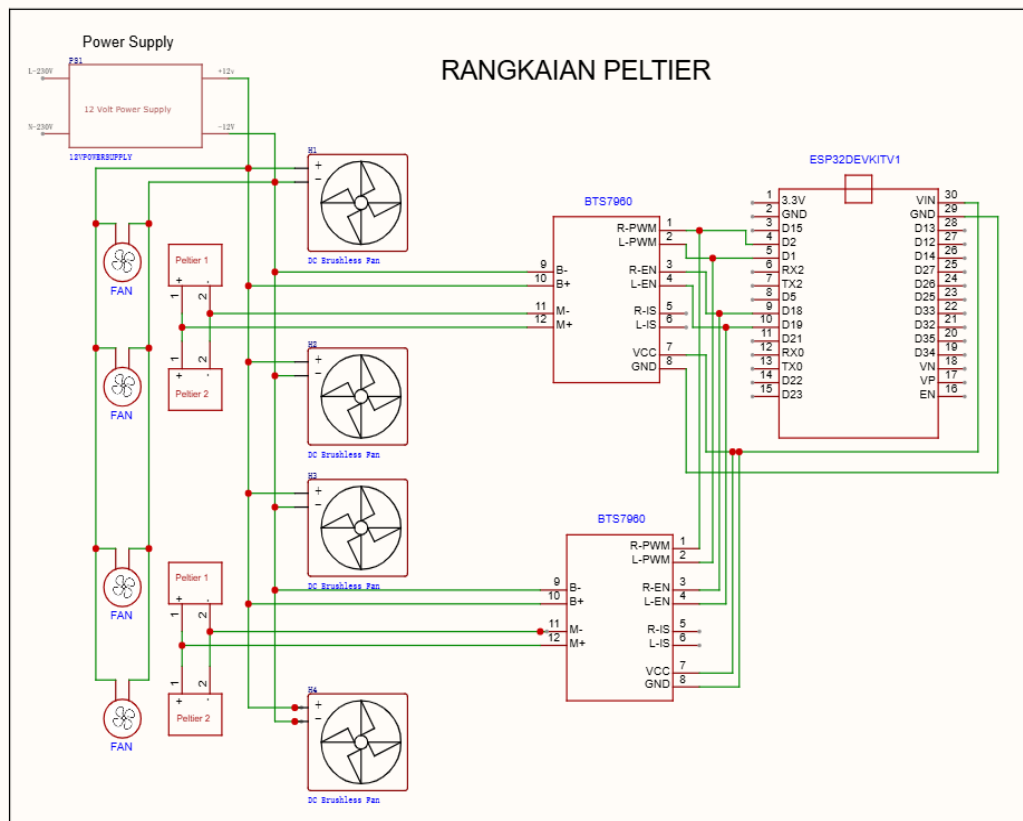
Bagian aktuator terdiri atas driver BTS7960 dan Peltier TEC1-12706. ESP32 menghasilkan sinyal PWM berdasarkan hasil perhitungan PID, kemudian sinyal tersebut diberikan kepada BTS7960. Driver mengatur penyaluran daya dari catu daya eksternal menuju Peltier. Sistem pelepasan panas menggunakan *heatsink* dan kipas agar panas pada sisi panas Peltier dibuang ke lingkungan.

Perancangan perangkat keras juga memperhatikan pemisahan antara rangkaian kendali dan rangkaian daya. ESP32, sensor, RTC, LCD, tombol, dan buzzer termasuk rangkaian kendali berdaya rendah. Peltier, BTS7960 bagian daya, dan kipas termasuk rangkaian yang membutuhkan arus lebih besar. Meskipun menggunakan jalur sumber daya yang berbeda, seluruh bagian menggunakan referensi *ground* yang sama agar sinyal kendali diterima dengan benar.

3.3.1 Rangkaian Peltier dan Driver BTS7960

Rangkaian Peltier merupakan bagian aktuator yang berfungsi menurunkan temperatur ruang ekstraksi *cold brew coffee*. Peltier tidak dihubungkan langsung ke ESP32 karena membutuhkan arus kerja yang relatif besar. Peltier dikendalikan menggunakan driver BTS7960, sedangkan ESP32 hanya memberikan sinyal kendali berupa PWM.

Rangkaian Peltier dan driver BTS7960 ditunjukkan pada Gambar 3.2.



Gambar 3. 2 Rangkaian Peltier dan driver BTS7960

Hubungan rangkaian Peltier dan BTS7960 disajikan pada Tabel 3.2.

Tabel 3. 2 Hubungan rangkaian Peltier dan BTS7960

No.	Komponen	Terminal	Hubungan
1	ESP32	GPIO2	RPWM BTS7960
2	ESP32	GPIO1	LPWM BTS7960
3	ESP32	GPIO18	R_EN BTS7960
4	ESP32	GPIO19	L_EN BTS7960
5	BTS7960	VCC	5 V logika
6	BTS7960	GND	GND sistem
7	BTS7960	B+	Positif catu daya Peltier
8	BTS7960	B-	Negatif catu daya Peltier
9	BTS7960	M+	Kabel positif Peltier
10	BTS7960	M-	Kabel negatif Peltier
11	Kipas DC	Positif dan negatif	Catu daya eksternal

Berdasarkan Tabel 3.2, terminal RPWM dan LPWM digunakan sebagai jalur kendali PWM dari ESP32. Pada sistem pendingin ini, polaritas Peltier dibuat tetap sehingga satu sisi Peltier selalu berfungsi sebagai sisi dingin dan sisi lainnya sebagai sisi panas. Terminal R_EN dan L_EN digunakan untuk mengaktifkan driver BTS7960.

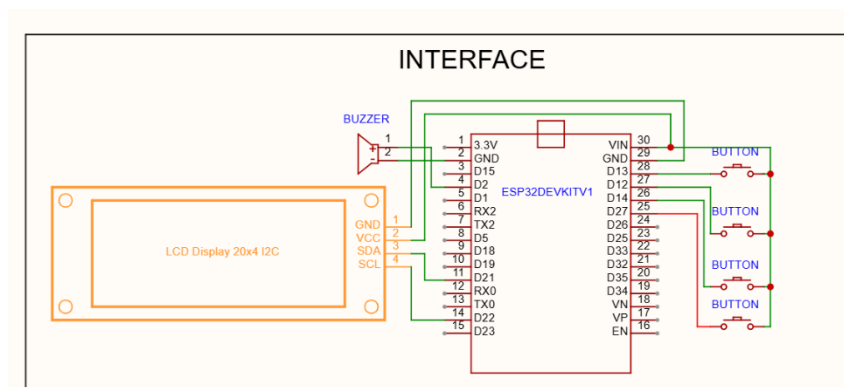
Catu daya eksternal dihubungkan ke terminal B+ dan B- BTS7960. Daya dari catu daya eksternal kemudian disalurkan menuju Peltier melalui terminal M+ dan M-. Dengan susunan ini, arus kerja Peltier tidak melewati ESP32. ESP32 hanya mengatur besar daya pendinginan melalui perubahan nilai *duty cycle* PWM.

Kipas DC digunakan untuk membuang panas dari sisi panas Peltier melalui *heatsink*. Pada rancangan ini, kipas dioperasikan selama proses pendinginan berlangsung agar pelepasan panas pada sisi panas Peltier tetap stabil.

3.3.2 Rangkaian Antarmuka Pengguna

Rangkaian antarmuka pengguna terdiri atas LCD 20×4 I²C, *push button*, dan buzzer. LCD digunakan untuk menampilkan informasi suhu aktual, suhu *setpoint*, waktu ekstraksi, nilai PWM, dan status sistem. *Push button* digunakan sebagai masukan untuk mengatur nilai *setpoint*, durasi, serta perintah mulai dan berhenti. Buzzer digunakan sebagai indikator suara ketika proses selesai atau terjadi kondisi tertentu.

Rangkaian antarmuka pengguna ditunjukkan pada **Gambar 3.3**.



Gambar 3. 3 Rangkaian antarmuka pengguna

Hubungan rangkaian antarmuka pengguna disajikan pada **Tabel 3.3**.

Tabel 3. 3 Hubungan rangkaian antarmuka pengguna

No.	Komponen	Terminal	Hubungan
1	LCD I ² C	SDA	GPIO21 ESP32
2	LCD I ² C	SCL	GPIO22 ESP32
3	LCD I ² C	VCC	5 V
4	LCD I ² C	GND	GND sistem
5	Tombol UP	Sinyal	GPIO12 ESP32
6	Tombol DOWN	Sinyal	GPIO13 ESP32
7	Tombol OK/START	Sinyal	GPIO14 ESP32
8	Tombol BACK/STOP	Sinyal	GPIO27 ESP32
9	Semua tombol	Terminal kedua	GND sistem
10	Buzzer	Sinyal	GPIO23 ESP32
11	Buzzer	GND	GND sistem

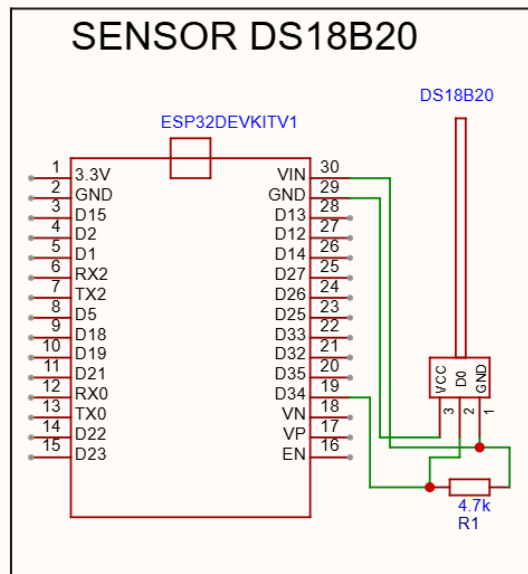
Berdasarkan **Tabel 3.3**, tombol menggunakan konfigurasi *active-low*. Salah satu terminal tombol dihubungkan ke GPIO ESP32, sedangkan terminal lainnya dihubungkan ke GND. Pada program, GPIO dikonfigurasi menggunakan *internal pull-up* sehingga kondisi tombol tidak ditekan terbaca HIGH dan kondisi tombol ditekan terbaca LOW.

LCD menggunakan komunikasi I²C melalui jalur SDA dan SCL. Jalur I²C LCD menggunakan GPIO yang sama dengan RTC DS3231, yaitu GPIO21 untuk SDA dan GPIO22 untuk SCL. Kedua perangkat tetap digunakan pada jalur yang sama karena memiliki alamat I²C yang berbeda.

3.3.3 Rangkaian Sensor Suhu DS18B20

Sensor DS18B20 digunakan untuk mengukur temperatur cairan ekstraksi. Data suhu dari sensor menjadi umpan balik bagi pengendali PID. Sensor DS18B20 dihubungkan ke ESP32 menggunakan komunikasi *1-Wire* melalui satu jalur data.

Rangkaian sensor suhu DS18B20 ditunjukkan pada Gambar 3.4.



Gambar 3. 4 Rangkaian sensor suhu DS18B20

Hubungan sensor DS18B20 dengan ESP32 disajikan pada Tabel 3.4.

Tabel 3. 4 Hubungan sensor DS18B20 dengan ESP32

No.	Terminal DS18B20	Hubungan	Fungsi
1	VDD	3,3 V ESP32	Catu daya sensor
2	DQ	GPIO4 ESP32	Jalur data <i>1-Wire</i>
3	GND	GND sistem	Referensi tegangan

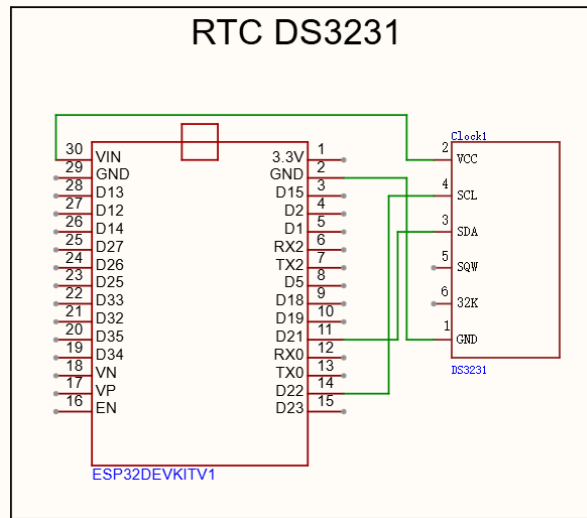
Berdasarkan Tabel 3.4, pin DQ sensor DS18B20 dihubungkan ke GPIO4 ESP32. Resistor *pull-up* sebesar 4,7 k Ω dipasang antara pin DQ dan tegangan 3,3 V agar jalur data berada pada kondisi HIGH ketika tidak terjadi komunikasi.

Sensor ditempatkan pada kotak ekstraksi dengan posisi yang tetap. Ujung sensor tidak ditempelkan langsung pada pelat pendingin agar pembacaan suhu tidak hanya merepresentasikan suhu pelat, tetapi lebih mendekati suhu *box*. Data suhu kemudian digunakan oleh ESP32 untuk menghitung error terhadap nilai *setpoint*.

3.3.4 Rangkaian RTC DS3231

RTC DS3231 digunakan sebagai sumber waktu aktual pada sistem. Data waktu dari RTC digunakan untuk mencatat waktu mulai, menghitung durasi ekstraksi, dan menentukan waktu selesai proses. RTC DS3231 berkomunikasi dengan ESP32 menggunakan antarmuka I²C.

Rangkaian RTC DS3231 ditunjukkan pada Gambar 3.5.



Gambar 3. 5 Rangkaian RTC DS3231

Hubungan RTC DS3231 dengan ESP32 disajikan pada Tabel 3.5.

Tabel 3. 5 Hubungan RTC DS3231 dengan ESP32

No.	Terminal DS3231	Hubungan	Fungsi
1	VCC	3,3 V ESP32	Catu daya modul RTC
2	GND	GND sistem	Referensi tegangan
3	SDA	GPIO21 ESP32	Jalur data I ² C
4	SCL	GPIO22 ESP32	Jalur <i>clock</i> I ² C
5	SQW	Tidak digunakan	Keluaran alarm atau gelombang kotak
6	32K	Tidak digunakan	Keluaran frekuensi 32,768 kHz

Berdasarkan Tabel 3.5, RTC DS3231 menggunakan jalur SDA dan SCL yang sama dengan LCD I²C. Jalur SDA dihubungkan ke GPIO21 ESP32, sedangkan jalur SCL dihubungkan ke GPIO22 ESP32. Terminal SQW dan 32K tidak digunakan karena sistem hanya membutuhkan pembacaan waktu.

RTC DS3231 dilengkapi baterai cadangan sehingga informasi waktu tetap tersimpan ketika catu daya utama terputus. Pada sistem ini, RTC digunakan sebagai acuan waktu proses ekstraksi, sedangkan timer internal ESP32 digunakan untuk

mengatur interval pembacaan sensor, perhitungan PID, pembaruan PWM, dan tampilan LCD.

3.4 Perancangan Perangkat Lunak

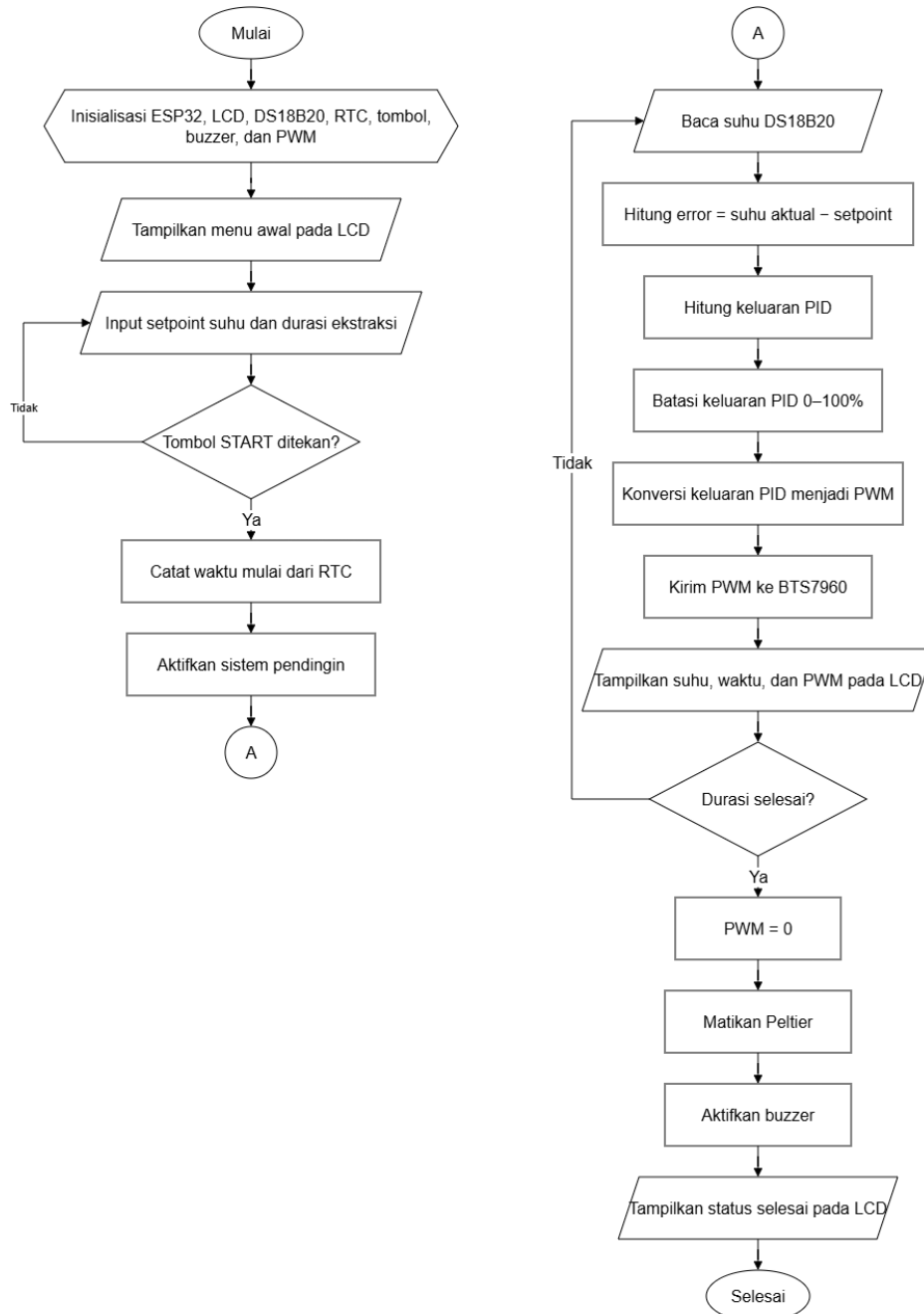
Perancangan perangkat lunak dilakukan untuk mengatur proses kerja sistem pendingin ekstraksi *cold brew coffee* secara terintegrasi. Program dijalankan pada ESP32 sebagai pusat pengendali sistem. Perangkat lunak bertugas membaca temperatur cairan dari sensor DS18B20, membaca waktu dari RTC DS3231, menerima masukan dari *push button*, menghitung keluaran pengendali PID, menghasilkan sinyal PWM untuk driver BTS7960, memperbarui tampilan LCD, serta mengaktifkan buzzer sebagai indikator suara.

Program dirancang dalam beberapa kondisi kerja, yaitu kondisi awal, kondisi pengaturan, kondisi proses berjalan, dan kondisi proses selesai. Pada kondisi awal, ESP32 melakukan inisialisasi seluruh komponen yang digunakan. Pada kondisi pengaturan, pengguna menentukan nilai *setpoint* suhu dan durasi ekstraksi melalui *push button*. Setelah proses dimulai, ESP32 menjalankan pembacaan suhu, pembacaan waktu, perhitungan PID, dan pembaruan keluaran PWM secara periodik.

Pembacaan suhu dari DS18B20 digunakan sebagai data umpan balik pengendali. Sensor DS18B20 menghasilkan data suhu digital melalui komunikasi *1-Wire* dan memiliki waktu konversi yang perlu diperhatikan dalam penentuan interval pembacaan [13]. Waktu proses diperoleh dari RTC DS3231 karena komponen tersebut berfungsi sebagai pencatat waktu aktual dengan antarmuka I²C [14]. Data suhu dan waktu tersebut menjadi dasar bagi ESP32 untuk menentukan kerja Peltier dan durasi proses ekstraksi.

Sinyal kendali aktuator diberikan dalam bentuk PWM. ESP32 menghasilkan PWM menggunakan periferal LEDC yang mendukung pengaturan frekuensi dan resolusi *duty cycle* [11]. Nilai *duty cycle* PWM kemudian dikirimkan ke driver BTS7960. Driver tersebut digunakan karena Peltier membutuhkan daya yang lebih besar daripada kemampuan keluaran langsung ESP32 [15]. ESP32 hanya berfungsi sebagai pengendali, sedangkan daya utama Peltier berasal dari catu daya eksternal.

Alur kerja perangkat lunak sistem ditunjukkan pada Gambar 3.6.



Gambar 3. 6 Diagram alir perangkat lunak sistem

Berdasarkan Gambar 3.6, program dimulai dengan inisialisasi ESP32, LCD, DS18B20, RTC DS3231, *push button*, buzzer, dan PWM. Setelah proses inisialisasi selesai, sistem menampilkan menu awal pada LCD. Pengguna kemudian mengatur nilai *setpoint* suhu dan durasi ekstraksi. Apabila tombol mulai belum ditekan, sistem tetap berada pada menu pengaturan.

Ketika tombol mulai ditekan, ESP32 mencatat waktu awal dari RTC DS3231 dan mengaktifkan sistem pendingin. Selama proses berlangsung, ESP32 membaca suhu aktual dari DS18B20, menghitung error terhadap nilai *setpoint*, menghitung keluaran PID, membatasi keluaran PID pada rentang kerja PWM, kemudian mengirimkan sinyal PWM ke BTS7960. Informasi suhu, waktu, dan PWM diperbarui pada LCD agar kondisi sistem dipantau oleh pengguna.

Proses tersebut berlangsung secara berulang sampai durasi ekstraksi terpenuhi. Setelah durasi selesai, ESP32 menghentikan keluaran PWM dengan memberikan nilai 0, sehingga Peltier tidak lagi menerima daya pendinginan dari driver. Selanjutnya, buzzer diaktifkan dan LCD menampilkan status bahwa proses ekstraksi telah selesai.

3.4.1 Pembacaan Sensor dan Waktu

Pembacaan sensor dan waktu merupakan bagian penting dalam perangkat lunak sistem karena kedua data tersebut digunakan sebagai dasar pengendalian suhu dan penentuan durasi proses ekstraksi. Data suhu diperoleh dari sensor DS18B20, sedangkan data waktu diperoleh dari RTC DS3231. Sensor DS18B20 berfungsi sebagai masukan umpan balik bagi pengendali PID, sementara RTC DS3231 digunakan sebagai acuan waktu mulai, waktu berjalan, dan waktu selesai proses ekstraksi *cold brew coffee*.

Pembacaan suhu dilakukan secara periodik oleh ESP32. Sensor DS18B20 tidak dibaca secara terus-menerus tanpa jeda karena sensor memerlukan waktu konversi sebelum menghasilkan data suhu yang valid. Pada sistem ini, interval pembacaan suhu dibuat sebesar 1 detik agar nilai suhu yang digunakan dalam perhitungan kendali lebih stabil dan sesuai dengan karakteristik sensor. Nilai suhu yang diperoleh dari DS18B20 kemudian disimpan sebagai suhu aktual.

Suhu aktual dibandingkan dengan nilai *setpoint* yang telah ditentukan oleh pengguna. Selisih antara suhu aktual dan suhu *setpoint* digunakan sebagai nilai error pada pengendali PID. Nilai error tersebut menjadi dasar bagi ESP32 untuk menentukan besar keluaran kendali yang diberikan kepada driver BTS7960 dalam bentuk sinyal PWM. Ketelitian dan kestabilan pembacaan sensor berpengaruh langsung terhadap kerja sistem pendingin.

RTC DS3231 digunakan untuk memperoleh data waktu aktual. Ketika pengguna menekan tombol mulai, ESP32 mencatat waktu awal proses berdasarkan data dari RTC. Selanjutnya, waktu berjalan dihitung dengan membandingkan waktu aktual terhadap waktu awal. Proses ekstraksi dinyatakan selesai apabila waktu berjalan telah mencapai durasi ekstraksi yang ditetapkan oleh pengguna.

Data yang digunakan dalam perangkat lunak sistem disajikan pada Tabel 3.6.

Tabel 3. 6 Data masukan pada perangkat lunak sistem

No.	Data	Sumber	Fungsi
1	Suhu aktual	DS18B20	Umpan balik pengendali PID
2	Waktu aktual	RTC DS3231	Acuan waktu proses ekstraksi
3	Nilai <i>setpoint</i>	Pengaturan pengguna	Target suhu pendinginan
4	Durasi ekstraksi	Pengaturan pengguna	Batas waktu proses ekstraksi
5	Status tombol	<i>Push button</i>	Perintah menu, mulai, dan berhenti

Berdasarkan Tabel 3.6, suhu aktual menjadi data utama dalam proses pengendalian temperatur. Waktu aktual digunakan untuk menjaga agar proses ekstraksi berlangsung sesuai durasi yang telah ditentukan. Nilai *setpoint* dan durasi ekstraksi merupakan parameter yang diatur oleh pengguna sebelum proses dimulai. Sementara itu, status tombol digunakan oleh ESP32 untuk menentukan perpindahan menu dan perintah kerja sistem.

Pada program, pembacaan sensor suhu juga dilengkapi dengan pemeriksaan kondisi data. Apabila sensor tidak terbaca atau menghasilkan nilai yang berada di luar batas wajar, ESP32 tidak mengaktifkan keluaran PWM menuju driver BTS7960. Langkah ini diperlukan agar Peltier tidak bekerja tanpa data suhu yang valid. Kondisi kesalahan tersebut ditampilkan pada LCD dan ditandai melalui buzzer sebagai peringatan kepada pengguna.

RTC DS3231 digunakan sebagai acuan durasi proses, sedangkan timer internal ESP32 digunakan untuk mengatur interval pembacaan sensor, pembaruan

tampilan LCD, pembacaan tombol, dan perhitungan kendali. Pembagian fungsi tersebut membuat pencatatan waktu ekstraksi tetap mengacu pada RTC, sementara proses kendali suhu berjalan secara periodik di dalam program ESP32.

3.4.2 Logika Program dan Keluaran PWM

Logika program dirancang untuk mengatur urutan kerja sistem pendingin otomatis mulai dari kondisi awal, pengaturan parameter, proses pendinginan, hingga proses selesai. Program dijalankan oleh ESP32 secara berulang untuk membaca data masukan, memproses kondisi sistem, dan memberikan keluaran kepada aktuator.

Pada saat sistem dinyalakan, ESP32 melakukan inisialisasi terhadap sensor DS18B20, RTC DS3231, LCD, push button, buzzer, dan pin keluaran PWM. Setelah proses inisialisasi selesai, sistem menampilkan menu pengaturan pada LCD. Pengguna dapat mengatur nilai *setpoint* suhu dan durasi proses menggunakan tombol UP, DOWN, dan OK.

Setelah tombol OK ditekan untuk memulai proses, ESP32 membaca suhu aktual dari sensor DS18B20. Suhu aktual tersebut dibandingkan dengan nilai *setpoint* yang telah ditentukan. Selisih antara suhu aktual dan *setpoint* digunakan oleh program sebagai dasar pengaturan keluaran pendinginan. Perhitungan pengendali PID dijelaskan secara khusus pada Subbab 3.6 agar pembahasan perangkat lunak tidak berulang.

Keluaran program diberikan dalam bentuk nilai PWM dengan rentang 0 sampai 255. Nilai PWM tersebut dikirimkan ke driver BTS7960 untuk mengatur daya kerja modul Peltier. Ketika suhu aktual masih jauh di atas *setpoint*, nilai PWM dibuat lebih besar agar daya pendinginan meningkat. Ketika suhu mulai mendekati *setpoint*, nilai PWM dikurangi agar suhu tidak turun terlalu jauh melewati target.

Hubungan kondisi suhu terhadap keluaran PWM ditunjukkan pada Tabel 3.7.

Tabel 3. 7 Hubungan kondisi suhu terhadap keluaran PWM

No	Kondisi suhu	Kondisi sistem	Keluaran PWM
1	Suhu aktual jauh di atas <i>setpoint</i>	Pendinginan maksimum	PWM tinggi
2	Suhu aktual mendekati <i>setpoint</i>	Pendinginan dikurangi	PWM menurun
3	Suhu aktual berada di sekitar <i>setpoint</i>	Suhu dijaga	PWM menyesuaikan
4	Suhu aktual di bawah <i>setpoint</i>	Pendinginan dikurangi atau dihentikan	PWM rendah atau 0
5	Sensor tidak terbaca	Sistem diamankan	PWM 0

Berdasarkan Tabel 3.7, keluaran PWM berubah mengikuti kondisi suhu aktual. Hal ini bertujuan agar daya pendinginan tidak diberikan secara tetap, tetapi disesuaikan dengan kebutuhan sistem. Pengaturan PWM ini menjadi keluaran akhir dari program sebelum sinyal dikirimkan ke driver BTS7960.

Program juga dilengkapi pembatas keluaran agar nilai PWM tidak melebihi rentang kerja yang digunakan. Nilai minimum PWM adalah 0, sedangkan nilai maksimum PWM adalah 255. Pembatasan ini diperlukan agar sinyal keluaran tetap sesuai dengan kemampuan pembangkitan PWM pada ESP32 dan kebutuhan kendali driver BTS7960.

Selain mengatur PWM, program memeriksa kondisi pembacaan sensor. Apabila sensor DS18B20 tidak terbaca atau menghasilkan data yang tidak valid, ESP32 menghentikan keluaran PWM untuk mencegah Peltier bekerja tanpa umpan balik suhu. Kondisi tersebut ditampilkan pada LCD dan dapat ditandai menggunakan buzzer sebagai peringatan kepada pengguna.

3.4.3 Tampilan LCD dan Indikator Buzzer

LCD digunakan sebagai perangkat keluaran informasi pada sistem. Informasi yang ditampilkan meliputi suhu aktual, suhu *setpoint*, waktu proses, nilai PWM, dan status kerja sistem. LCD 20×4 dipilih karena menyediakan empat baris tampilan sehingga beberapa parameter ditampilkan secara bersamaan. Modul LCD

menggunakan antarmuka I²C melalui ekspander PCF8574 sehingga komunikasi dengan ESP32 dilakukan menggunakan jalur SDA dan SCL [16], [17].

Rancangan tampilan LCD disajikan pada Tabel 3.8.

Tabel 3. 8 Rancangan tampilan LCD 20×4

Baris	Informasi yang ditampilkan
1	Status sistem atau mode kerja
2	Suhu aktual dan suhu <i>setpoint</i>
3	Waktu proses atau sisa waktu ekstraksi
4	Nilai PWM atau status aktuator

Berdasarkan Tabel 3.8, tampilan LCD dirancang agar pengguna memantau kondisi utama sistem tanpa harus membaca data serial. Baris pertama menunjukkan kondisi sistem, seperti mode pengaturan, proses berjalan, atau proses selesai. Baris kedua menampilkan suhu aktual dan suhu target. Baris ketiga digunakan untuk menampilkan waktu proses, sedangkan baris keempat menampilkan nilai PWM atau status kerja Peltier.

Push button digunakan untuk memberikan masukan kepada sistem. Tombol digunakan untuk mengatur nilai *setpoint*, mengatur durasi ekstraksi, memulai proses, dan menghentikan sistem apabila diperlukan. Pada program, tombol dikonfigurasi menggunakan logika *active-low*, sehingga kondisi tombol tidak ditekan terbaca HIGH dan kondisi tombol ditekan terbaca LOW. Penggunaan tombol mekanis juga memerlukan penanganan *debounce* karena kontak tombol menghasilkan perubahan logika sesaat ketika ditekan atau dilepas [18].

Buzzer digunakan sebagai indikator suara. Indikator ini berfungsi memberikan tanda ketika tombol ditekan, proses ekstraksi selesai, atau terjadi kondisi kesalahan seperti sensor suhu tidak terbaca. Buzzer tidak berperan langsung dalam proses pengendalian suhu, tetapi berfungsi sebagai bagian dari antarmuka pengguna. Pengaktifan buzzer diatur oleh ESP32 berdasarkan status sistem yang sedang berlangsung.

Dengan rancangan perangkat lunak tersebut, ESP32 menjalankan fungsi pembacaan sensor, pencatatan waktu, pengendalian PID, pembangkitan PWM,

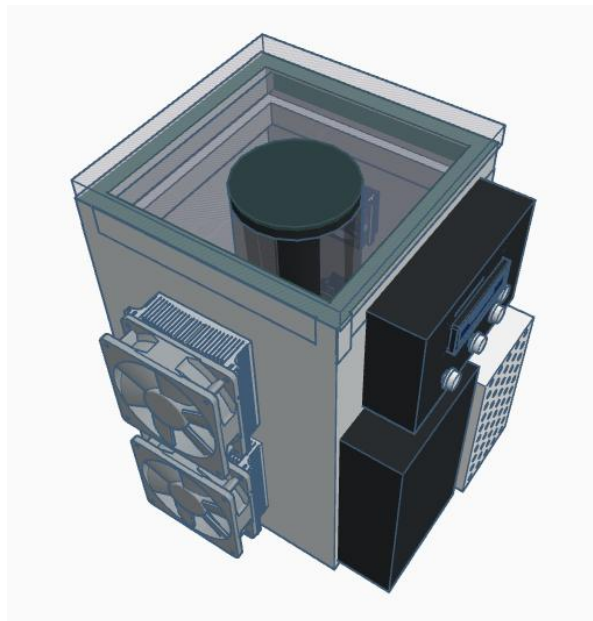
tampilan informasi, dan indikator suara secara terintegrasi. Bagian berikutnya menjelaskan perancangan mekanik dan sistem pendingin karena respons suhu alat dipengaruhi oleh susunan fisik Peltier, pelat dingin, *heatsink*, kipas, posisi sensor, dan isolasi termal.

3.5 Perancangan Mekanik dan Sistem Pendingin

Perancangan mekanik dilakukan untuk menentukan susunan fisik alat pendingin ekstraksi *cold brew coffee*. Bagian mekanik mencakup wadah ekstraksi, posisi Peltier, pelat dingin, *heatsink*, kipas, *thermal paste*, isolasi termal, dan penempatan sensor DS18B20. Perancangan mekanik diperlukan karena respons temperatur sistem tidak hanya dipengaruhi oleh kendali PID, tetapi juga oleh perpindahan panas antara cairan, wadah, Peltier, dan lingkungan.

Pada sistem pendingin termoelektrik, Peltier bekerja dengan memindahkan panas dari sisi dingin menuju sisi panas. Sisi dingin digunakan untuk menyerap panas dari ruang ekstraksi, sedangkan sisi panas harus dilengkapi dengan sistem pelepasan panas agar temperatur sisi panas tidak meningkat secara berlebihan. Penggunaan *heatsink*, kipas, dan *thermal paste* diperlukan untuk membantu proses pelepasan panas pada sisi panas Peltier [2], [10].

Desain mekanik sistem pendingin ditunjukkan pada Gambar 3.7.



Gambar 3. 7 Desain mekanik sistem pendingin ekstraksi *cold brew coffee*

Berdasarkan Gambar 3.7, wadah ekstraksi ditempatkan pada bagian yang berhubungan dengan sisi dingin Peltier melalui pelat penghantar panas. Pelat dingin berfungsi sebagai media perpindahan panas dari wadah atau cairan menuju sisi dingin Peltier. Sisi panas Peltier dipasangkan dengan *heatsink* dan kipas untuk membuang panas ke lingkungan. Bagian ruang ekstraksi diberi isolasi termal agar panas dari lingkungan tidak mudah masuk ke area yang sedang didinginkan.

Komponen mekanik dan sistem pendingin yang digunakan disajikan pada Tabel 3.9.

Tabel 3. 9 Komponen mekanik dan sistem pendingin

No.	Komponen	Fungsi
1	Botol ekstraksi	Menampung cairan <i>cold brew coffee</i> selama proses pendinginan
2	Peltier TEC1-12706	Memindahkan panas dari sisi dingin menuju sisi panas
3	<i>Thermal paste</i>	Mengurangi hambatan termal pada bidang kontak
4	<i>Heatsink</i>	Memperluas permukaan pelepasan panas pada sisi panas Peltier
5	Kipas DC	Meningkatkan aliran udara pada <i>heatsink</i>
6	Dudukan sensor	Menjaga posisi DS18B20 tetap selama pengujian

Berdasarkan Tabel 3.9, setiap komponen memiliki fungsi yang saling berkaitan dalam proses pendinginan. Pelat dingin, Peltier, *heatsink*, dan kipas membentuk jalur perpindahan panas utama. Isolasi termal digunakan untuk mengurangi beban pendinginan dari lingkungan, sedangkan dudukan sensor digunakan agar posisi pembacaan suhu tetap sama pada setiap pengujian.

3.5.1 Desain Konstruksi Alat

Konstruksi alat dirancang agar wadah ekstraksi berada pada area pendinginan yang tetap. Posisi wadah dibuat berdekatan dengan pelat dingin agar panas dari cairan berpindah menuju sisi dingin Peltier. Sambungan antara wadah, pelat dingin, dan Peltier dibuat serapat mungkin untuk mengurangi hambatan termal.

Bagian rangka atau kotak alat digunakan untuk menempatkan komponen elektronik dan komponen pendingin. Komponen elektronik seperti ESP32, driver

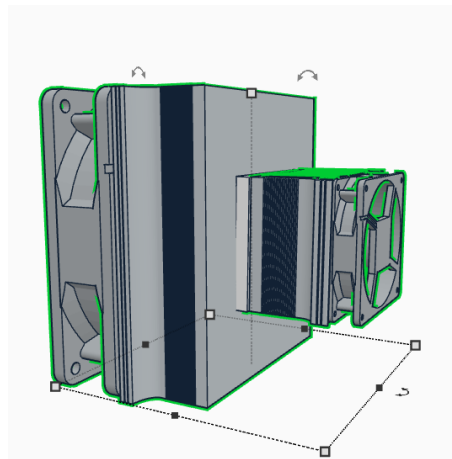
BTS7960, LCD, tombol, dan buzzer ditempatkan pada bagian yang tidak terkena cairan. Pemisahan area elektronik dan area cairan diperlukan untuk mengurangi risiko gangguan akibat kelembapan atau tumpahan selama proses ekstraksi.

Ruang untuk *heatsink* dan kipas dibuat terbuka terhadap udara luar. Hal ini diperlukan karena sisi panas Peltier harus membuang panas ke lingkungan. Apabila aliran udara pada *heatsink* tertutup, panas tertahan dan mengurangi kemampuan pendinginan Peltier. Posisi kipas diarahkan agar udara mengalir melewati sirip-sirip *heatsink*.

3.5.2 Susunan Peltier, Pelat Dingin, *Heatsink*, dan Kipas

Susunan Peltier dibuat dengan memperhatikan arah sisi dingin dan sisi panas. Sisi dingin Peltier ditempelkan pada pelat dingin yang berhubungan dengan wadah ekstraksi. Sisi panas Peltier ditempelkan pada *heatsink* yang dilengkapi kipas. Susunan ini bertujuan agar panas dari cairan diserap oleh sisi dingin dan dibuang melalui sisi panas.

Susunan Peltier, pelat dingin, *heatsink*, dan kipas ditunjukkan pada Gambar 3.8.



Gambar 3. 8 Susunan Peltier, pelat dingin, *heatsink*, dan kipas

Berdasarkan Gambar 3.8, Peltier ditempatkan di antara pelat dingin dan *heatsink*. *Thermal paste* digunakan pada bidang kontak antara Peltier dengan pelat dingin serta antara Peltier dengan *heatsink*. Penggunaan *thermal paste* bertujuan mengisi celah udara kecil pada permukaan kontak agar perpindahan panas berlangsung lebih baik.

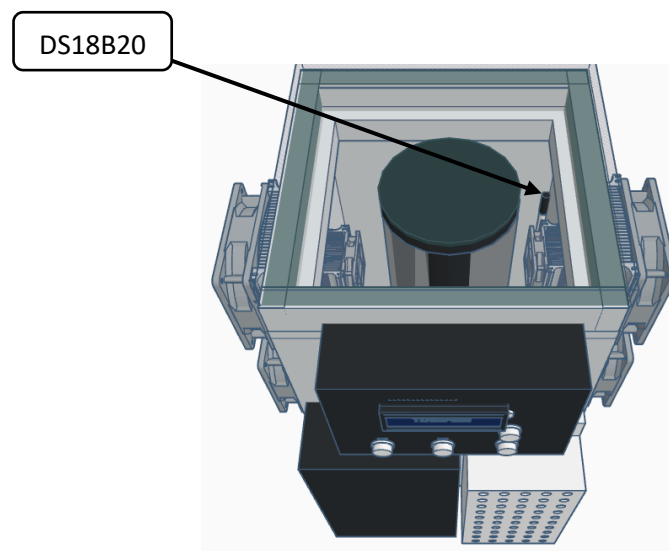
Peltier tipe TEC1-12706 digunakan sebagai aktuator pendingin. Berdasarkan spesifikasinya, modul TEC1-12706 merupakan modul termoelektrik satu tingkat dengan tegangan dan arus kerja yang relatif besar [20]. Peltier tidak dihubungkan langsung ke ESP32, tetapi dikendalikan melalui driver BTS7960 dan catu daya eksternal.

Tekanan pemasangan Peltier perlu dibuat merata. Tekanan yang tidak merata menyebabkan kontak termal tidak optimal dan berpotensi memberikan beban mekanis pada pelat keramik Peltier. Selain itu, bagian sisi panas harus selalu didinginkan menggunakan *heatsink* dan kipas selama Peltier bekerja. Kenaikan temperatur sisi panas menurunkan kemampuan sisi dingin dalam menyerap panas [2], [10].

3.5.3 Posisi Sensor

Sensor DS18B20 ditempatkan pada cairan ekstraksi untuk membaca temperatur aktual. Posisi sensor dibuat tetap pada setiap pengujian agar data suhu yang diperoleh dibandingkan. Ujung sensor tidak ditempelkan langsung pada pelat dingin karena pembacaan tersebut lebih merepresentasikan temperatur pelat daripada temperatur cairan secara umum.

Penempatan sensor DS18B20 pada ruang ekstraksi ditunjukkan pada Gambar 3.9.



Gambar 3. 9 Posisi sensor DS18B20 pada wadah ekstraksi

Berdasarkan Gambar 3.9, ujung sensor ditempatkan pada bagian chamber dengan kedalaman yang tetap. Kedalaman dan posisi sensor perlu dijaga agar perubahan data suhu lebih dipengaruhi oleh respons pendinginan sistem, bukan oleh perubahan posisi sensor. Sensor juga tidak ditempatkan terlalu dekat dengan dinding yang bersentuhan langsung dengan pelat dingin untuk mengurangi kemungkinan pembacaan lokal yang terlalu rendah.

Perancangan mekanik ini menjadi dasar dalam pengujian sistem. Posisi Peltier, pelat dingin, *heatsink*, kipas, sensor, dan isolasi dibuat tetap selama pengujian *open-loop* maupun *closed-loop*. Konsistensi susunan mekanik diperlukan agar data identifikasi sistem dan pengujian PID diperoleh dari kondisi alat yang sama.

3.6 Perancangan dan Penalaan Kontroler PID

Perancangan kontroler PID dilakukan untuk mengatur daya pendinginan modul Peltier agar suhu ekstraksi *cold brew coffee* dapat mencapai dan mempertahankan nilai *setpoint*. Kontroler PID digunakan karena sistem pendingin termoelektrik memiliki respons suhu yang lambat dan dipengaruhi oleh beban termal, suhu lingkungan, kemampuan pelepasan panas, serta isolasi ruang pendingin.

Pada sistem ini, ESP32 berfungsi sebagai pengendali utama. Sensor DS18B20 membaca suhu aktual media ekstraksi. Nilai suhu aktual dibandingkan dengan nilai *setpoint* untuk memperoleh nilai kesalahan atau *error*. Nilai *error* tersebut diproses oleh kontroler PID untuk menghasilkan keluaran kendali berupa nilai PWM. Nilai PWM dikirimkan ke driver BTS7960 untuk mengatur daya kerja modul Peltier.

Penalaan parameter PID dilakukan menggunakan metode Ziegler–Nichols berbasis kurva reaksi proses. Metode ini menggunakan data respons sistem dari pengujian *open-loop*. Pada pengujian tersebut, PID belum diaktifkan dan sistem diberi perubahan masukan berupa PWM dari 0 menjadi 255. Respons suhu terhadap perubahan PWM dicatat terhadap waktu. Data tersebut digunakan untuk menentukan gain proses (K), waktu tunda (L), dan konstanta waktu (T).

Tahapan perancangan dan penalaan kontroler PID terdiri atas pengujian *open-loop*, penentuan parameter K , L , dan T , perhitungan parameter PID berdasarkan

aturan Ziegler–Nichols, serta implementasi parameter PID pada ESP32. Tahapan ini digunakan agar parameter kontroler tidak ditentukan secara acak, tetapi berdasarkan respons aktual sistem pendingin.

3.6.1 Pengujian *Open-Loop* Sistem Pendingin

Pengujian *open-loop* dilakukan untuk mengetahui respons alami sistem pendingin terhadap perubahan masukan PWM. Pada tahap ini, kontroler PID belum diaktifkan. Sistem dijalankan dengan kondisi awal PWM 0 sampai suhu berada pada kondisi relatif stabil. Setelah kondisi awal tercapai, nilai PWM diubah menjadi 255 sehingga modul Peltier bekerja pada daya maksimum.

Selama pengujian berlangsung, ESP32 mencatat data waktu, nilai PWM, dan suhu hasil pembacaan sensor DS18B20. Data suhu yang digunakan merupakan suhu hasil kalibrasi sensor agar respons sistem lebih mendekati kondisi aktual. Pengujian *open-loop* dilakukan untuk memperoleh kurva penurunan suhu terhadap waktu.

Masukan sistem pada pengujian ini adalah nilai PWM yang diberikan kepada driver BTS7960, sedangkan keluaran sistem adalah suhu media ekstraksi yang dibaca oleh sensor DS18B20. Hubungan masukan dan keluaran tersebut digunakan untuk mengetahui karakteristik pendinginan sistem.

Parameter pengujian *open-loop* ditunjukkan pada Tabel 3.10.

Tabel 3. 10 Parameter pengujian *open-loop* sistem pendingin

No	Parameter	Keterangan
1	Mode pengujian	<i>Open-loop</i>
2	Kondisi awal	PWM 0 sampai suhu stabil
3	Masukan step	PWM 0 menjadi PWM 255
4	Aktuator	Modul Peltier melalui driver BTS7960
5	Sensor	DS18B20
6	Data yang dicatat	Waktu, PWM, dan suhu
7	Tujuan pengujian	Menentukan respons suhu untuk memperoleh K , L , dan T

Pengujian *open-loop* menghasilkan grafik respons suhu terhadap waktu. Grafik tersebut digunakan sebagai dasar penentuan parameter sistem. Pada sistem pendingin, peningkatan nilai PWM menyebabkan peningkatan daya pendinginan sehingga suhu media ekstraksi mengalami penurunan.

3.6.2 Penentuan Parameter K, L, dan T

Parameter K , L , dan T diperoleh dari grafik respons suhu hasil pengujian *open-loop*. Parameter K merupakan gain proses yang menunjukkan perbandingan antara perubahan keluaran dan perubahan masukan. Pada sistem ini, keluaran berupa perubahan suhu, sedangkan masukan berupa perubahan nilai PWM.

Gain proses dihitung menggunakan Persamaan (3.1).

$$K = \left| \frac{\Delta T}{\Delta PWM} \right| \quad (3.1)$$

Keterangan:

K = gain proses

ΔT = perubahan suhu dari kondisi awal menuju kondisi akhir ($^{\circ}C$)

ΔPWM = perubahan nilai PWM yang diberikan ke driver

Tanda mutlak digunakan karena sistem bekerja sebagai pendingin. Saat PWM dinaikkan, suhu mengalami penurunan sehingga perubahan suhu dapat bernilai negatif. Perhitungan parameter PID menggunakan nilai gain positif agar hasil parameter kontroler dapat diterapkan pada program ESP32.

Waktu tunda (L) merupakan waktu yang menunjukkan keterlambatan respons sistem setelah masukan PWM diberikan. Konstanta waktu (T) menunjukkan waktu yang dibutuhkan sistem untuk bergerak dari awal respons menuju daerah karakteristik utama respons sistem. Nilai L dan T ditentukan menggunakan metode garis singgung pada kurva reaksi proses.

Langkah penentuan L dan T dilakukan sebagai berikut:

1. Membuat grafik suhu terhadap waktu dari data pengujian *open-loop*.
2. Menentukan bagian kurva yang memiliki kemiringan terbesar.
3. Membuat garis singgung pada bagian kurva tersebut.

4. Menentukan titik potong garis singgung dengan garis suhu awal sebagai nilai L .
5. Menentukan titik potong garis singgung dengan garis suhu akhir sebagai nilai $L + T$.
6. Menghitung nilai T dari selisih antara $L + T$ dan L .

Nilai T dihitung menggunakan Persamaan (3.2).

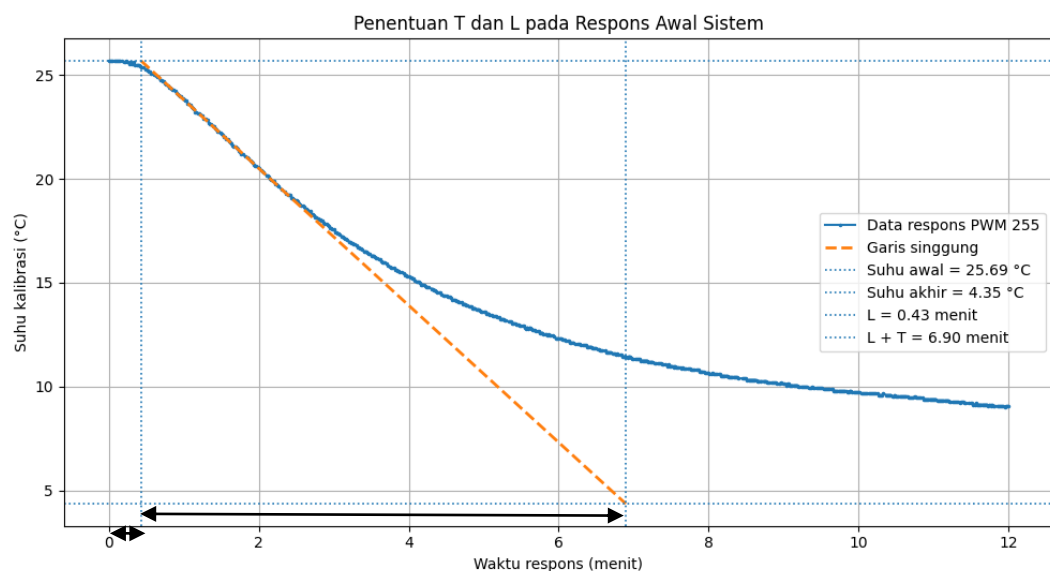
$$T = (L + T) - L \quad (3.2)$$

Keterangan:

T = konstanta waktu sistem

L = waktu tunda sistem

$L + T$ = titik potong garis singgung terhadap garis suhu akhir



Gambar 3. 10 Penentuan nilai L dan T menggunakan metode garis singgung

Pada pengujian yang dilakukan, terdapat segmen respons yang mengalami gangguan berupa kenaikan suhu ketika PWM tetap berada pada nilai 255. Segmen tersebut tidak digunakan dalam penentuan L dan T karena tidak merepresentasikan respons pendinginan normal sistem. Penentuan L dan T dilakukan pada respons awal sistem sebelum gangguan terjadi.

3.6.3 Perhitungan Parameter PID Menggunakan Metode Ziegler–Nichols

Setelah nilai K , L , dan T diperoleh, parameter kontroler PID dihitung menggunakan aturan Ziegler–Nichols metode kurva reaksi proses. Parameter yang dihitung meliputi konstanta proporsional (K_p), waktu integral (T_i), dan waktu derivatif (T_d).

Untuk kontroler PID, aturan Ziegler–Nichols ditunjukkan pada Persamaan (3.3), Persamaan (3.4), dan Persamaan (3.5).

$$K_p = 1,2 \frac{T}{KL} \quad (3.3)$$

$$T_i = 2L \quad (3.4)$$

$$T_d = 0,5L \quad (3.5)$$

Keterangan:

K_p = konstanta proporsional

T_i = waktu integral

T_d = waktu derivatif

K = gain proses

L = waktu tunda

T = konstanta waktu

Nilai T_i dan T_d kemudian dikonversi menjadi konstanta integral (K_i) dan konstanta derivatif (K_d). Konversi tersebut ditunjukkan pada Persamaan (3.6) dan Persamaan (3.7).

$$K_i = \frac{K_p}{T_i} \quad (3.6)$$

$$K_d = K_p \times T_d \quad (3.7)$$

Keterangan:

K_i = konstanta integral

K_d = konstanta derivatif

Hasil perhitungan K_p , K_i , dan K_d digunakan sebagai nilai awal parameter kontroler PID. Nilai tersebut diterapkan pada ESP32 untuk mengatur keluaran

PWM berdasarkan selisih antara suhu aktual dan suhu *setpoint*. Apabila respons sistem masih terlalu agresif atau menghasilkan fluktuasi besar, nilai parameter dapat disesuaikan pada tahap pengujian *closed-loop* dengan tetap menjadikan hasil Ziegler–Nichols sebagai dasar penalaan awal.

3.6.4 Implementasi PID pada ESP32

Implementasi PID dilakukan pada program ESP32. Program membaca suhu aktual dari sensor DS18B20, kemudian membandingkannya dengan suhu *setpoint*. Selisih antara suhu aktual dan *setpoint* digunakan sebagai nilai *error*.

Pada sistem pendingin ini, *error* didefinisikan menggunakan Persamaan (3.8).

$$e(k) = T_{aktual}(k) - T_{setpoint} \quad (3.8)$$

Keterangan:

$e(k)$ = *error* suhu pada pembacaan ke- k

$T_{aktual}(k)$ = suhu aktual hasil pembacaan sensor pada pembacaan ke- k

$T_{setpoint}$ = suhu target yang ditentukan

Definisi *error* tersebut digunakan karena sistem bekerja untuk menurunkan suhu. Saat suhu aktual lebih tinggi daripada *setpoint*, nilai *error* bernilai positif. Kondisi tersebut menyebabkan kontroler meningkatkan nilai PWM agar daya pendinginan bertambah. Saat suhu aktual mendekati *setpoint*, nilai PWM dikurangi agar suhu tidak turun terlalu jauh.

Perhitungan PID diskrit pada program ESP32 ditunjukkan pada Persamaan (3.9).

$$u(k) = K_p e(k) + K_i \sum e(k) \Delta t + K_d \frac{e(k) - e(k-1)}{\Delta t} \quad (3.9)$$

Keterangan:

$u(k)$ = keluaran kontroler pada pembacaan ke- k

$e(k)$ = *error* saat ini

$e(k-1)$ = *error* sebelumnya

Δt = interval waktu pembacaan sensor

Keluaran kontroler PID dibatasi pada rentang 0 sampai 255. Nilai 0 menunjukkan Peltier tidak diberi daya, sedangkan nilai 255 menunjukkan Peltier

bekerja pada daya maksimum. Pembatasan keluaran diperlukan agar nilai PID sesuai dengan rentang PWM yang digunakan pada ESP32.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Pengujian Awal dan Evaluasi Perbaikan Sistem Pendingin

4.1.1 Pengujian Kondisi Awal Sistem Pendingin

Pengujian kondisi awal dilakukan menggunakan dua unit Peltier dan ruang pendingin berbahan styrofoam biasa. Pengujian dilakukan pada ruangan tanpa AC dengan suhu lingkungan sekitar 29 °C selama 120 menit. Kondisi pengujian ditunjukkan pada Tabel 4.1.

Tabel 4. 1 Kondisi Pengujian Awal Sistem Pendingin

No.	Parameter Pengujian	Kondisi
1	Jumlah Peltier	2 unit
2	Material isolasi	Styrofoam biasa
3	Kondisi ruangan	Ruangan tanpa AC
4	Suhu lingkungan	29 °C
5	Durasi pengujian	120 menit
6	Interval pengambilan data	20 menit
7	Kondisi kerja Peltier	Aktif maksimal

Berdasarkan Tabel 4.1, pengujian awal dilakukan menggunakan dua Peltier pada ruangan tanpa AC. Penggunaan styrofoam biasa dan suhu lingkungan sekitar 29 °C merupakan kondisi awal sebelum dilakukan perbaikan rancangan.

Data perubahan suhu terhadap waktu pada pengujian awal ditunjukkan pada Tabel 4.2.

Tabel 4. 2 Data Suhu terhadap Waktu pada Kondisi Awal

No.	Waktu (menit)	Suhu Aktual (°C)
1	0	28,89
2	10	20,75
3	20	20,12

No.	Waktu (menit)	Suhu Aktual (°C)
4	30	19,56
5	40	19,50
6	50	19,62
7	60	19,50
8	70	19,81
9	80	19,69
10	90	19,44
11	100	19,50
12	110	19,56
13	120	19,44

Berdasarkan Tabel 4.1, suhu awal sistem sebesar 28,89 °C. Pada 10 menit pertama, suhu turun menjadi 20,75 °C atau mengalami penurunan sebesar 8,14 °C. Suhu kemudian turun secara bertahap hingga mencapai 19,56 °C pada menit ke-30.

Setelah menit ke-30, suhu cenderung berada pada rentang 19,44–19,81 °C. Suhu minimum sebesar 19,44 °C pertama kali tercapai pada menit ke-90 dan kembali tercatat pada menit ke-120. Kondisi tersebut menunjukkan bahwa kemampuan pendinginan mulai mencapai batasnya setelah sekitar 30–40 menit pengujian.

Penurunan suhu dihitung menggunakan persamaan:

$$\Delta T = T_{awal} - T_{min}$$

$$\Delta T = 28,89 - 19,44$$

$$\Delta T = 9,45 \text{ }^{\circ}\text{C}$$

Hasil perhitungan menunjukkan bahwa sistem mampu menghasilkan penurunan suhu sebesar 9,45 °C.

Karena suhu minimum pertama kali tercapai pada menit ke-90, laju penurunan suhu rata-rata menuju suhu minimum dihitung sebagai berikut:

$$\text{Laju penurunan suhu} = \Delta T / t$$

$$\text{Laju penurunan suhu} = 9,45 / 90$$

$$\text{Laju penurunan suhu} = 0,105 \text{ }^{\circ}\text{C}/\text{menit}$$

Apabila dihitung berdasarkan keseluruhan durasi 120 menit, laju penurunan suhu rata-ratanya adalah:

$$\text{Laju penurunan suhu keseluruhan} = 9,45 / 120$$

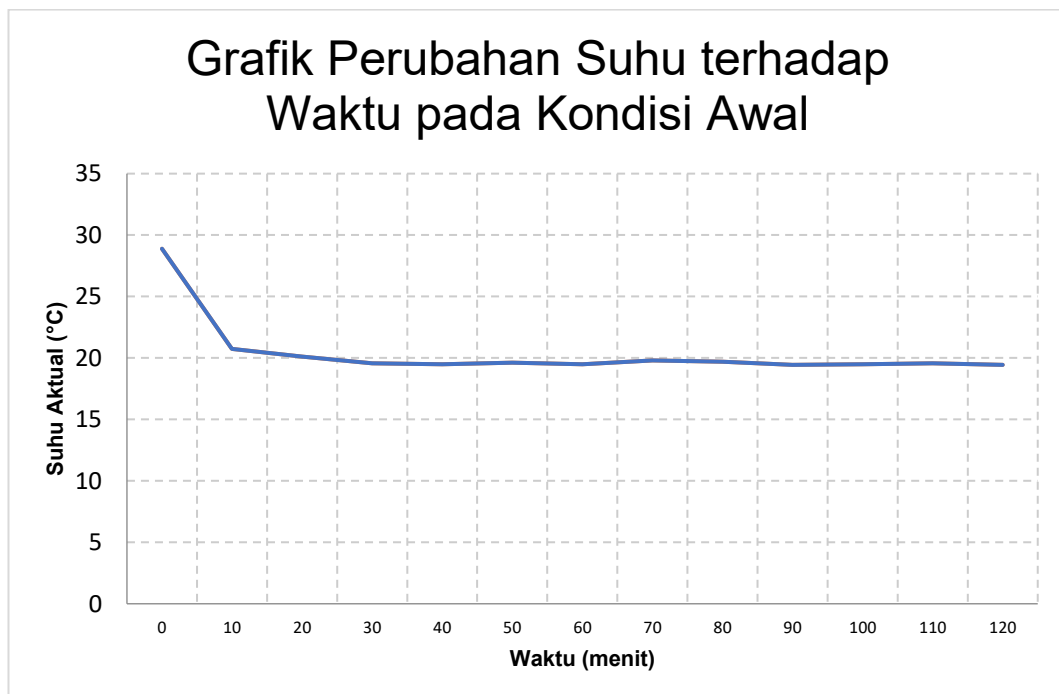
$$\text{Laju penurunan suhu keseluruhan} = 0,079 \text{ } ^\circ\text{C}/\text{menit}$$

Penurunan suhu terbesar terjadi pada 10 menit pertama, yaitu sebesar 8,14 °C. Nilai tersebut setara dengan sekitar 86,14% dari total penurunan suhu selama pengujian. Setelah itu, laju penurunan suhu semakin kecil.

Pada menit ke-40 sampai menit ke-120, suhu rata-rata tercatat sekitar 19,56 °C dengan rentang perubahan sebesar 0,37 °C. Hal ini menunjukkan bahwa sistem telah mendekati kondisi keseimbangan termal, yaitu kapasitas pendinginan Peltier hampir seimbang dengan panas yang masuk dari lingkungan.

Meskipun sistem mampu menurunkan suhu sebesar 9,45 °C, suhu minimum 19,44 °C masih berada di atas *setpoint* penelitian, yaitu 5 °C, 8 °C, dan 12 °C. Konfigurasi awal menggunakan dua unit Peltier dan styrofoam biasa belum mampu memenuhi kebutuhan pendinginan ekstraksi *cold brew coffee*.

Data pada Tabel 4.1 selanjutnya dapat disajikan dalam grafik berikut.



Gambar 4. 1 Grafik Perubahan Suhu terhadap Waktu pada Kondisi Awal Sistem Pendingin

4.1.2 Pengujian Sistem Pendingin Setelah Perbaikan

Perbaikan sistem dilakukan dengan menambah jumlah Peltier dari dua unit menjadi empat unit dan mengganti material styrofoam biasa dengan PU foam. Pengujian dilakukan pada ruangan ber-AC dengan suhu lingkungan sekitar 24–25 °C selama 120 menit. Kondisi pengujian setelah perbaikan ditunjukkan pada Tabel 4.3.

Tabel 4. 3 Kondisi Pengujian Sistem Pendingin Setelah Perbaikan

No.	Parameter Pengujian	Kondisi
1	Jumlah Peltier	4 unit
2	Material isolasi	PU foam
3	Kondisi ruangan	Ruangan ber-AC
4	Suhu lingkungan	25,9 °C
5	Durasi pengujian	120 menit
6	Interval pengambilan data	10 menit
7	Kondisi kerja Peltier	Aktif maksimal

Berdasarkan Tabel 4.3, pengujian setelah perbaikan dilakukan menggunakan empat Peltier dan PU foam. Pengujian dilaksanakan pada ruangan ber-AC sehingga kondisi lingkungan berbeda dengan pengujian awal.

Data perubahan suhu terhadap waktu setelah perbaikan ditunjukkan pada Tabel 4.4.

Tabel 4. 4 Data Suhu terhadap Waktu Setelah Perbaikan

Waktu (menit)	Suhu (°C)
0	25,69
10	12,05
20	9,33
30	7,53
40	6,80
50	6,40
60	5,73

Waktu (menit)	Suhu (°C)
70	4,87
80	4,47
90	4,27
100	4,27
110	4,27
120	4,27

Berdasarkan hasil pengujian, sistem pendingin setelah perbaikan mampu menurunkan suhu dari 25,69 °C menjadi 4,27 °C. Penurunan suhu total yang dihasilkan sebesar 21,42 °C. Suhu minimum pertama kali tercapai pada menit ke-90, sehingga laju penurunan suhu rata-rata menuju suhu minimum adalah 0,24 °C/menit.

Penurunan suhu paling besar terjadi pada 10 menit pertama, yaitu sebesar 13,64 °C. Setelah menit ke-10, laju penurunan suhu semakin kecil hingga sistem mencapai suhu minimum. Berdasarkan data pengukuran, suhu mendekati 12 °C pada menit ke-10, berada di bawah 8 °C pada menit ke-30, dan berada di bawah 5 °C pada menit ke-70.

Pada menit ke-90 sampai menit ke-120, suhu tetap tercatat sebesar 4,27 °C. Hasil tersebut menunjukkan bahwa sistem telah mencapai batas pendinginan pada kondisi pengujian dan mampu mempertahankan suhu minimum selama 30 menit terakhir. Rancangan setelah perbaikan dapat digunakan untuk mencapai rentang *setpoint* ekstraksi 5 °C, 8 °C, dan 12 °C.

4.1.3 Evaluasi Hasil Perbaikan Sistem Pendingin

Hasil perhitungan dari kedua pengujian dirangkum pada Tabel 4.5.

Tabel 4. 5 Perbandingan Hasil Pengujian Sebelum dan Setelah Perbaikan

No.	Parameter Hasil	Kondisi Awal	Setelah Perbaikan
1	Suhu awal	28,89 °C	25,69 °C
2	Suhu minimum	19,44 °C	4,27 °C
3	Penurunan suhu	9,45 °C	21,42 °C

No.	Parameter Hasil	Kondisi Awal	Setelah Perbaikan
4	Waktu pertama mencapai suhu minimum	90 menit	90 menit
5	Durasi pengujian	120 menit	120 menit
6	Laju penurunan suhu rata-rata menuju suhu minimum	0,11 °C/menit	0,24 °C/menit
7	Jumlah Peltier	2 unit	4 unit
8	Material isolasi	Styrofoam biasa	PU foam
9	Kondisi ruangan	Tanpa AC	Menggunakan AC
10	Suhu lingkungan	Sekitar 30 °C	Sekitar 24–25 °C
11	Kemampuan mencapai 12 °C	Tidak tercapai	Tercapai
12	Kemampuan mencapai 8 °C	Tidak tercapai	Tercapai
13	Kemampuan mencapai 5 °C	Tidak tercapai	Tercapai
14	Kondisi suhu menit ke-90–120	19,44–19,56 °C	Tetap 4,27 °C

Berdasarkan Tabel 4.5, konfigurasi awal mampu menurunkan suhu dari 28,89 °C menjadi 19,44 °C. Penurunan suhu yang diperoleh sebesar 9,45 °C. Sementara itu, sistem setelah perbaikan mampu menurunkan suhu dari 25,69 °C menjadi 4,27 °C dengan penurunan suhu sebesar 21,42 °C.

Laju penurunan suhu rata-rata dihitung berdasarkan waktu ketika suhu minimum pertama kali tercapai, yaitu pada menit ke-90.

Untuk kondisi awal:

$$\text{Laju penurunan suhu} = 9,45 / 90 = 0,105 \text{ °C/menit}$$

Dibulatkan menjadi:

$$\text{Laju penurunan suhu} = 0,11 \text{ °C/menit}$$

Untuk kondisi setelah perbaikan:

$$\text{Laju penurunan suhu} = 21,42 / 90 = 0,238 \text{ °C/menit}$$

Dibulatkan menjadi:

$$\text{Laju penurunan suhu} = 0,24 \text{ °C/menit}$$

Selisih penurunan suhu antara kedua konfigurasi dihitung sebagai berikut:

$$\text{Selisih penurunan suhu} = 21,42 - 9,45$$

$$\text{Selisih penurunan suhu} = 11,97 \text{ °C}$$

Secara deskriptif, peningkatan kemampuan penurunan suhu dihitung sebagai berikut:

$$\text{Persentase peningkatan} = (11,97 / 9,45) \times 100\%$$

$$\text{Persentase peningkatan} = 126,67\%$$

Hasil tersebut menunjukkan bahwa kemampuan penurunan suhu sistem setelah perbaikan lebih besar sekitar 126,67% dibandingkan konfigurasi awal. Nilai ini merupakan perbandingan performa keseluruhan dan tidak dapat dinyatakan hanya sebagai pengaruh penambahan jumlah Peltier. Pada kedua pengujian terdapat perubahan jumlah Peltier, material isolasi, dan suhu lingkungan.

Pada kondisi awal, suhu mulai relatif stabil pada rentang 19,44–19,81 °C setelah menit ke-30. Suhu tersebut masih berada di atas seluruh *setpoint* penelitian. Konfigurasi dua Peltier dan styrofoam biasa belum mampu mencapai *setpoint* 12 °C, 8 °C, maupun 5 °C.

Setelah perbaikan, sistem mampu mencapai suhu di bawah 12 °C pada menit ke-20, di bawah 8 °C pada menit ke-30, dan di bawah 5 °C pada menit ke-70. Suhu minimum sebesar 4,27 °C pertama kali tercapai pada menit ke-90 dan tetap tercatat sampai menit ke-120. Hasil ini menunjukkan bahwa rancangan setelah perbaikan telah mampu mencapai seluruh *setpoint* yang digunakan dalam penelitian.

Peningkatan kinerja diperoleh setelah jumlah Peltier ditambah dari dua menjadi empat unit, material isolasi diganti dari styrofoam biasa menjadi PU foam, dan pengujian dilakukan pada ruangan ber-AC. Hasil pengujian menunjukkan peningkatan kinerja konfigurasi sistem secara keseluruhan, bukan pengaruh satu komponen secara terpisah.

4.2 Hasil Kalibrasi Sensor DS18B20

Kalibrasi sensor DS18B20 dilakukan untuk menyesuaikan hasil pembacaan sensor terhadap alat ukur referensi. Pada pengujian ini, sensor DS18B20 dibandingkan dengan thermometer digital. Kalibrasi diperlukan karena suhu yang terbaca oleh DS18B20 digunakan sebagai suhu aktual pada sistem kendali PID. Apabila pembacaan sensor memiliki selisih terhadap suhu referensi, maka error PID yang dihitung oleh ESP32 juga terpengaruh.

Data kalibrasi diperoleh dari pembacaan suhu mentah DS18B20 dan suhu referensi. Titik interpolasi tersebut berada pada garis regresi, sehingga tidak mengubah nilai koefisien kalibrasi yang telah diperoleh.

Hasil data kalibrasi sensor DS18B20 ditunjukkan pada Tabel 4.6.

Tabel 4. 6 Data kalibrasi sensor DS18B20 terhadap suhu referensi

No.	Suhu DS18B20 mentah, $x(^{\circ}\text{C})$	Suhu Termometer Digital, $y(^{\circ}\text{C})$
1	24,00	24,70
2	18,25	18,80
3	17,50	18,30
4	15,50	16,50
5	12,75	13,70
6	10,25	11,10
7	3,50	4,20
8	3,00	3,20
9	8,00	8,6
10	20,00	20,8

Error suhu setelah kalibrasi dihitung menggunakan **Persamaan (4.1)**.

$$e_T = T_{\text{mentah}} - T_{\text{referensi}} \quad (4.1)$$

Keterangan:

e_T = error suhu setelah kalibrasi

T_{mentah} = suhu DS18B20 sebelum dikalibrasi

$T_{\text{referensi}}$ = suhu referensi

Berdasarkan Tabel 4.3, dari hasil pembacaan DS18B20 sebelum kalibrasi, nilai error terbesar pada data pengukuran terjadi pada suhu mentah 15,50 °C dengan error sebesar 1,00°C dan rata-rata errornya sebesar 0,715

Berdasarkan Tabel 4.3, nilai x merupakan suhu mentah yang terbaca oleh sensor DS18B20, sedangkan nilai y merupakan suhu referensi. Data tersebut

kemudian diolah menggunakan regresi linear untuk memperoleh persamaan kalibrasi sensor. Bentuk umum regresi linear ditunjukkan pada **Persamaan (4.1)**.

$$y = ax + b \quad (4.1)$$

Keterangan:

y = suhu referensi

x = suhu mentah DS18B20

a = koefisien kemiringan garis regresi

b = konstanta offset

Nilai koefisien a dihitung menggunakan **Persamaan (4.2)**.

$$a = \frac{n\sum xy - \sum x \sum y}{n\sum x^2 - (\sum x)^2} \quad (4.2)$$

Nilai konstanta b dihitung menggunakan **Persamaan (4.3)**.

$$b = \bar{y} - a\bar{x} \quad (4.3)$$

Keterangan:

n = jumlah data pengukuran

$\sum x$ = jumlah data suhu DS18B20 mentah

$\sum y$ = jumlah data suhu referensi

$\sum x^2$ = jumlah kuadrat data suhu DS18B20 mentah

$\sum xy$ = jumlah hasil perkalian x dan y

$\bar{x}$ = rata-rata suhu DS18B20 mentah

$\bar{y}$ = rata-rata suhu referensi

Dari **Tabel 4.2** diperoleh nilai:

$$n = 10$$

$$\sum x = 132,75$$

$$\sum y = 139,9614$$

$$\sum x^2 = 2208,4375$$

$$\sum xy = 2310,0600$$

Nilai koefisien a dihitung sebagai berikut.

$$a = \frac{10(2310,0600) - (132,75)(139,9614)}{10(2208,4375) - (132,75)^2}$$

$$a = \frac{23100,6000 - 18579,8759}{22084,3750 - 17622,5625}$$

$$a = \frac{4520,7241}{4461,8125}$$

$$a = 1,0132$$

Selanjutnya, nilai rata-rata x dan y dihitung sebagai berikut.

$$\bar{x} = \frac{\sum x}{n}$$

$$\bar{x} = \frac{132,75}{10} = 13,2750$$

$$\bar{y} = \frac{\sum y}{n}$$

$$\bar{y} = \frac{139,9614}{10} = 13,9961$$

Nilai konstanta b dihitung menggunakan **Persamaan (4.3)**.

$$b = \bar{y} - a\bar{x}$$

$$b = 13,9961 - (1,0132)(13,2750)$$

$$b = 13,9961 - 13,4502$$

$$b = 0,5459$$

Berdasarkan hasil perhitungan tersebut, diperoleh persamaan kalibrasi sensor DS18B20 seperti ditunjukkan pada **Persamaan (4.4)**.

$$T_{\text{kalibrasi}} = 1,0132 \times T_{\text{mentah}} + 0,5459 \quad (4.4)$$

Keterangan:

$T_{\text{kalibrasi}}$ = suhu DS18B20 setelah dikalibrasi

T_{mentah} = suhu awal yang terbaca oleh sensor DS18B20

1,0132 = koefisien pengali hasil regresi linear

0,5459 = nilai offset hasil regresi linear

Persamaan (4.4) digunakan untuk mengubah suhu mentah DS18B20 menjadi suhu hasil kalibrasi. Nilai suhu yang digunakan pada sistem bukan suhu

mentah secara langsung, melainkan suhu yang telah dikoreksi berdasarkan persamaan kalibrasi.

Hasil penerapan persamaan kalibrasi terhadap data pengujian ditunjukkan pada Tabel 4.4.

Tabel 4. 7 Hasil suhu DS18B20 setelah kalibrasi

No.	Suhu DS18B20 mentah (°C)	Suhu referensi (°C)	Suhu setelah kalibrasi (°C)	Error (°C)
1	24,00	24,70	24,86	0,16
2	18,25	18,80	19,04	0,24
3	17,50	18,30	18,28	0,02
4	15,50	16,50	16,25	0,25
5	12,75	13,70	13,46	0,24
6	10,25	11,10	10,93	0,17
7	3,50	4,20	4,09	0,11
8	3,00	3,20	3,59	0,39
9	8,00	8,60	8,60	0,00
10	20,00	20,80	20,80	0,00

Error suhu setelah kalibrasi dihitung menggunakan **Persamaan (4.5)**.

$$e_T = T_{\text{kalibrasi}} - T_{\text{referensi}} \quad (4.5)$$

Keterangan:

e_T = error suhu setelah kalibrasi

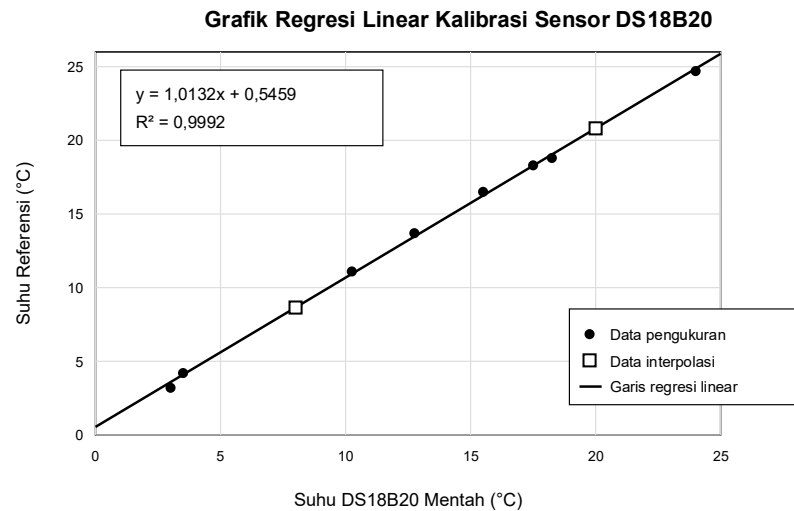
$T_{\text{kalibrasi}}$ = suhu DS18B20 setelah dikalibrasi

$T_{\text{referensi}}$ = suhu referensi

Berdasarkan Tabel 4.4, hasil pembacaan DS18B20 setelah kalibrasi sudah mendekati nilai suhu referensi. Selisih pembacaan masih terdapat pada beberapa titik, tetapi nilainya relatif kecil. Nilai error terbesar pada data pengukuran terjadi pada suhu mentah 3,00 °C dengan error sebesar 0,39 °C dan rata-rata errornya 0,158

°C. Hasil ini masih diterima untuk sistem pendingin yang menggunakan DS18B20 sebagai sensor umpan balik.

Grafik hubungan antara suhu mentah DS18B20 dan suhu referensi ditunjukkan pada Gambar 4.2.



Gambar 4. 2 Grafik regresi linear kalibrasi sensor DS18B20

Berdasarkan **Gambar 4.2**, hubungan antara suhu mentah Dlam S18B20 dan suhu referensi didekati menggunakan persamaan linear. Persamaan tersebut digunakan sebagai koreksi pembacaan sensor pada program ESP32. Dengan adanya kalibrasi ini, suhu aktual yang digunakan dalam perhitungan error PID menjadi lebih sesuai dengan suhu referensi.

Pada sistem pendingin, suhu hasil kalibrasi digunakan pada pengujian *open-loop* dan *closed-loop*. Pada pengujian *open-loop*, suhu hasil kalibrasi digunakan sebagai data keluaran sistem untuk proses identifikasi fungsi alih. Pada pengujian *closed-loop*, suhu hasil kalibrasi digunakan sebagai suhu aktual yang dibandingkan dengan suhu *setpoint*. Proses kalibrasi sensor DS18B20 menjadi tahap penting sebelum sistem PID diterapkan pada alat.

4.3 Hasil Pengujian RTC DS3231

Pengujian RTC DS3231 dilakukan untuk mengetahui kemampuan modul RTC dalam membaca dan mempertahankan data waktu. RTC DS3231 digunakan pada sistem untuk menghitung durasi proses ekstraksi *cold brew coffee*. Waktu yang

terbaca dari RTC digunakan sebagai acuan untuk menentukan lama proses pendinginan dan ekstraksi berlangsung.

Pengujian dilakukan dengan membandingkan waktu yang terbaca pada RTC DS3231 terhadap waktu acuan. Waktu acuan yang digunakan berupa jam pada laptop, smartphone, atau jam digital. Data yang dibandingkan meliputi jam, menit, dan detik. Selain itu, pengujian juga dilakukan untuk melihat apakah RTC tetap menyimpan waktu ketika sistem dimatikan sementara, karena modul DS3231 dilengkapi baterai cadangan untuk menjaga data waktu tetap berjalan [14].

Hasil pengujian RTC DS3231 ditunjukkan pada Tabel 4.8.

Tabel 4. 8 Hasil pengujian RTC DS3231

No	Waktu Acuan Laptop	Waktu RTC DS3231	Selisih Waktu	Keterangan
1	02:57:00	02:57:26	+26 detik	Stabil
2	02:57:01	02:57:27	+26 detik	Stabil
3	02:57:02	02:57:28	+26 detik	Stabil
4	02:57:03	02:57:29	+26 detik	Stabil
5	02:57:04	02:57:30	+26 detik	Stabil
6	02:57:05	02:57:31	+26 detik	Stabil
7	02:57:06	02:57:32	+26 detik	Stabil
8	02:57:07	02:57:33	+26 detik	Stabil
9	02:57:08	02:57:34	+26 detik	Stabil
10	02:57:09	02:57:35	+26 detik	Stabil

Berdasarkan Tabel 4.5, pengujian dilakukan untuk memastikan bahwa waktu yang terbaca pada RTC DS3231 sesuai dengan waktu acuan. Selisih waktu dihitung dari perbedaan antara waktu RTC dan waktu acuan. Apabila selisih waktu kecil atau tidak berubah secara signifikan selama pengujian, maka RTC digunakan sebagai pewaktu pada sistem.

Selisih waktu RTC dihitung menggunakan **Persamaan (4.6)**.

$$\Delta t = t_{\text{RTC}} - t_{\text{acuan}} \quad (4.6)$$

Keterangan:

Δt = selisih waktu antara RTC dan waktu acuan

t_{RTC} = waktu yang terbaca pada RTC DS3231

t_{acuan} = waktu acuan dari jam pembanding

Berdasarkan data pertama pada Tabel 4.4, selisih waktu RTC dihitung sebagai berikut.

$$\Delta t = 02:57:26 - 02:57:00$$

$$\Delta t = +26$$

Pada sistem ini, RTC DS3231 digunakan untuk menghitung durasi proses ekstraksi. Durasi proses dihitung berdasarkan selisih antara waktu mulai dan waktu saat sistem berjalan. Perhitungan durasi ekstraksi ditunjukkan pada **Persamaan (4.7)**.

$$t_{\text{durasi}} = t_{\text{sekarang}} - t_{\text{mulai}} \quad (4.7)$$

Keterangan:

t_{durasi} = durasi proses ekstraksi

t_{sekarang} = waktu saat sistem sedang berjalan

t_{mulai} = waktu ketika proses ekstraksi dimulai

Berdasarkan hasil pengujian, RTC DS3231 dinyatakan digunakan apabila waktu yang terbaca sesuai dengan waktu acuan dan tetap berjalan ketika sistem dimatikan sementara. RTC digunakan sebagai pewaktu untuk menentukan durasi ekstraksi *cold brew coffee*. Pada sistem ini, RTC bekerja bersama ESP32 untuk mencatat waktu mulai, menghitung waktu berjalan, dan memberikan indikator ketika durasi ekstraksi telah selesai.

4.4 Hasil Pengujian Driver BTS7960 dan Peltier

Pengujian driver BTS7960 dan Peltier dilakukan untuk mengetahui respons aktuator pendingin terhadap perubahan nilai PWM dari ESP32. Pengujian dilakukan secara otomatis dengan menaikkan nilai PWM dari 0 sampai 255 dengan interval 25. Setiap nilai PWM diberikan selama 5 menit. Sebelum pengujian PWM bertahap dimulai, sistem diberi tahap stabilisasi awal selama 10 menit pada PWM 0.

Hasil pengujian driver BTS7960 dan Peltier ditunjukkan pada Tabel 4.9.

Tabel 4. 9 Hasil pengujian driver BTS7960 dan Peltier

No	PWM	Suhu Awal (°C)	Suhu Akhir (°C)
1	0	12,652	12,253
2	25	12,253	11,521
3	50	11,521	10,789
4	75	10,789	9,592
5	100	9,592	8,860
6	125	8,860	8,129
7	150	8,129	7,663
8	175	7,663	6,865
9	200	6,865	5,535
10	225	5,601	4,271
11	250	4,271	3,007
12	255	3,007	2,542

Berdasarkan Tabel 4.9, driver BTS7960 mampu merespons perubahan nilai PWM yang diberikan oleh ESP32. Pada PWM 0, suhu awal tahap pengujian adalah 12,652 °C dan suhu akhir menjadi 12,253 °C. Saat nilai PWM dinaikkan secara bertahap, suhu akhir pada setiap tahap cenderung menurun.

Pada PWM 25, suhu turun dari 12,253 °C menjadi 11,521 °C. Pada PWM 100, suhu turun dari 9,592 °C menjadi 8,860 °C. Pada PWM 200, suhu turun dari 6,865 °C menjadi 5,535 °C. Pada PWM 255, suhu akhir yang tercatat adalah 2,542 °C.

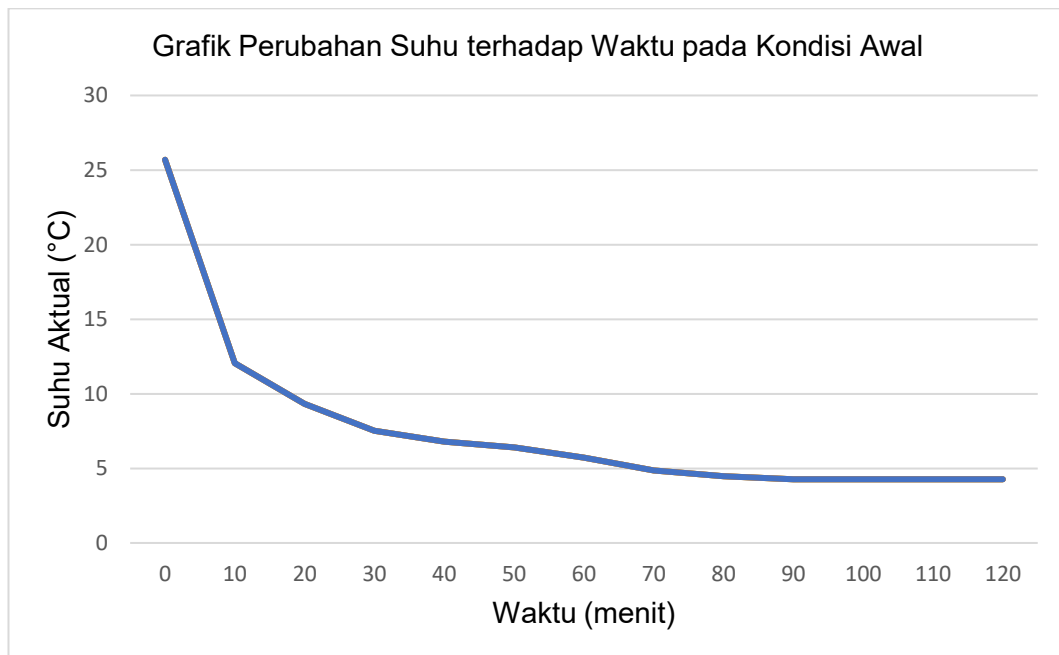
Hasil pengujian menunjukkan bahwa rangkaian ESP32, driver BTS7960, dan Peltier telah bekerja sesuai fungsi. ESP32 memberikan sinyal PWM, BTS7960 mengatur daya menuju Peltier, dan Peltier menghasilkan efek pendinginan. Pengujian ini digunakan sebagai pengujian fungsi aktuator, bukan sebagai penentuan karakteristik mutlak setiap nilai PWM, karena pengujian dilakukan secara bertahap tanpa mengembalikan suhu ke kondisi awal pada setiap tahap.

4.5 Hasil Pengujian *Open-Loop* Sistem Pendingin

Pengujian *open-loop* dilakukan untuk mengetahui respons suhu sistem pendingin sebelum kontroler PID diterapkan. Pada pengujian ini, PID belum diaktifkan sehingga ESP32 hanya memberikan nilai PWM secara langsung kepada driver BTS7960. Masukan sistem berupa perubahan nilai PWM, sedangkan keluaran sistem berupa suhu media ekstraksi yang dibaca oleh sensor DS18B20.

Pengujian diawali dengan kondisi PWM 0 untuk memperoleh suhu awal sistem. Setelah suhu berada pada kondisi relatif stabil, nilai PWM diubah menjadi 255 sehingga modul Peltier bekerja pada daya maksimum. Data yang dicatat meliputi waktu pengujian, nilai PWM, suhu mentah, dan suhu hasil kalibrasi. Data suhu hasil kalibrasi digunakan sebagai dasar analisis karena nilai tersebut telah dikoreksi terhadap alat ukur pembanding.

Pada pengujian ini, sistem menggunakan empat modul Peltier yang dikendalikan melalui driver BTS7960. Perubahan suhu yang terjadi menunjukkan respons langsung sistem pendingin terhadap pemberian PWM maksimum. Grafik hasil pengujian *open-loop* ditunjukkan pada Gambar 4.4.



Gambar 4. 3 Grafik respons suhu sistem pada pengujian *open-loop*

Berdasarkan Gambar 4.4, suhu sistem mengalami penurunan setelah PWM dinaikkan menjadi 255. Suhu awal rata-rata pada kondisi stabilisasi adalah 25,69

°C, sedangkan suhu akhir rata-rata pada bagian akhir respons adalah 4,35 °C. Penurunan suhu tersebut menunjukkan bahwa sistem pendingin mampu memberikan respons terhadap masukan PWM maksimum.

Ringkasan hasil pengujian *open-loop* ditunjukkan pada Tabel 4.10.

Tabel 4. 10 Ringkasan hasil pengujian *open-loop*

No	Parameter	Hasil
1	PWM awal	0
2	PWM akhir	255
3	Suhu awal rata-rata	25,69 °C
4	Suhu akhir rata-rata	4,35 °C
5	Perubahan suhu (ΔT)	21,34 °C
6	Perubahan PWM (ΔPWM)	255
7	Gain proses (K)	0,0837 °C/PWM

Gain proses dihitung berdasarkan perbandingan perubahan suhu terhadap perubahan nilai PWM. Karena sistem bekerja sebagai pendingin, kenaikan PWM menyebabkan suhu menurun. Perhitungan gain proses menggunakan nilai mutlak perubahan suhu agar parameter yang digunakan pada penalaan PID bernilai positif.

$$K = \left| \frac{\Delta T}{\Delta PWM} \right|$$

$$K = \left| \frac{25,69 - 4,35}{255 - 0} \right|$$

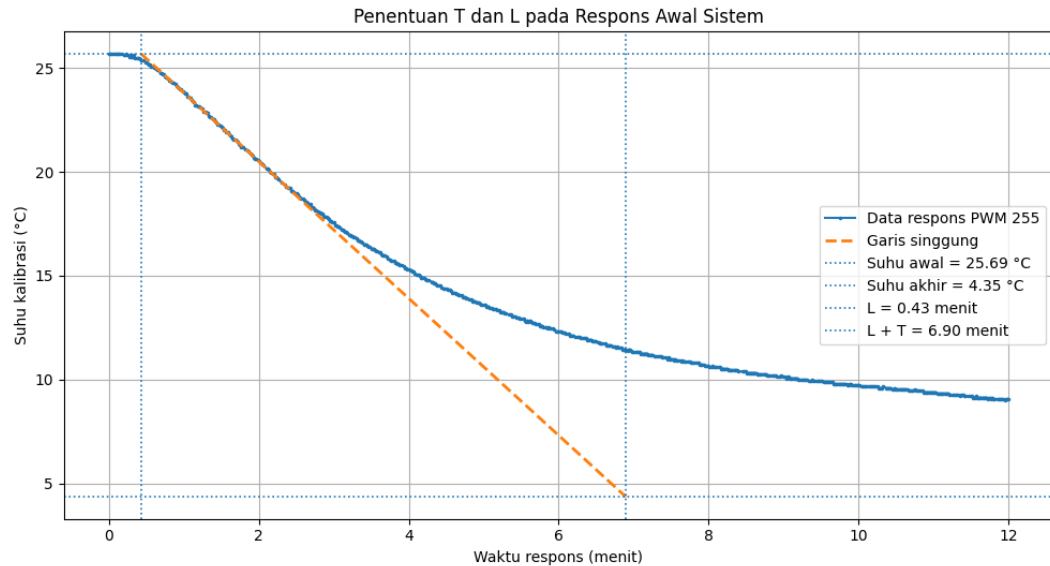
$$K = \frac{21,34}{255}$$

$$K = 0,0837 \text{ } ^\circ\text{C/PWM}$$

Nilai $K = 0,0837 \text{ } ^\circ\text{C/PWM}$ menunjukkan bahwa setiap perubahan satu satuan PWM menghasilkan perubahan suhu rata-rata sebesar 0,0837 °C berdasarkan rentang pengujian *open-loop*. Nilai ini digunakan sebagai gain proses dalam perhitungan parameter PID metode Ziegler–Nichols.

Penentuan waktu tunda (L) dan konstanta waktu (T) dilakukan menggunakan metode garis singgung pada kurva reaksi proses. Garis singgung dibuat pada bagian

respons awal yang memiliki kemiringan terbesar. Titik potong garis singgung terhadap garis suhu awal digunakan sebagai waktu tunda (L), sedangkan titik potong garis singgung terhadap garis suhu akhir digunakan sebagai nilai ($L + T$).



Gambar 4. 4 Penentuan nilai L dan T pada respons awal sistem

Berdasarkan hasil pengolahan grafik pada Gambar 4.x, diperoleh nilai waktu tunda (L) sebesar 0,43 menit. Titik potong garis singgung terhadap garis suhu akhir berada pada waktu 6,90 menit, sehingga nilai konstanta waktu (T) dapat dihitung sebagai berikut.

$$T = (L + T) - L$$

$$T = 6,90 - 0,43$$

$$T = 6,47 \text{ menit}$$

Ringkasan parameter hasil penentuan kurva reaksi proses ditunjukkan pada Tabel 4.11.

Tabel 4. 11 Parameter hasil kurva reaksi proses

No	Parameter	Nilai
1	Slope garis singgung	-3,300 °C/menit
2	Waktu tunda (L)	0,43 menit
3	Titik ($L + T$)	6,90 menit

No	Parameter	Nilai
4	Konstanta waktu (T)	6,47 menit
5	Gain proses (K)	0,0837 °C/PWM

Gain proses (K) digunakan untuk menyatakan besar perubahan keluaran sistem terhadap perubahan masukan. Pada pengujian *open-loop*, masukan sistem berupa nilai PWM yang diberikan kepada driver BTS7960, sedangkan keluaran sistem berupa perubahan suhu media ekstraksi yang terbaca oleh sensor DS18B20. Karena sistem bekerja sebagai pendingin, kenaikan nilai PWM menyebabkan suhu mengalami penurunan.

Secara umum, gain proses dinyatakan pada Persamaan (4.8).

$$K = \frac{\Delta y}{\Delta u}$$

Keterangan:

K = gain proses

Δy = perubahan keluaran sistem

Δu = perubahan masukan sistem

Pada sistem pendingin ini, keluaran sistem dinyatakan sebagai perubahan suhu, sedangkan masukan sistem dinyatakan sebagai perubahan nilai PWM. Karena arah respons suhu menurun saat PWM dinaikkan, perhitungan gain proses menggunakan nilai mutlak. Persamaan gain proses pada sistem pendingin ditunjukkan pada Persamaan (4.9).

$$K = \left| \frac{\Delta T}{\Delta PWM} \right|$$

Berdasarkan hasil pengujian *open-loop*, suhu awal rata-rata sistem adalah 25,69 °C, sedangkan suhu akhir rata-rata adalah 4,35 °C. Nilai PWM berubah dari 0 menjadi 255. Perubahan suhu dan perubahan PWM dihitung sebagai berikut.

$$\Delta T = T_{awal} - T_{akhir}$$

$$\Delta T = 25,69 - 4,35$$

$$\Delta T = 21,34^{\circ}C$$

$$\Delta PWM = PWM_{akhir} - PWM_{awal}$$

$$\Delta PWM = 255 - 0$$

$$\Delta PWM = 255$$

Nilai gain proses dihitung sebagai berikut.

$$K = \left| \frac{21,34}{255} \right|$$

$$K = 0,0837 \text{ } ^\circ\text{C}/PWM$$

Nilai $K = 0,0837 \text{ } ^\circ\text{C}/PWM$ menunjukkan bahwa pada rentang pengujian *open-loop*, setiap perubahan satu satuan PWM menghasilkan perubahan suhu rata-rata sebesar $0,0837 \text{ } ^\circ\text{C}$. Nilai ini digunakan sebagai gain proses dalam perhitungan parameter PID metode Ziegler–Nichols.

Selain gain proses, penentuan parameter Ziegler–Nichols juga membutuhkan nilai kemiringan kurva atau *slope*. Slope menunjukkan laju perubahan suhu terhadap waktu pada bagian kurva respons. Pada pengujian ini, *slope* digunakan untuk menentukan garis singgung pada bagian respons awal yang memiliki penurunan suhu paling tajam.

Secara matematis, *slope* dihitung menggunakan Persamaan (4.10).

$$m = \frac{\Delta T}{\Delta t}$$

Keterangan:

$m = \text{slope}$ atau kemiringan kurva ($^\circ\text{C}/\text{menit}$)

$\Delta T =$ perubahan suhu ($^\circ\text{C}$)

$\Delta t =$ perubahan waktu (menit)

Berdasarkan hasil pengolahan data *open-loop*, diperoleh nilai *slope* garis singgung sebesar:

$$m = -3,300 \text{ } ^\circ\text{C}/\text{menit}$$

Nilai negatif menunjukkan bahwa suhu mengalami penurunan terhadap waktu setelah PWM diberikan. Artinya, pada bagian respons paling curam, suhu turun sekitar $3,300 \text{ } ^\circ\text{C}$ setiap menit. Nilai *slope* ini tidak digunakan langsung untuk menghitung K_p , K_i , dan K_d , tetapi digunakan untuk membentuk garis singgung

pada kurva reaksi proses. Garis singgung tersebut menjadi dasar penentuan waktu tunda (L) dan konstanta waktu (T).

Gain proses (K) dan *slope* memiliki fungsi yang berbeda. Gain proses menunjukkan perbandingan perubahan suhu total terhadap perubahan PWM, sedangkan *slope* menunjukkan kecepatan perubahan suhu terhadap waktu pada bagian kurva yang paling curam. Nilai K digunakan dalam rumus K_p , sedangkan *slope* digunakan untuk membantu menentukan L dan T melalui metode garis singgung.

4.6 Perhitungan Parameter PID dengan Metode Ziegler–Nichols

Perhitungan parameter PID dilakukan berdasarkan hasil pengujian *open-loop* yang telah dijelaskan pada Subbab 4.5. Parameter yang digunakan dalam perhitungan adalah gain proses (K), waktu tunda (L), dan konstanta waktu (T). Berdasarkan hasil pengolahan kurva reaksi proses, diperoleh nilai:

$$K = 0,0837 \text{ } ^\circ\text{C}/\text{PWM}$$

$$L = 0,43 \text{ menit}$$

$$T = 6,47 \text{ menit}$$

Parameter PID dihitung menggunakan aturan Ziegler–Nichols metode kurva reaksi proses. Untuk kontroler PID, persamaan yang digunakan adalah sebagai berikut.

$$K_p = 1,2 \frac{T}{K \times L}$$

$$T_i = 2L$$

$$T_d = 0,5L$$

Perhitungan nilai K_p dilakukan sebagai berikut.

$$K_p = 1,2 \frac{6,47}{0,0837 \times 0,43}$$

$$K_p = \frac{7,764}{0,035991}$$

$$K_p = 215,72$$

Nilai waktu integral (T_i)dihitung sebagai berikut.

$$T_i = 2L$$

$$T_i = 2 \times 0,43$$

$$T_i = 0,86 \text{ menit}$$

Nilai waktu derivatif (T_d)dihitung sebagai berikut.

$$T_d = 0,5L$$

$$T_d = 0,5 \times 0,43$$

$$T_d = 0,215 \text{ menit}$$

Nilai T_i dan T_d kemudian dikonversi menjadi konstanta integral (K_i)dan konstanta derivatif (K_d).

$$K_i = \frac{K_p}{T_i}$$

$$K_i = \frac{215,72}{0,86}$$

$$K_i = 250,84$$

$$K_d = K_p \times T_d$$

$$K_d = 215,72 \times 0,215$$

$$K_d = 46,38$$

Hasil perhitungan awal parameter PID menggunakan metode Ziegler–Nichols ditunjukkan pada Tabel 4.12.

Tabel 4. 12 Hasil perhitungan awal parameter PID

No	Parameter	Nilai
1	Gain proses (K)	0,0837 °C/PWM
2	Waktu tunda (L)	0,43 menit
3	Konstanta waktu (T)	6,47 menit
4	Konstanta proporsional (K_p)	215,72

No	Parameter	Nilai
5	Waktu integral (T_i)	0,86 menit
6	Waktu derivatif (T_d)	0,215 menit
7	Konstanta integral (K_i)	250,84
8	Konstanta derivatif (K_d)	46,38

Nilai K_p , K_i , dan K_d tersebut merupakan parameter awal hasil perhitungan Ziegler–Nichols. Nilai ini digunakan sebagai dasar implementasi kontroler PID pada ESP32. Karena sistem pendingin Peltier memiliki respons termal lambat dan keluaran PWM dibatasi pada rentang 0 sampai 255, parameter hasil perhitungan perlu diuji kembali pada kondisi *closed-loop* untuk melihat respons aktual sistem.

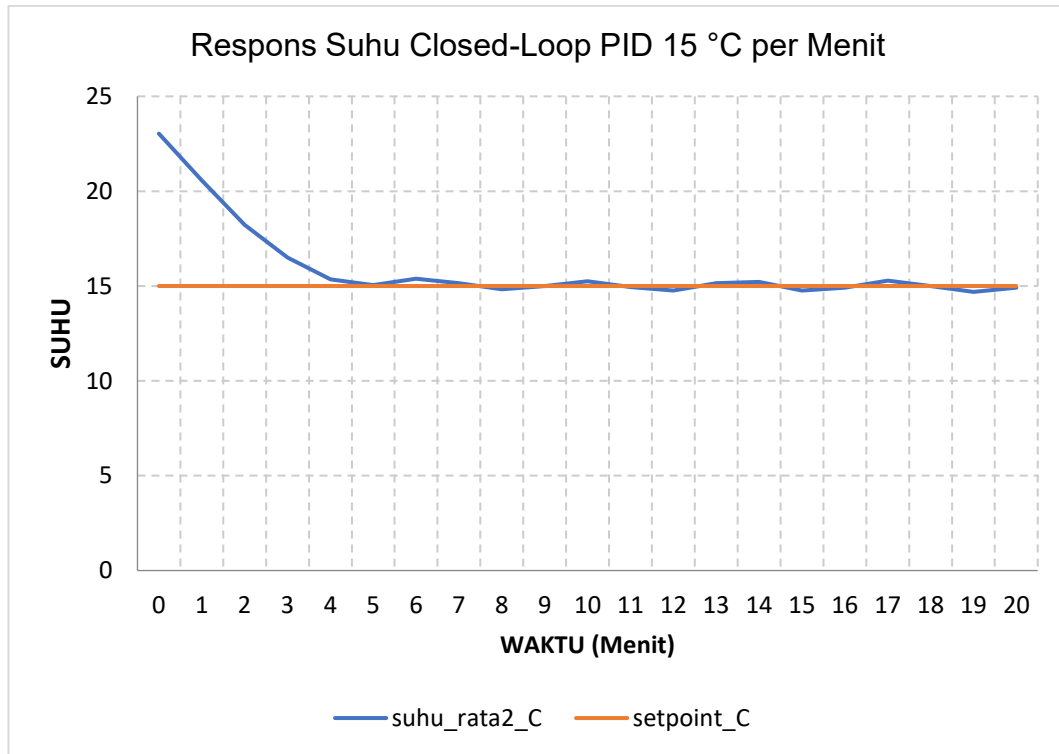
Dalam implementasi program ESP32, keluaran PID dibatasi agar tidak melebihi rentang PWM. Nilai minimum PWM adalah 0 dan nilai maksimum PWM adalah 255. Pembatasan ini dilakukan agar keluaran kontroler tetap sesuai dengan kemampuan driver BTS7960 dan aktuator Peltier.

4.7 Hasil Pengujian *Closed-Loop* PID

4.7.1 Pengujian *Closed-Loop* PID *Setpoint* 15 °C

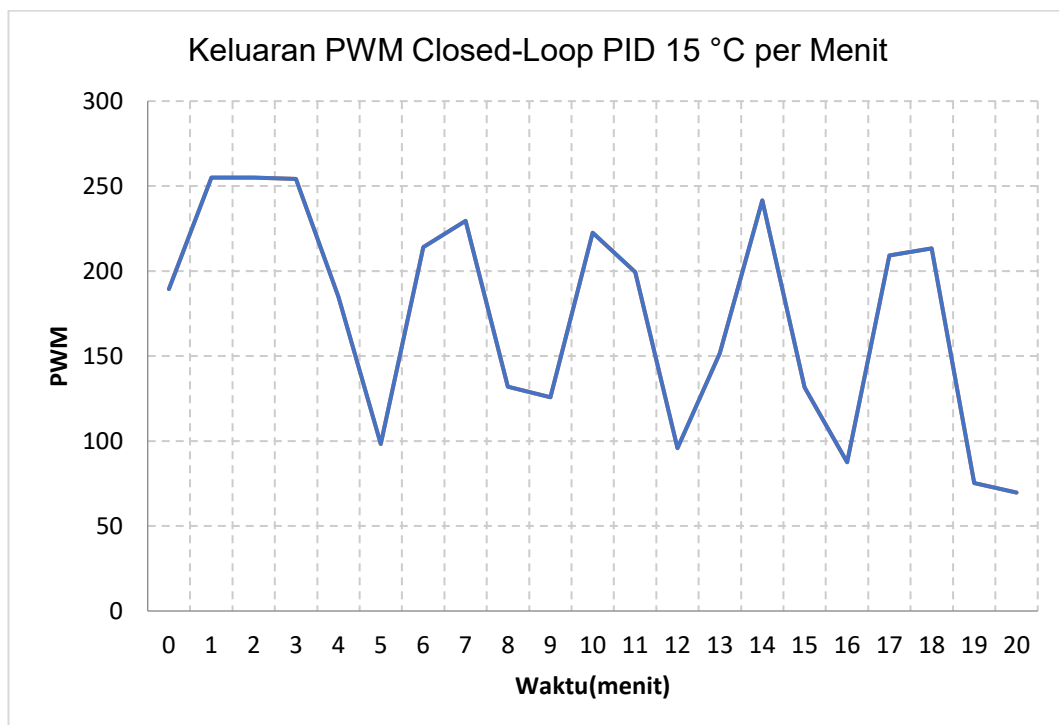
Pengujian pertama dilakukan pada *setpoint* 15 °C. Suhu awal saat PID aktif adalah 23,96 °C. Setelah sistem dijalankan, suhu menurun menuju nilai *setpoint*. Pada awal pengujian, nilai PWM meningkat hingga mencapai 255 karena selisih antara suhu aktual dan *setpoint* masih besar. Setelah suhu mendekati *setpoint*, nilai PWM mulai menurun sebagai respons terhadap berkurangnya *error*.

Respons suhu sistem pada pengujian *closed-loop* PID dengan *setpoint* 15 °C ditunjukkan pada Gambar 4.6.



Gambar 4. 5 Grafik respons suhu sistem pada pengujian *closed-loop* PID *setpoint* 15 °C

Perubahan nilai PWM selama pengujian *closed-loop* PID dengan *setpoint* 15 °C ditunjukkan pada Gambar 4.7.



Gambar 4. 6 Grafik keluaran PWM pada pengujian *closed-loop* PID *setpoint* 15 °C

Ringkasan hasil pengujian *closed-loop* PID pada *setpoint* 15 °C ditunjukkan pada Tabel 4.13.

Tabel 4. 13 Hasil pengujian *closed-loop* PID *setpoint* 15 °C

No	Parameter	Nilai
1	<i>Setpoint</i>	15,00 °C
2	Suhu awal saat PID aktif	23,96 °C
3	Waktu mencapai daerah <i>setpoint</i>	4,62 menit
4	Suhu minimum	14,65 °C
5	<i>Overshoot</i> pendinginan	0,35 °C
6	Suhu rata-rata kondisi stabil	15,03 °C
7	<i>Steady-state error</i>	0,026 °C
8	Fluktuasi suhu stabil	0,865 °C
9	PWM maksimum	255

Waktu mencapai daerah *setpoint* dihitung dari selisih waktu saat suhu masuk ke daerah *setpoint* terhadap waktu mulai PID aktif. Pada pengujian 15 °C, PID mulai aktif pada waktu sekitar 45 s dan daerah *setpoint* tercapai pada waktu sekitar 322 s. Perhitungan waktu pencapaian *setpoint* adalah sebagai berikut.

$$t = \frac{322 - 45}{60}$$

$$t = 4,62 \text{ menit}$$

Suhu minimum yang tercatat selama pengujian adalah 14,65 °C. Karena sistem bekerja sebagai pendingin, *overshoot* terjadi ketika suhu turun melewati nilai *setpoint*. Besar *overshoot* dihitung sebagai berikut.

$$Overshoot = T_{setpoint} - T_{minimum}$$

$$Overshoot = 15,00 - 14,65$$

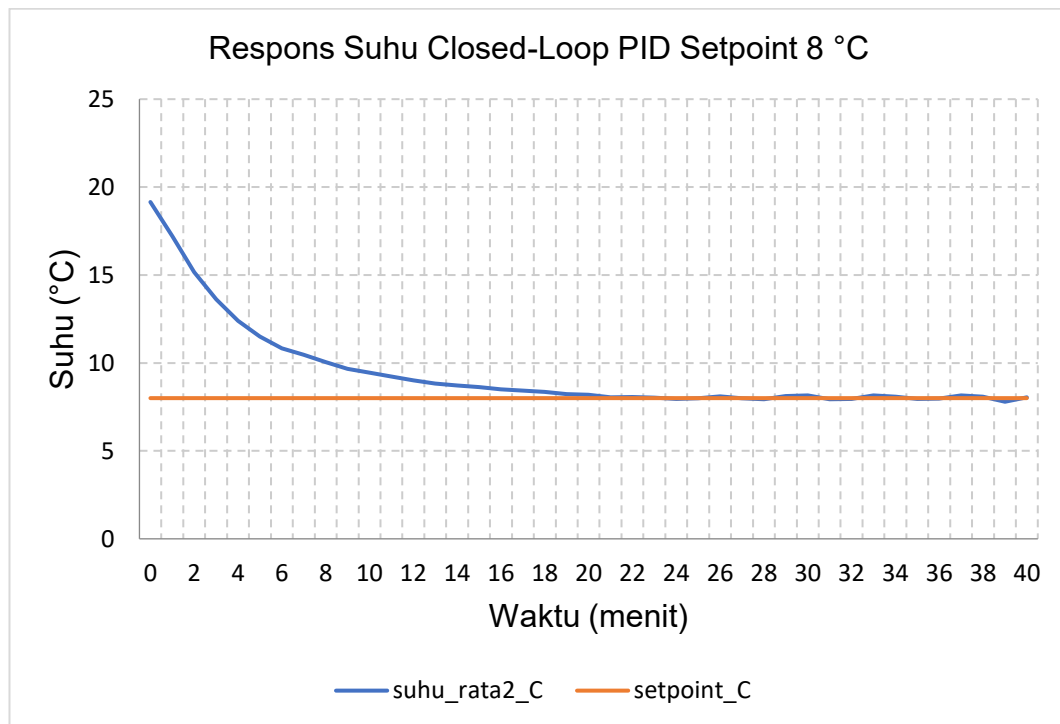
$$Overshoot = 0,35^{\circ}C$$

Hasil pengujian menunjukkan bahwa sistem mampu mencapai daerah *setpoint* 15 °C dalam waktu 4,62 menit. Nilai *steady-state error* sebesar 0,026 °C menunjukkan bahwa suhu rata-rata kondisi stabil berada sangat dekat dengan *setpoint*. Fluktuasi suhu sebesar 0,865 °C menunjukkan bahwa suhu masih berubah di sekitar *setpoint*, tetapi tetap berada dalam rentang kendali.

4.7.2 Pengujian *Closed-Loop* PID *Setpoint* 8 °C

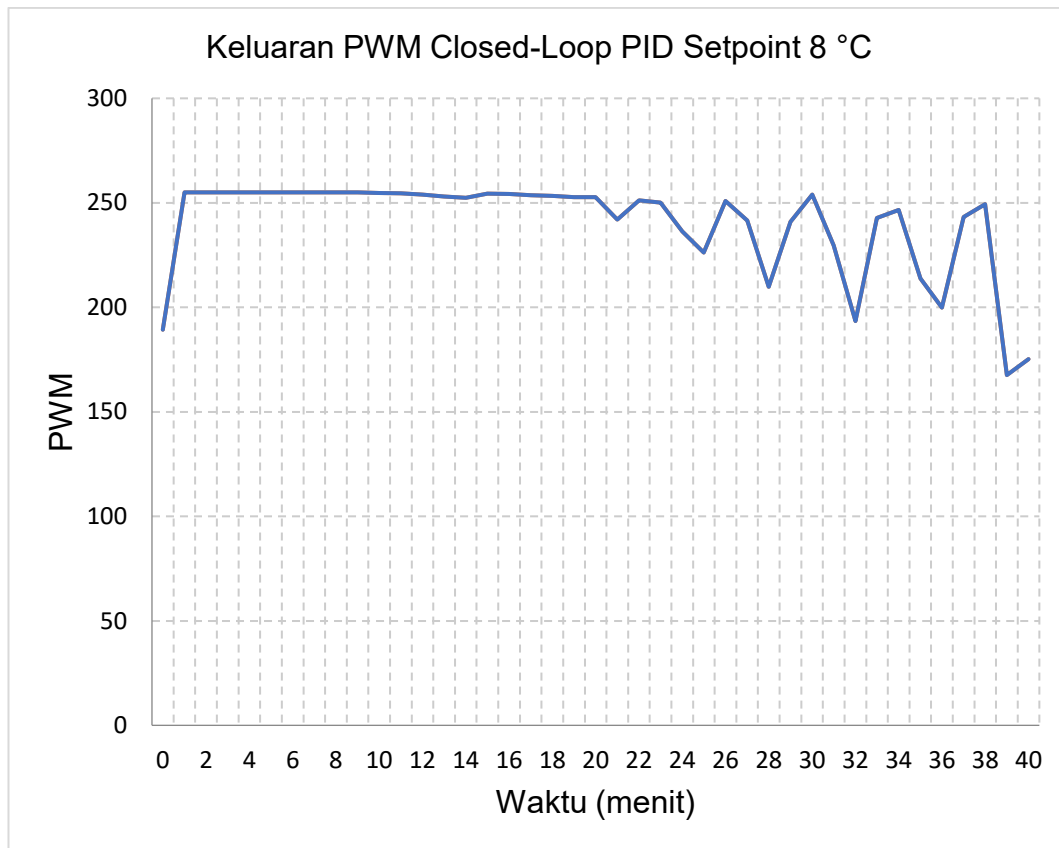
Pengujian kedua dilakukan pada *setpoint* 8 °C. *Setpoint* ini digunakan untuk melihat kemampuan sistem pada kondisi pendinginan yang lebih berat. Pada pengujian ini, suhu awal saat PID aktif adalah 19,90 °C. Nilai PWM meningkat hingga mencapai 255 dan bertahan cukup lama karena sistem membutuhkan daya pendinginan yang besar untuk menurunkan suhu menuju 8 °C.

Respons suhu sistem pada pengujian *closed-loop* PID dengan *setpoint* 8 °C ditunjukkan pada Gambar 4.8.



Gambar 4. 7 Grafik respons suhu sistem pada pengujian *closed-loop* PID *setpoint* 8 °C

Perubahan nilai PWM selama pengujian *closed-loop* PID dengan *setpoint* 8 °C ditunjukkan pada Gambar 4.9.



Gambar 4. 8 Grafik keluaran PWM pada pengujian *closed-loop* PID *setpoint* 8 °C

Ringkasan hasil pengujian *closed-loop* PID pada *setpoint* 8 °C ditunjukkan pada Tabel 4.14.

Tabel 4. 14 Hasil pengujian *closed-loop* PID *setpoint* 8 °C

No	Parameter	Nilai
1	<i>Setpoint</i>	8,00 °C
2	Suhu awal saat PID aktif	19,90 °C
3	Waktu mencapai daerah <i>setpoint</i>	19,33 menit
4	Suhu minimum	7,73 °C
5	<i>Overshoot</i> pendinginan	0,27 °C
6	Suhu rata-rata kondisi stabil	8,04 °C
7	<i>Steady-state error</i>	0,037 °C
8	Fluktuasi suhu stabil	0,532 °C
9	PWM maksimum	255
10	PWM rata-rata kondisi stabil	230,59

Pada pengujian 8 °C, sistem membutuhkan waktu 19,33 menit untuk mencapai daerah *setpoint*. Waktu tersebut lebih lama dibandingkan pengujian 15 °C karena selisih antara suhu awal dan suhu target lebih besar. Nilai PWM juga lebih lama berada pada nilai maksimum 255, yang menunjukkan bahwa sistem membutuhkan daya pendinginan lebih besar.

Suhu minimum selama pengujian adalah 7,73 °C. Besar *overshoot* dihitung sebagai berikut.

$$Overshoot = T_{setpoint} - T_{minimum}$$

$$Overshoot = 8,00 - 7,73$$

$$Overshoot = 0,27^{\circ}C$$

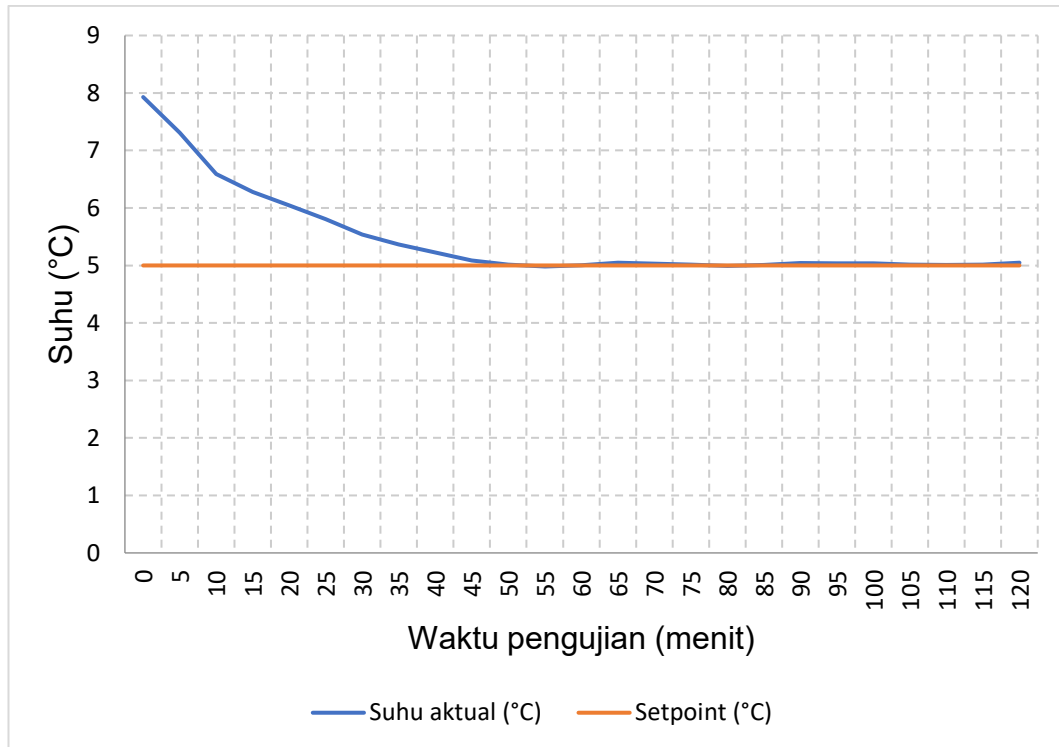
Nilai *overshoot* sebesar 0,27 °C menunjukkan bahwa suhu hanya turun sedikit melewati *setpoint*. Setelah mencapai daerah *setpoint*, suhu rata-rata berada pada 8,04 °C dengan *steady-state error* sebesar 0,037 °C. Fluktuasi suhu stabil sebesar 0,532 °C menunjukkan bahwa sistem mampu mempertahankan suhu di sekitar *setpoint* 8 °C.

4.7.3 Pengujian Closed-Loop PID Setpoint 5 °C

Pengujian tambahan dilakukan pada *setpoint* 5 °C untuk mengetahui kemampuan sistem pada kondisi pendinginan yang lebih rendah. Data yang dianalisis dibatasi selama 2 jam sejak perintah START diberikan. Pembatasan waktu ini dilakukan karena pengujian ditujukan untuk melihat respons awal dan kemampuan sistem mempertahankan suhu pada daerah *setpoint*, bukan untuk menganalisis keseluruhan durasi ekstraksi.

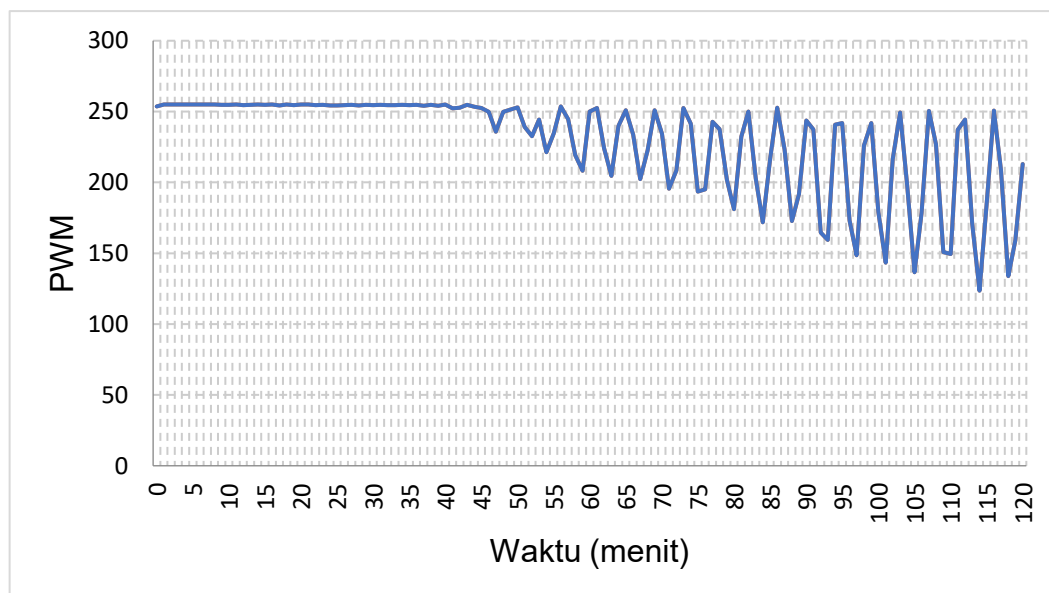
Pada pengujian ini, suhu awal saat START adalah 7,93 °C. Setelah sistem dijalankan, nilai PWM meningkat hingga mencapai 255 karena suhu aktual masih berada di atas *setpoint*. Sistem mencapai daerah *setpoint* pada waktu 38,63 menit dengan suhu 5,14 °C.

Respons suhu sistem pada pengujian *closed-loop* PID dengan *setpoint* 5 °C ditunjukkan pada Gambar 4.10.



Gambar 4. 9 Grafik respons suhu sistem pada pengujian *closed-loop* PID *setpoint* 5 °C

Perubahan nilai PWM selama pengujian *closed-loop* PID dengan *setpoint* 5 °C ditunjukkan pada Gambar 4.11.



Gambar 4. 10 Grafik keluaran PWM pada pengujian *closed-loop* PID *setpoint* 5 °C

Ringkasan hasil pengujian *closed-loop* PID pada *setpoint* 5 °C ditunjukkan pada Tabel 4.15.

Tabel 4. 15 Hasil pengujian *closed-loop* PID *setpoint* 5 °C

No	Parameter	Nilai
1	<i>Setpoint</i>	5,00 °C
2	Lama data dianalisis	2 jam
3	Suhu awal saat START	7,93 °C
4	Waktu mencapai daerah <i>setpoint</i>	38,63 menit
5	Suhu saat daerah <i>setpoint</i> tercapai	5,14 °C
6	Suhu minimum selama pengujian	4,74 °C
7	<i>Overshoot</i> pendinginan	0,26 °C
8	Suhu rata-rata kondisi stabil	5,03 °C
9	<i>Steady-state error</i>	0,026 °C
10	Fluktuasi suhu stabil	0,598 °C
11	PWM maksimum	255
12	PWM rata-rata kondisi stabil	217,59
13	<i>Scale</i> PID	1,00

Besar *overshoot* pendinginan dihitung dari selisih antara *setpoint* dan suhu minimum.

$$Overshoot = T_{setpoint} - T_{minimum}$$

$$Overshoot = 5,00 - 4,74$$

$$Overshoot = 0,26^{\circ}C$$

Kesalahan keadaan tunak dihitung dari selisih antara suhu rata-rata kondisi stabil dan nilai *setpoint*.

$$e_{ss} = | T_{rata-rata} - T_{setpoint} |$$

$$e_{ss} = | 5,03 - 5,00 |$$

$$e_{ss} = 0,026^{\circ}C$$

Hasil pengujian menunjukkan bahwa sistem mampu mencapai daerah *setpoint* 5 °C dalam waktu 38,63 menit. Waktu ini lebih lama dibandingkan pengujian 15 °C dan 8 °C karena target suhu lebih rendah. Setelah mencapai daerah

setpoint, suhu rata-rata berada pada 5,03 °C dengan *steady-state error* sebesar 0,026 °C. Fluktuasi suhu stabil sebesar 0,598 °C menunjukkan bahwa sistem masih mengalami perubahan suhu di sekitar *setpoint*, tetapi suhu tetap berada dekat dengan nilai target.

PWM maksimum pada pengujian ini mencapai 255, sedangkan PWM rata-rata pada kondisi stabil sebesar 217,59. Nilai tersebut menunjukkan bahwa sistem membutuhkan daya pendinginan besar untuk mempertahankan suhu pada 5 °C. Hasil ini memperkuat bahwa beban pendinginan semakin besar ketika nilai *setpoint* semakin rendah.

4.7.4 Perbandingan Hasil Pengujian *Closed-Loop* PID

Perbandingan hasil pengujian *closed-loop* PID dilakukan pada tiga nilai *setpoint*, yaitu 15 °C, 8 °C, dan 5 °C. *Setpoint* 15 °C digunakan untuk mewakili pendinginan sedang, *setpoint* 8 °C digunakan untuk mewakili pendinginan rendah, sedangkan *setpoint* 5 °C digunakan sebagai pengujian tambahan untuk melihat kemampuan sistem pada kondisi pendinginan yang lebih berat. Pada pengujian 5 °C, data yang dianalisis dibatasi selama 2 jam sejak perintah START diberikan. Log pengujian menunjukkan bahwa *setpoint* diubah menjadi 5 °C, kemudian sistem dijalankan dengan parameter PID yang sama, yaitu $K_p = 215,72$, $K_i = 4,181$, $K_d = 2782,79$.

Perbandingan hasil pengujian *closed-loop* ditunjukkan pada Tabel 4.16.

Tabel 4. 16 Perbandingan hasil pengujian *closed-loop* PID

No	Parameter	<i>Setpoint</i> 15 °C	<i>Setpoint</i> 8 °C	<i>Setpoint</i> 5 °C
1	Suhu awal saat PID aktif	23,96 °C	19,90 °C	7,93 °C
2	Lama data dianalisis	Sampai stabil	Sampai stabil	2 jam
3	Waktu mencapai daerah <i>setpoint</i>	4,62 menit	19,33 menit	38,63 menit
4	Suhu saat daerah <i>setpoint</i> tercapai	15,00 °C	8,00 °C	5,14 °C

No	Parameter	<i>Setpoint</i> 15 °C	<i>Setpoint</i> 8 °C	<i>Setpoint</i> 5 °C
5	Suhu minimum	14,65 °C	7,73 °C	4,74 °C
6	<i>Overshoot</i> pendinginan	0,35 °C	0,27 °C	0,26 °C
7	Suhu rata-rata kondisi stabil	15,03 °C	8,04 °C	5,03 °C
8	<i>Steady-state error</i>	0,026 °C	0,037 °C	0,026 °C
9	Fluktuasi suhu stabil	0,865 °C	0,532 °C	0,598 °C
10	PWM maksimum	255	255	255
11	PWM rata-rata kondisi stabil	159,47	230,59	217,59
12	<i>Scale</i> PID	1,00	1,00	1,00

Berdasarkan Tabel 4.16, semakin rendah nilai *setpoint*, waktu yang dibutuhkan sistem untuk mencapai daerah *setpoint* semakin lama. Pada *setpoint* 15 °C, sistem mencapai daerah *setpoint* dalam waktu 4,62 menit. Pada *setpoint* 8 °C, waktu yang dibutuhkan meningkat menjadi 19,33 menit. Pada *setpoint* 5 °C, sistem membutuhkan waktu 38,63 menit untuk mencapai daerah *setpoint*. Hal ini menunjukkan bahwa beban pendinginan meningkat ketika suhu target semakin rendah.

Pada pengujian 5 °C, suhu awal saat START adalah 7,93 °C. Artinya, pengujian 5 °C tidak dimulai dari suhu ruang, melainkan dari kondisi sistem yang sudah dingin setelah pengujian sebelumnya. Karena itu, hasil waktu pencapaian *setpoint* 5 °C digunakan sebagai data tambahan untuk melihat kemampuan sistem menuju suhu rendah, bukan sebagai perbandingan langsung terhadap pendinginan dari suhu ruang.

Nilai *overshoot* pada ketiga pengujian masih berada pada rentang kecil. Pada *setpoint* 15 °C, *overshoot* sebesar 0,35 °C. Pada *setpoint* 8 °C, *overshoot* sebesar 0,27 °C. Pada *setpoint* 5 °C, *overshoot* sebesar 0,26 °C. Nilai tersebut menunjukkan bahwa kontroler PID mampu mengurangi daya pendinginan ketika suhu mendekati *setpoint* sehingga suhu tidak turun terlalu jauh melewati target.

Nilai *steady-state error* juga relatif kecil pada ketiga *setpoint*. Pada *setpoint* 15 °C diperoleh *steady-state error* sebesar 0,026 °C, pada *setpoint* 8 °C sebesar 0,037 °C, dan pada *setpoint* 5 °C sebesar 0,026 °C. Hasil ini menunjukkan bahwa suhu rata-rata setelah kondisi stabil berada dekat dengan nilai *setpoint*.

Perbandingan PWM menunjukkan bahwa *setpoint* yang lebih rendah membutuhkan aksi kendali lebih besar. PWM maksimum pada ketiga pengujian mencapai 255. Pada kondisi stabil, PWM rata-rata pada *setpoint* 15 °C adalah 159,47, sedangkan pada *setpoint* 8 °C sebesar 230,59 dan pada *setpoint* 5 °C sebesar 217,59. Nilai PWM yang tinggi pada *setpoint* 8 °C dan 5 °C menunjukkan bahwa sistem membutuhkan daya pendinginan besar untuk mempertahankan suhu rendah.

Secara keseluruhan, pengujian *closed-loop* menunjukkan bahwa sistem pendingin otomatis mampu mencapai dan mempertahankan suhu pada *setpoint* 15 °C, 8 °C, dan 5 °C. Parameter PID hasil metode Ziegler–Nichols mampu digunakan sebagai parameter awal pengendalian suhu, dengan nilai *overshoot* dan *steady-state error* yang kecil pada ketiga *setpoint*.

4.8 Perbandingan Hasil Ekstraksi Menggunakan Sistem Pendingin Otomatis dan Kulkas Biasa

Pengujian ini dilakukan untuk membandingkan hasil ekstraksi *cold brew coffee* menggunakan sistem pendingin otomatis berbasis PID dengan hasil ekstraksi menggunakan kulkas biasa. Parameter yang dibandingkan adalah pH dan *Total Dissolved Solids* (TDS). Nilai pH digunakan untuk melihat tingkat keasaman hasil ekstraksi, sedangkan TDS digunakan untuk melihat jumlah padatan terlarut pada hasil ekstraksi.

Pengujian dilakukan pada kondisi ekstraksi yang sama, yaitu suhu 5 °C selama 20 jam. Sampel pertama diekstraksi menggunakan sistem pendingin otomatis berbasis PID, sedangkan sampel kedua diekstraksi menggunakan kulkas biasa. Setiap metode pendinginan diuji sebanyak tiga kali untuk memperoleh data pembandingan yang lebih baik.

Tabel 4. 17 Perbandingan pH dan TDS hasil ekstraksi menggunakan sistem pendingin otomatis dan kulkas biasa

No	Metode Pendinginan	Suhu Ekstraksi	Durasi	Ulangan	pH	TDS (ppm)
1	Sistem pendingin otomatis PID	5 °C	20 jam	1	5,4	1060
2	Sistem pendingin otomatis PID	5 °C	20 jam	2	5,3	1060
3	Sistem pendingin otomatis PID	5 °C	20 jam	3	5,3	1010
4	Kulkas rumah	5 °C	20 jam	1	5,3	983
5	Kulkas rumah	5 °C	20 jam	2	5,4	1010
6	Kulkas rumah	5 °C	20 jam	3	5,4	1010

Rata-rata hasil pengujian pH dan TDS ditunjukkan pada Tabel 4.18.

Tabel 4. 18 Rata-rata pH dan TDS hasil ekstraksi

No	Metode Pendinginan	Rata-rata pH	Rata-rata TDS (ppm)
1	Sistem pendingin otomatis PID	5,33	1043,33
2	Kulkas biasa	5,37	1001,00

Berdasarkan Tabel 4.18, hasil ekstraksi menggunakan sistem pendingin otomatis PID menghasilkan rata-rata pH sebesar 5,33, sedangkan hasil ekstraksi menggunakan kulkas biasa menghasilkan rata-rata pH sebesar 5,37. Selisih rata-rata pH antara kedua metode adalah 0,04. Nilai ini menunjukkan bahwa tingkat keasaman hasil ekstraksi pada kedua metode relatif berdekatan.

Pada parameter TDS, hasil ekstraksi menggunakan sistem pendingin otomatis PID menghasilkan rata-rata TDS sebesar 1043,33 ppm. Hasil ekstraksi menggunakan kulkas biasa menghasilkan rata-rata TDS sebesar 1001,00 ppm. Selisih rata-rata TDS antara kedua metode adalah 42,33 ppm. Nilai TDS pada sistem pendingin otomatis PID lebih tinggi dibandingkan kulkas biasa.

Perbedaan nilai TDS menunjukkan bahwa jumlah padatan terlarut pada hasil ekstraksi menggunakan sistem pendingin otomatis PID lebih besar dibandingkan hasil ekstraksi menggunakan kulkas biasa. Perbedaan ini dapat terjadi karena sistem pendingin otomatis PID mengendalikan suhu berdasarkan *setpoint*, sedangkan kulkas biasa bekerja menggunakan sistem pendinginan internal yang tidak secara langsung mengatur suhu *box* ekstraksi.

Hasil pengujian pH menunjukkan perbedaan yang kecil antara kedua metode pendinginan. Rentang pH pada sistem pendingin otomatis PID berada pada 5,3 sampai 5,4, sedangkan rentang pH pada kulkas biasa juga berada pada 5,3 sampai 5,4. Hal ini menunjukkan bahwa kedua metode menghasilkan tingkat keasaman yang relatif sama pada kondisi ekstraksi 5 °C selama 20 jam.

Data pH dan TDS pada pengujian ini digunakan sebagai data pembandingan hasil ekstraksi, bukan sebagai dasar penilaian mutu kopi secara menyeluruh. Penilaian utama tugas akhir ini tetap berada pada kinerja sistem pendingin otomatis, yaitu kemampuan sistem mencapai dan mempertahankan suhu *setpoint* menggunakan kontroler PID.

BAB V

PENUTUP

5.1 Kesimpulan

1. Sistem pendingin otomatis berhasil dirancang dan dibangun menggunakan ESP32 sebagai pengendali utama, sensor DS18B20 sebagai pembaca suhu, RTC DS3231 sebagai acuan waktu, driver BTS7960 sebagai pengatur daya, dan Peltier TEC1-12706 sebagai aktuator pendingin. Sistem mampu membaca suhu aktual, mengatur durasi ekstraksi, menampilkan informasi pada LCD, serta menghasilkan sinyal PWM untuk mengatur kerja Peltier.
2. Karakteristik sistem pendingin diperoleh melalui pengujian *open-loop* dengan perubahan masukan PWM dari 0 menjadi 255. Hasil pengujian menghasilkan gain proses $K = 0,0837 \text{ }^{\circ}\text{C}/\text{PWM}$, waktu tunda $L = 0,43$ menit, dan konstanta waktu $T = 6,47$ menit. Nilai tersebut digunakan sebagai dasar penentuan parameter kontroler PID menggunakan metode Ziegler–Nichols.
3. Parameter kontroler PID yang diperoleh dari metode Ziegler–Nichols adalah $K_p = 215,72$, $K_i = 4,181$, dan $K_d = 2782,79$. Parameter tersebut diterapkan pada program ESP32 untuk mengatur keluaran PWM berdasarkan selisih antara suhu aktual dan nilai *setpoint*.
4. Pengujian *closed-loop* menunjukkan bahwa sistem mampu mencapai dan mempertahankan suhu di sekitar *setpoint*. Pada *setpoint* $15 \text{ }^{\circ}\text{C}$, sistem mencapai daerah *setpoint* dalam 4,62 menit dengan *overshoot* $0,35 \text{ }^{\circ}\text{C}$ dan *steady-state error* $0,026 \text{ }^{\circ}\text{C}$. Pada *setpoint* $8 \text{ }^{\circ}\text{C}$, sistem mencapai daerah *setpoint* dalam 19,33 menit dengan *overshoot* $0,27 \text{ }^{\circ}\text{C}$ dan *steady-state error* $0,037 \text{ }^{\circ}\text{C}$. Pada *setpoint* $5 \text{ }^{\circ}\text{C}$, sistem mencapai daerah *setpoint* dalam 38,63 menit dengan *overshoot* $0,26 \text{ }^{\circ}\text{C}$ dan *steady-state error* $0,026 \text{ }^{\circ}\text{C}$. Hasil tersebut menunjukkan bahwa kontroler PID mampu mengatur suhu sistem pendingin otomatis dengan nilai *overshoot* dan *steady-state error* yang kecil.
5. Pengujian pH dan TDS digunakan sebagai data pendukung penerapan sistem pada proses ekstraksi *cold brew coffee*. Pada pengujian $5 \text{ }^{\circ}\text{C}$ selama

20 jam, sistem pendingin otomatis PID menghasilkan rata-rata pH 5,33 dan TDS 1043,33 ppm, sedangkan kulkas biasa menghasilkan rata-rata pH 5,37 dan TDS 1001,00 ppm. Data tersebut menunjukkan hasil ekstraksi dari kedua metode berada pada nilai yang berdekatan, tetapi tidak digunakan sebagai dasar penilaian mutu kopi secara menyeluruh.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran untuk pengembangan sistem selanjutnya.

1. Sistem pendingin dapat dikembangkan dengan peningkatan pada sisi pelepasan panas Peltier, seperti penggunaan *heatsink* yang lebih besar, kipas dengan aliran udara lebih tinggi, atau sistem pendingin air. Perbaikan pelepasan panas diperlukan untuk mempercepat penurunan suhu dan meningkatkan kestabilan sistem.
2. Isolasi termal ruang pendingin perlu ditingkatkan agar panas dari lingkungan tidak mudah masuk ke ruang ekstraksi. Penggunaan bahan isolasi yang lebih baik dapat mengurangi beban kerja Peltier dan menurunkan kebutuhan PWM saat sistem berada pada kondisi stabil.
3. Pengujian *closed-loop* sebaiknya dilakukan dengan jumlah pengulangan lebih banyak pada setiap *setpoint* agar kestabilan sistem dapat dianalisis dengan data yang lebih kuat.
4. Sistem dapat dikembangkan dengan fitur pencatatan data otomatis, seperti penyimpanan data suhu, waktu, *setpoint*, dan PWM ke kartu memori atau komputer. Fitur ini dapat mempermudah analisis respons sistem tanpa pencatatan manual.
5. Pengujian pH dan TDS pada penelitian selanjutnya dapat diperluas dengan parameter tambahan seperti kadar kafein, senyawa fenolik, atau uji sensoris. Pengujian tersebut perlu dilakukan apabila fokus penelitian diarahkan pada kualitas hasil ekstraksi kopi, bukan hanya pada kinerja sistem pendingin.

DAFTAR PUSTAKA

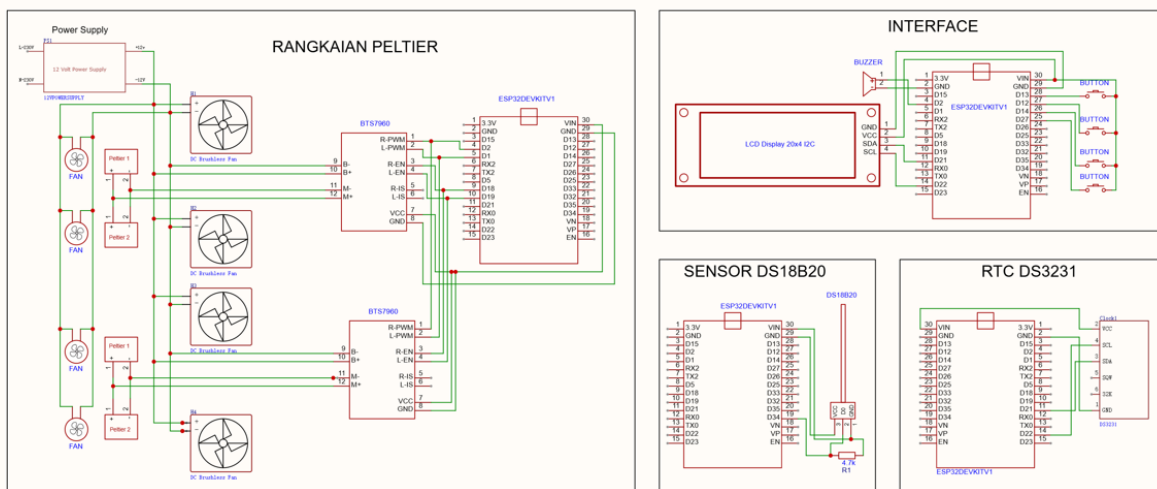
- [1] K. H. Ang, G. Chong, and Y. Li, “PID control system analysis, design, and technology,” *IEEE Transactions on Control Systems Technology*, vol. 13, no. 4, pp. 559–576, 2005, doi: 10.1109/TCST.2005.847331.
- [2] A. Kherkhar, Y. Chiba, A. Tlemçani, and H. Mamur, “Thermal investigation of a *thermoelectric cooler* based on Arduino and PID control approach,” *Case Studies in Thermal Engineering*, vol. 36, art. no. 102249, 2022, doi: 10.1016/j.csite.2022.102249.
- [3] J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Transactions of the ASME*, vol. 64, no. 8, pp. 759–768, 1942.
- [4] N. Córdoba, L. Pataquiva, C. Osorio, F. L. M. Moreno, and R. Y. Ruiz, “Effect of grinding, extraction time and type of coffee on the physicochemical and flavour characteristics of *cold brew coffee*,” *Scientific Reports*, vol. 9, no. 1, art. no. 8440, 2019, doi: 10.1038/s41598-019-44886-w.
- [5] M. K. Anam, I. Ilmirrizki, and Tijaniyah, “Perancangan control temperature *thermoelectric cooler* menggunakan kontrol PID,” *Journal of Renewable Energy, Electronics and Control*, vol. 2, no. 2, pp. 7–18, 2022, doi: 10.31284/j.jreec.2022.v2i1.2105.
- [6] X. Wang and L. T. Lim, “Modeling study of coffee extraction at different temperature and grind size conditions to better understand the cold and hot brewing process,” *Journal of Food Process Engineering*, vol. 44, no. 8, art. no. e13748, 2021, doi: 10.1111/jfpe.13748.
- [7] R. Tanglao, “Iced *cold brew coffee*,” *Wikimedia Commons*, 2007.

- [8] N. H. Zakaria, K. Whanmek, S. Thangsiri, W. Chathiran, W. Srichamnong, U. Suttisansanee, and C. Santivarangkna, “Optimization of *cold brew coffee* using central composite design and its properties compared with hot brew coffee,” *Foods*, vol. 12, no. 12, art. no. 2412, 2023, doi: 10.3390/foods12122412.
- [9] N. Z. Rao, M. Fuller, and M. D. Grim, “Physicochemical characteristics of hot and *cold brew coffee* chemistry: The effects of roast level and brewing temperature on compound extraction,” *Foods*, vol. 9, no. 7, art. no. 902, 2020, doi: 10.3390/foods9070902.
- [10] M. S. Pua, A. H. J. Ontowirjo, and P. D. K. Manembu, “Studi perbandingan kontrol PID dan metode ON-OFF pada sistem kotak pendingin menggunakan thermoelectric,” *Jurnal Teknik Elektro dan Komputer*, pp. 1–13, 2022.
- [11] Espressif Systems, “LED Control (LEDC),” *ESP-IDF Programming Guide*, 2024.
- [12] Espressif Systems, “ESP32 series datasheet,” *Technical Documentation*, 2021.
- [13] Maxim Integrated, “DS18B20 programmable resolution 1-Wire digital thermometer,” *Data Sheet*, 2019.
- [14] Maxim Integrated, “DS3231 extremely accurate I²C-integrated RTC/TCXO/crystal,” *Data Sheet*, 2015.
- [15] Infineon Technologies AG, “BTS7960 high current PN half bridge,” *Data Sheet*, 2004.
- [16] Winstar Display Co., Ltd., “WH2004A LCD module specification,” *Data*

Sheet.

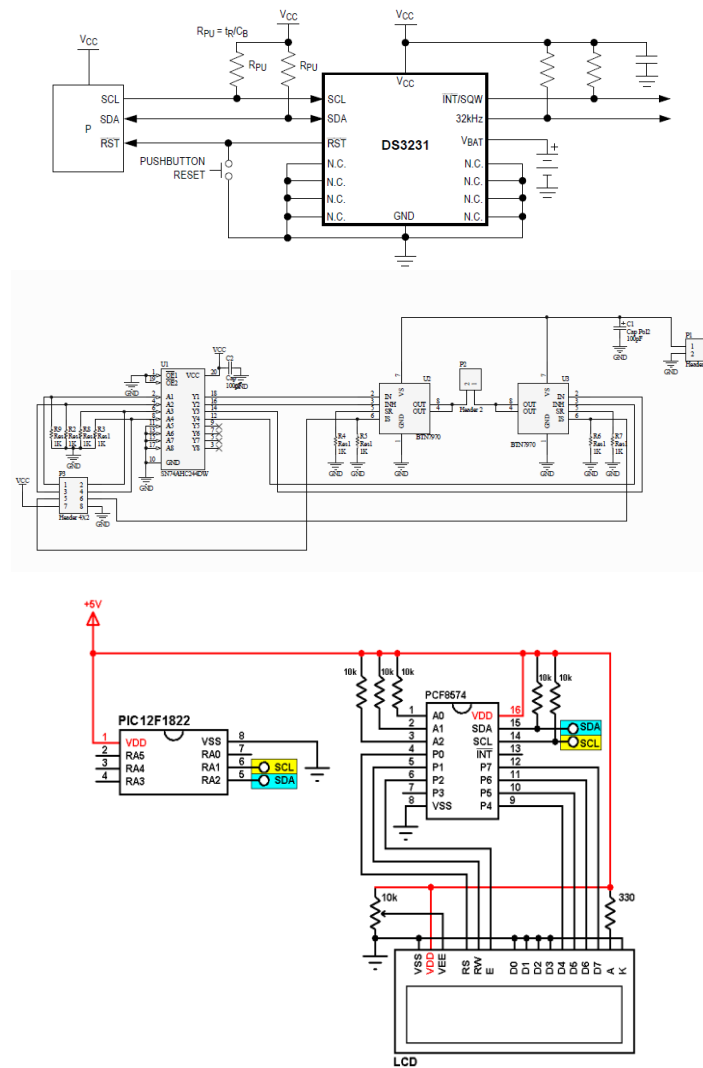
- [17] NXP Semiconductors, “PCF8574 remote 8-bit I/O expander for I²C-bus,” Data Sheet.
- [18] Omron Corporation, “B3F tactile switch,” Data Sheet.
- [19] TDK Corporation, “Piezoelectric buzzers PS series,” Data Sheet.
- [20] Thermonamic Electronics Corp., “Specification of thermoelectric module TEC1-12706,” Data Sheet.

RANGKAIAN SKEMATIK

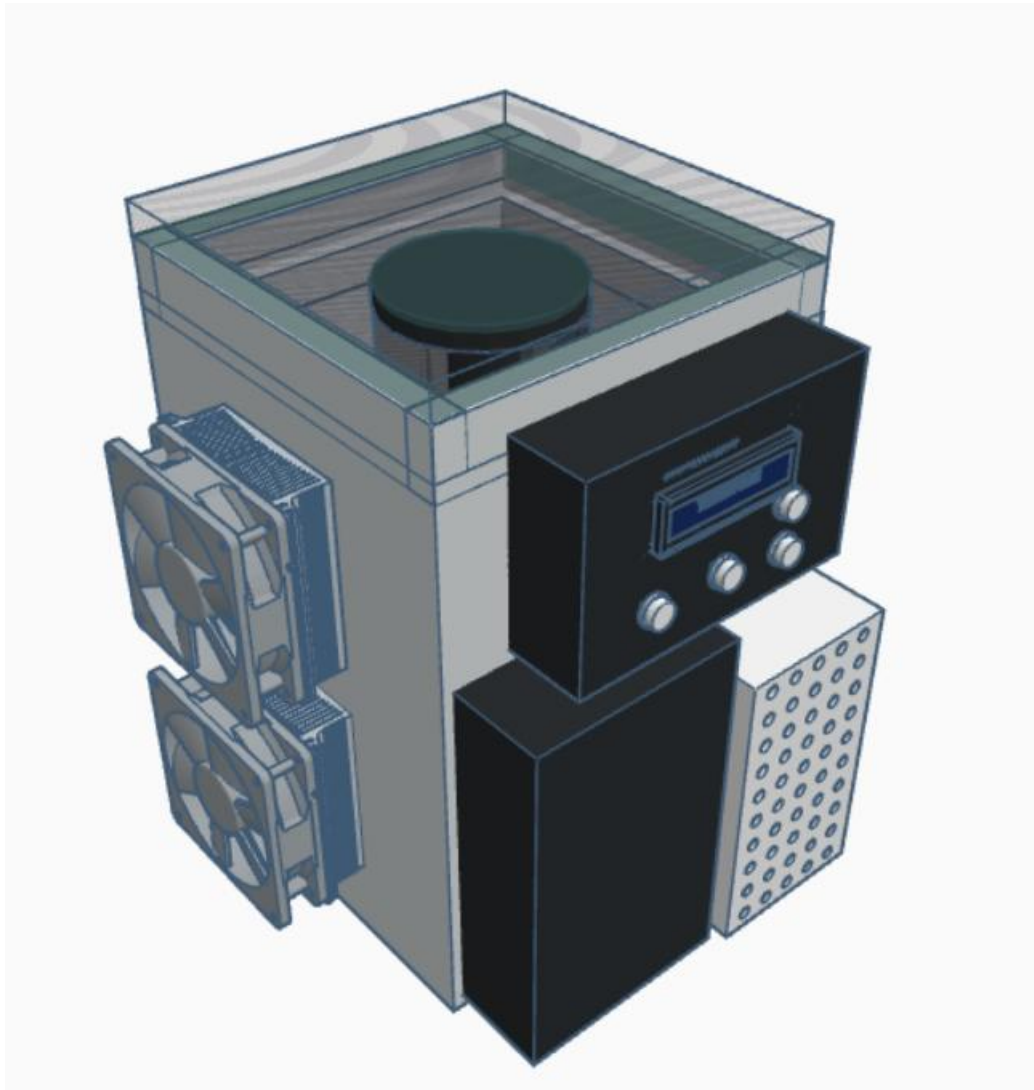


TITLE: SCH_coba_2026-07-07 (2) REV: 1.0
 Company: POLITEKNIK NEGERI PADANG Sheet: 1/1
 Date: 2026-07-06 Drawn By: ANUGERAH ILAHI

I	II	III					
			Nama Bagian	No. Bag	Bahan	Ukuran	Keterangan
			Rangkaian Skematik				Skala
							Dgbr
			POLITEKNIK NEGERI PADANG				Dprs
							No. BP: 2211013017
							Anugerah Ilahi
							TIM PENGUJI

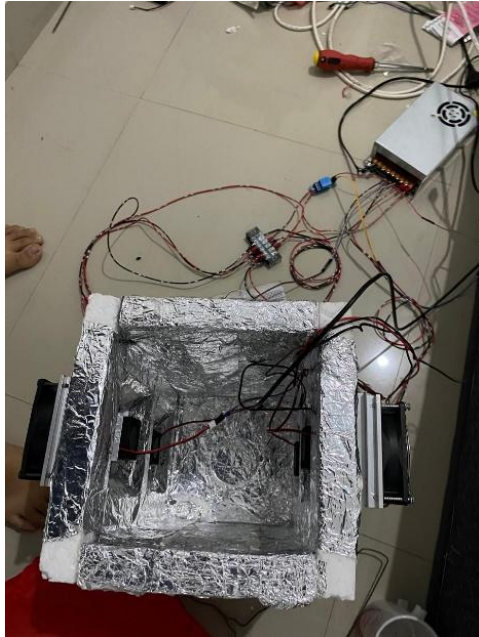


I	II	III						
			Nama Bagian	No. Bag	Bahan	Ukuran	Keterangan	
			Rangkaian Sensor DS3231, Modul Driver BTS7960, dan LCD I2C			Skala	Dgbr	Anugerah Ilahi
							Dprs	TIM PENGUJI
			POLITEKNIK NEGERI PADANG			No. BP: 2211013017		



I	II	III					
			Nama Bagian	No. Bag	Bahan	Ukuran	Keterangan
			Proses Pembuatan Alat				Skala
							Dgbr
							Dprs
			POLITEKNIK NEGERI PADANG				No. BP: 2211013017

Lampiran: Proses Pembuatan Alat





Lampiran: Program Arduino IDE

```
#include <Arduino.h>

#include <Wire.h>

#include <OneWire.h>

#include <DallasTemperature.h>

#include <RTCLib.h>

#include <LiquidCrystal_I2C.h>

#ifndef ESP_ARDUINO_VERSION_MAJOR
#define ESP_ARDUINO_VERSION_MAJOR 2
#endif

// ===== ALAMAT LCD =====
// Umumnya 0x27. Jika LCD tidak tampil, ubah menjadi 0x3F.
#define LCD_ADDR 0x27
#define LCD_COLS 20
#define LCD_ROWS 4

LiquidCrystal_I2C lcd(LCD_ADDR, LCD_COLS, LCD_ROWS);

// ===== PIN =====
#define PIN_SENSOR 4

#define PIN_I2C_SDA 21
#define PIN_I2C_SCL 22

#define PIN_BTN_DOWN 12
#define PIN_BTN_UP 5
#define PIN_BTN_OK 13

#define OK_ACTIVE_LOW false
#define UP_ACTIVE_LOW false
#define DOWN_ACTIVE_LOW false

#define PIN_BUZZER 23

#define BUZZER_PAKAI_TONE 1
```

```

#define BUZZER_ACTIVE_LOW false

const int BUZZER_TONE_FREQ = 2500;

const int CH_BUZZER = 7;

const unsigned long BUZZER_KLIK_MS = 80;

const unsigned long BUZZER_STEP_MS = 220;

const byte POLA_BUZZER_SELESAI[8] = {1, 1, 0, 0, 1, 1, 0, 0};

const byte BUZZER_SELESAI_ULANG = 6;

enum ModeBuzzer {

    BUZZER_DIAM,

    BUZZER_KLIK_TOMBOL,

    BUZZER_SELESAI

};

ModeBuzzer modeBuzzer = BUZZER_DIAM;

unsigned long waktuBuzzerMs = 0;

byte indeksPolaBuzzer = 0;

byte jumlahUlangPolaBuzzer = 0;

#define PIN_RPWM1 25

#define PIN_LPWM1 26

#define PIN_REN1 27

#define PIN_LEN1 14

#define PIN_RPWM2 32

#define PIN_LPWM2 33

#define PIN_REN2 18

#define PIN_LEN2 19

// ===== PWM ESP32 =====

const int PWM_FREQ = 5000;

const int PWM_RESOLUTION = 8;

const int CH_RPWM1 = 0;

const int CH_LPWM1 = 1;

const int CH_RPWM2 = 2;

const int CH_LPWM2 = 3;

// ===== SENSOR =====

```

```

OneWire oneWire(PIN_SENSOR);

DallasTemperature sensors(&oneWire);


RTC_DS3231 rtc;

bool rtcTersedia = false;


// Kalibrasi DS18B20

const float CAL_A = 1.0642f;

const float CAL_B = -0.6508f;


// ===== PID ZN NORMAL =====

// Hasil Ziegler-Nichols metode kurva reaksi proses dari data open-loop:

// K = 0.0837 C/PWM

// L = 0.43 menit = 25.8 detik

// T = 6.47 menit = 388.2 detik

// Kp = 215.72

// Ti = 51.6 detik

// Td = 12.9 detik

// Karena program memakai DT = 1 detik, maka:

// Ki = Kp / Ti = 4.181 per detik

// Kd = Kp * Td = 2782.79 detik

const float KP_NORMAL_ZN = 215.72f;

const float KI_NORMAL_ZN = 4.181f;

const float KD_NORMAL_ZN = 2782.79f;


// ===== PID ZN DINGIN =====

// Untuk uji awal closed-loop, mode normal dan dingin dibuat sama.

// Jika respons setpoint 5-8 C terlalu agresif, nilai mode dingin dapat diturunkan lagi.

const float KP_DINGIN_ZN = 215.72f;

const float KI_DINGIN_ZN = 4.181f;

const float KD_DINGIN_ZN = 2782.79f;


// Tetap otomatis sesuai permintaan.

const float SCALE_OTOMATIS = 1.0f;

const int PWM_MAX_TETAP = 255;

const int PWM_MIN = 0;


// ===== SETPOINT =====

float setpointC = 8.0f;

```

```

int durasiJam = 12;

const float SP_MIN = 5.0f;

const float SP_MAX = 20.0f;

const float BATAS_MODE_DINGIN = 8.0f;

const float BATAS_MODE_NORMAL = 9.0f;

const int DURASI_JAM_MIN = 1;

const int DURASI_JAM_MAX = 24;

// ===== PRESET =====

struct PresetColdBrew {

    const char *nama;

    float suhu;

    int jam;

};

const PresetColdBrew presetList[] = {

    {"5 C / 24 Jam", 5.0f, 24},

    {"8 C / 20 Jam", 8.0f, 20},

    {"12 C / 16 Jam", 12.0f, 16},

    {"15 C / 15 Jam", 15.0f, 15},

    {"20 C / 12 Jam", 20.0f, 12}

};

const int JUMLAH_PRESET = sizeof(presetList) / sizeof(presetList[0]);

// ===== SAMPLING =====

const unsigned long SAMPLE_MS = 1000UL;

const float DT = 1.0f;

const float TF = 1.0f;

const float ALPHA_D = TF / (TF + DT);

const int PWM_RAMP_STEP = 8;

// ===== PROTEKSI =====

const float SUHU_MIN_VALID = -10.0f;

const float SUHU_MAX_VALID = 60.0f;

```

```

const float SUHU_MIN_ABSOLUT = 4.20f;

const float OVERSHOOT_NORMAL_C = 2.0f;

const float OVERSHOOT_DINGIN_C = 1.0f;

const byte MAX_SENSOR_HOLD = 5;

// ===== TOMBOL =====

const unsigned long DEBOUNCE_MS = 80;

const unsigned long LOCKOUT_MS = 180;

struct Tombol {

    byte pin;

    bool stablePressed;

    bool lastReadingPressed;

    unsigned long lastChangeMs;

    unsigned long lastEventMs;

};

Tombol tombolOK = {PIN_BTN_OK, false, false, 0, 0};

Tombol tombolUP = {PIN_BTN_UP, false, false, 0, 0};

Tombol tombolDOWN = {PIN_BTN_DOWN, false, false, 0, 0};

const byte BTN_NONE = 0;

const byte BTN_PRESS = 1;

// ===== MODE =====

enum ModePID {

    MODE_NORMAL,

    MODE_DINGIN

};

ModePID modeAktif = MODE_DINGIN;

// ===== UI STATE =====

enum UIState {

    UI_MAIN_MENU,

    UI_PRESET_SELECT,

    UI_MANUAL_TEMP,

```



```

    UI_MANUAL_TIME,

    UI_CONFIRM_START,

    UI_RUNNING,

    UI_FINISHED

};

UIState uiState = UI_MAIN_MENU;

int menuUtamaIndex = 0;    // 0 preset, 1 manual

int presetIndex = 0;

bool dariPreset = true;

// ===== VARIABEL PID =====

bool pidAktif = false;

float integralTerm = 0.0f;

float derivativeFilter = 0.0f;

float suhuSebelumnya = NAN;

float suhuValidTerakhir = NAN;

float suhuRawTerakhir = NAN;

bool punyaSuhuValid = false;

int pwmAktual = 0;

unsigned long waktuMulaiSistem = 0;

unsigned long waktuKontrol = 0;

unsigned long waktuStartMs = 0;

// Timer proses cold brew baru mulai setelah suhu mencapai setpoint.

bool timerProsesAktif = false;

bool setpointPernahTercapai = false;

unsigned long waktuMulaiTimerMs = 0;

unsigned long durasiTotalDetik = 0;

unsigned long sisaDetik = 0;

uint32_t epochSelesai = 0;

bool waktuSelesaiValid = false;

```

```

unsigned long totalSensorError = 0;

byte errorSensorBeruntun = 0;

String bufferSerial = "";

// LCD update
unsigned long waktuLCDTerakhir = 0;

const unsigned long LCD_UPDATE_MS = 500;

// ===== PWM ESP32 =====

void setupPWM() {
  #if ESP_ARDUINO_VERSION_MAJOR >= 3

    ledcAttach(PIN_RPWM1, PWM_FREQ, PWM_RESOLUTION);
    ledcAttach(PIN_LPWM1, PWM_FREQ, PWM_RESOLUTION);
    ledcAttach(PIN_RPWM2, PWM_FREQ, PWM_RESOLUTION);
    ledcAttach(PIN_LPWM2, PWM_FREQ, PWM_RESOLUTION);

  #else

    ledcSetup(CH_RPWM1, PWM_FREQ, PWM_RESOLUTION);
    ledcSetup(CH_LPWM1, PWM_FREQ, PWM_RESOLUTION);
    ledcSetup(CH_RPWM2, PWM_FREQ, PWM_RESOLUTION);
    ledcSetup(CH_LPWM2, PWM_FREQ, PWM_RESOLUTION);

    ledcAttachPin(PIN_RPWM1, CH_RPWM1);
    ledcAttachPin(PIN_LPWM1, CH_LPWM1);
    ledcAttachPin(PIN_RPWM2, CH_RPWM2);
    ledcAttachPin(PIN_LPWM2, CH_LPWM2);

  #endif
}

void tulisPWMChannel(int pin, int channel, int nilai) {
  nilai = constrain(nilai, 0, 255);

  #if ESP_ARDUINO_VERSION_MAJOR >= 3

    ledcWrite(pin, nilai);

  #else

    ledcWrite(channel, nilai);

  #endif
}

```

```

void tulisPWMDriver(int pwmPendingin) {

    pwmPendingin = constrain(pwmPendingin, 0, 255);

    tulisPWMChannel(PIN_RPWM1, CH_RPWM1, pwmPendingin);
    tulisPWMChannel(PIN_LPWM1, CH_LPWM1, 0);

    tulisPWMChannel(PIN_RPWM2, CH_RPWM2, pwmPendingin);
    tulisPWMChannel(PIN_LPWM2, CH_LPWM2, 0);
}

// ===== FUNGSI DASAR =====

float batasiFloat(float nilai, float minimum, float maksimum) {

    if (nilai < minimum) return minimum;

    if (nilai > maksimum) return maksimum;

    return nilai;
}

const char *namaMode(ModePID mode) {

    return (mode == MODE_DINGIN) ? "DINGIN" : "NORMAL";
}

float kpAktif() {

    float kpDasar = (modeAktif == MODE_DINGIN)

        ? KP_DINGIN_ZN

        : KP_NORMAL_ZN;

    return kpDasar * SCALE_OTOMATIS;
}

float kiAktif() {

    float kiDasar = (modeAktif == MODE_DINGIN)

        ? KI_DINGIN_ZN

        : KI_NORMAL_ZN;

    return kiDasar * SCALE_OTOMATIS;
}

float kdAktif() {

    float kdDasar = (modeAktif == MODE_DINGIN)

        ? KD_DINGIN_ZN

```

```

        : KD_NORMAL_ZN;

    return kdDasar * SCALE_OTOMATIS;
}

void lcdPrintPad(byte col, byte row, String teks) {

    lcd.setCursor(col, row);

    if (teks.length() > LCD_COLS - col) {

        teks = teks.substring(0, LCD_COLS - col);

    }

    lcd.print(teks);

    int sisa = LCD_COLS - col - teks.length();

    for (int i = 0; i < sisa; i++) {

        lcd.print(' ');

    }

}

String format2(int nilai) {

    if (nilai < 10) {

        return "0" + String(nilai);

    }

    return String(nilai);

}

String formatDurasi(unsigned long detik) {

    unsigned long jam = detik / 3600UL;

    unsigned long menit = (detik % 3600UL) / 60UL;

    unsigned long sec = detik % 60UL;

    return String(jam) + ":" + format2(menit) + ":" + format2(sec);

}

String getJamRTC() {

    if (!rtcTersedia) {

        return "--:--";

    }

}

```

```

DateTime now = rtc.now();

return format2(now.hour()) + ":" + format2(now.minute());
}

String getRTCString() {
    if (!rtcTersedia) {
        return "NO_RTC";
    }

    DateTime now = rtc.now();

    char buffer[22];

    snprintf(
        buffer,
        sizeof(buffer),
        "%04d-%02d-%02d_%02d:%02d:%02d",
        now.year(),
        now.month(),
        now.day(),
        now.hour(),
        now.minute(),
        now.second()
    );

    return String(buffer);
}

// ===== KENDALI PWM =====

void tulisPWM(int targetPWM) {
    targetPWM = constrain(targetPWM, PWM_MIN, PWM_MAX_TETAP);

    if (targetPWM > pwmAktual + PWM_RAMP_STEP) {
        pwmAktual += PWM_RAMP_STEP;
    } else if (targetPWM < pwmAktual - PWM_RAMP_STEP) {
        pwmAktual -= PWM_RAMP_STEP;
    } else {
        pwmAktual = targetPWM;
    }
}

```

```

pwmAktual = constrain(pwmAktual, PWM_MIN, PWM_MAX_TETAP);

tulisPWMDriver(pwmAktual);

}

void pertahankanPWM() {

pwmAktual = constrain(pwmAktual, PWM_MIN, PWM_MAX_TETAP);

tulisPWMDriver(pwmAktual);

}

void matikanPeltier() {

pwmAktual = 0;

tulisPWMDriver(0);

}

// ===== BUZZER =====

void setupBuzzer() {

#if BUZZER_PAKAI_TONE

    #if ESP_ARDUINO_VERSION_MAJOR >= 3

        ledcAttach(PIN_BUZZER, BUZZER_TONE_FREQ, 8);

        ledcWrite(PIN_BUZZER, 0);

    #else

        ledcSetup(CH_BUZZER, BUZZER_TONE_FREQ, 8);

        ledcAttachPin(PIN_BUZZER, CH_BUZZER);

        ledcWrite(CH_BUZZER, 0);

    #endif

#else

    pinMode(PIN_BUZZER, OUTPUT);

    digitalWrite(PIN_BUZZER, BUZZER_ACTIVE_LOW ? HIGH : LOW);

#endif

}

void tulisBuzzer(bool nyala) {

#if BUZZER_PAKAI_TONE

    int duty = nyala ? 128 : 0;

    #if ESP_ARDUINO_VERSION_MAJOR >= 3

        ledcWrite(PIN_BUZZER, duty);

    #else

        ledcWrite(CH_BUZZER, duty);

    #endif

}

```

```

#else

    if (BUZZER_ACTIVE_LOW) {

        digitalWrite(PIN_BUZZER, nyala ? LOW : HIGH);

    } else {

        digitalWrite(PIN_BUZZER, nyala ? HIGH : LOW);

    }

#endif
}

void hentikanBuzzer() {

    modeBuzzer = BUZZER_DIAM;

    indeksPolaBuzzer = 0;

    jumlahUlangPolaBuzzer = 0;

    tulisBuzzer(false);

}

void bunyiTombol() {

    if (modeBuzzer == BUZZER_SELESAI) return;

    modeBuzzer = BUZZER_KLIK_TOMBOL;

    waktuBuzzerMs = millis();

    tulisBuzzer(true);

}

void mulaiBuzzerSelesai() {

    modeBuzzer = BUZZER_SELESAI;

    indeksPolaBuzzer = 0;

    jumlahUlangPolaBuzzer = 0;

    waktuBuzzerMs = millis();

    tulisBuzzer(POLA_BUZZER_SELESAI[indeksPolaBuzzer] == 1);

}

void updateBuzzer() {

    unsigned long nowMs = millis();

    if (modeBuzzer == BUZZER_DIAM) return;

    if (modeBuzzer == BUZZER_KLIK_TOMBOL) {

        if (nowMs - waktuBuzzerMs >= BUZZER_KLIK_MS) {

            hentikanBuzzer();

```

```

    }

    return;
}

if (modeBuzzer == BUZZER_SELESAI) {

    if (nowMs - waktuBuzzerMs >= BUZZER_STEP_MS) {

        waktuBuzzerMs = nowMs;

        indeksPolaBuzzer++;

        if (indeksPolaBuzzer >= 8) {

            indeksPolaBuzzer = 0;

            jumlahUlangPolaBuzzer++;

            if (jumlahUlangPolaBuzzer >= BUZZER_SELESAI_ULANG) {

                hentikanBuzzer();

                return;

            }

        }

        tulisBuzzer(POLA_BUZZER_SELESAI[indeksPolaBuzzer] == 1);

    }

}

void tesBuzzerAwal() {

    tulisBuzzer(true);

    delay(120);

    tulisBuzzer(false);

    delay(100);

    tulisBuzzer(true);

    delay(120);

    tulisBuzzer(false);

}

// ===== SENSOR =====

bool bacaSuhu(float &rawC, float &kalibrasiC) {

    // Non-blocking: ambil hasil konversi sebelumnya,

    // lalu minta konversi baru untuk pembacaan berikutnya.

    // Ini membuat tombol tidak tertahan proses DS18B20 12-bit.

```



```

rawC = sensors.getTempCByIndex(0);
sensors.requestTemperatures();

if (rawC == DEVICE_DISCONNECTED_C ||
    isnan(rawC) ||
    rawC < SUHU_MIN_VALID ||
    rawC > SUHU_MAX_VALID) {
    return false;
}

kalibrasiC = CAL_A * rawC + CAL_B;

if (isnan(kalibrasiC) ||
    kalibrasiC < SUHU_MIN_VALID ||
    kalibrasiC > SUHU_MAX_VALID) {
    return false;
}

return true;
}

// ===== MODE PID =====
ModePID modeDariSetpoint(float sp, ModePID modeSebelumnya) {
    if (sp <= BATAS_MODE_DINGIN) {
        return MODE_DINGIN;
    }

    if (sp >= BATAS_MODE_NORMAL) {
        return MODE_NORMAL;
    }

    return modeSebelumnya;
}

void perbaruiModeDariSetpoint() {
    ModePID modeBaru = modeDariSetpoint(setpointC, modeAktif);

    if (modeBaru != modeAktif) {
        modeAktif = modeBaru;
    }
}

```

```

    derivativeFilter = 0.0f;

    suhuSebelumnya = NAN;

    integralTerm = 0.0f;

}

}

void resetPID() {

    integralTerm = 0.0f;

    derivativeFilter = 0.0f;

    suhuSebelumnya = NAN;

}

bool suhuMencapaiSetpoint(float suhuC) {

    // Sistem ini hanya mendinginkan, jadi proses dianggap siap ketika
    // suhu aktual sudah sama dengan atau lebih rendah dari setpoint.
    // Toleransi 0.2 C dipakai agar timer tidak terlalu lama menunggu
    // karena noise sensor kecil.

    return suhuC <= (setpointC + 0.20f);

}

void mulaiTimerProses() {

    if (timerProsesAktif) {

        return;

    }

    timerProsesAktif = true;

    setpointPernahTercapai = true;

    waktuMulaiTimerMs = millis();

    sisaDetik = durasiTotalDetik;

    if (rtcTersedia) {

        DateTime now = rtc.now();

        epochSelesai = now.unixtime() + durasiTotalDetik;

        waktuSelesaiValid = true;

    } else {

        epochSelesai = 0;

        waktuSelesaiValid = false;

    }

}

```

```

Serial.println("# SETPOINT TERCAPAI - TIMER DIMULAI");
}

// ===== START/STOP =====

void mulaiPIDKontrol() {
    perbaruiModeDariSetpoint();
    resetPID();

    errorSensorBeruntun = 0;
    pidAktif = true;

    waktuStartMs = millis();

    // Durasi proses disiapkan, tetapi countdown belum berjalan.
    // Countdown baru dimulai setelah suhu mencapai setpoint.
    timerProsesAktif = false;
    setpointPernahTercapai = false;
    waktuMulaiTimerMs = 0;

    durasiTotalDetik = (unsigned long)durasiJam * 3600UL;
    sisaDetik = durasiTotalDetik;

    epochSelesai = 0;
    waktuSelesaiValid = false;

    uiState = UI_RUNNING;
    lcd.clear();

    Serial.println("# START PENDINGINAN");
    Serial.println("# TIMER AKAN MULAI SETELAH SETPOINT TERCAPAI");
    Serial.print("# SP=");
    Serial.println(setpointC, 1);
    Serial.print("# DURASI_JAM=");
    Serial.println(durasiJam);
}

void stopPIDKontrol(const char *alasan) {
    pidAktif = false;
    resetPID();
}

```

```

matikanPeltier();

timerProsesAktif = false;

setpointPernahTercapai = false;

waktuMulaiTimerMs = 0;

waktuSelesaiValid = false;

Serial.print("# STOP: ");

Serial.println(alasan);
}

void updateSisaWaktu() {

    if (!pidAktif) {

        return;

    }

    // Selama suhu belum mencapai setpoint, timer belum berjalan.

    if (!timerProsesAktif) {

        sisaDetik = durasiTotalDetik;

        return;

    }

    if (rtcTersedia && waktuSelesaiValid) {

        DateTime now = rtc.now();

        uint32_t nowEpoch = now.unixtime();

        if (nowEpoch >= epochSelesai) {

            sisaDetik = 0;

        } else {

            sisaDetik = epochSelesai - nowEpoch;

        }

    } else {

        unsigned long berjalan = (millis() - waktuMulaiTimerMs) / 1000UL;

        if (berjalan >= durasiTotalDetik) {

            sisaDetik = 0;

        } else {

            sisaDetik = durasiTotalDetik - berjalan;

        }

    }

}

```

```

}

if (sisadetik == 0) {
    stopPIDKontrol("WAKTU SELESAI");
    mulaiBuzzerSelesai();
    uiState = UI_FINISHED;
    lcd.clear();
}
}

// ===== TOMBOL =====

void initTombol(Tombol &t) {
    bool activeLow = true;

    if (t.pin == PIN_BTN_OK) {
        activeLow = OK_ACTIVE_LOW;
    } else if (t.pin == PIN_BTN_UP) {
        activeLow = UP_ACTIVE_LOW;
    } else if (t.pin == PIN_BTN_DOWN) {
        activeLow = DOWN_ACTIVE_LOW;
    }

    if (activeLow) {
        pinMode(t.pin, INPUT_PULLUP);
    } else {
        pinMode(t.pin, INPUT_PULLDOWN);
    }

    t.stablePressed = false;
    t.lastReadingPressed = false;
    t.lastChangeMs = millis();
    t.lastEventMs = 0;
}

bool bacaTombolMentah(Tombol &t) {
    int nilai = digitalRead(t.pin);
    bool activeLow = true;

    if (t.pin == PIN_BTN_OK) {

```

```

        activeLow = OK_ACTIVE_LOW;
    } else if (t.pin == PIN_BTN_UP) {
        activeLow = UP_ACTIVE_LOW;
    } else if (t.pin == PIN_BTN_DOWN) {
        activeLow = DOWN_ACTIVE_LOW;
    }

    if (activeLow) {
        return nilai == LOW;
    } else {
        return nilai == HIGH;
    }
}

byte bacaEventTombol(Tombol &t) {
    unsigned long nowMs = millis();
    bool readingPressed = bacaTombolMentah(t);

    if (readingPressed != t.lastReadingPressed) {
        t.lastReadingPressed = readingPressed;
        t.lastChangeMs = nowMs;
    }

    if ((nowMs - t.lastChangeMs) > DEBOUNCE_MS &&
        readingPressed != t.stablePressed) {
        t.stablePressed = readingPressed;

        // Event hanya satu kali saat tombol berubah menjadi ditekan.
        // Tidak ada auto-repeat agar menu tidak turun sendiri.
        if (t.stablePressed &&
            (nowMs - t.lastEventMs >= LOCKOUT_MS)) {
            t.lastEventMs = nowMs;
            return BTN_PRESS;
        }
    }

    return BTN_NONE;
}

```

```

void prosesOKPendek() {

    if (uiState == UI_RUNNING) {

        stopPIDKontrol("TOMBOL OK");

        uiState = UI_MAIN_MENU;

        lcd.clear();

        return;

    }

    if (uiState == UI_MAIN_MENU) {

        if (menuUtamaIndex == 0) {

            dariPreset = true;

            uiState = UI_PRESET_SELECT;

        } else {

            dariPreset = false;

            uiState = UI_MANUAL_TEMP;

        }

        lcd.clear();

        return;

    }

    if (uiState == UI_PRESET_SELECT) {

        setpointC = presetList[presetIndex].suhu;

        durasiJam = presetList[presetIndex].jam;

        dariPreset = true;

        perbaruiModeDariSetpoint();

        uiState = UI_CONFIRM_START;

        lcd.clear();

        return;

    }

    if (uiState == UI_MANUAL_TEMP) {

        dariPreset = false;

        perbaruiModeDariSetpoint();

        uiState = UI_MANUAL_TIME;

        lcd.clear();

        return;

    }

    if (uiState == UI_MANUAL_TIME) {

```

```

    uiState = UI_CONFIRM_START;

    lcd.clear();

    return;
}

if (uiState == UI_CONFIRM_START) {

    mulaiPIDKontrol();

    return;
}

if (uiState == UI_FINISHED) {

    uiState = UI_MAIN_MENU;

    lcd.clear();

    return;
}
}

void prosesUpDown(int arah) {

    if (uiState == UI_MAIN_MENU) {

        menuUtamaIndex += arah;

        if (menuUtamaIndex < 0) menuUtamaIndex = 1;

        if (menuUtamaIndex > 1) menuUtamaIndex = 0;

        return;
    }

    if (uiState == UI_PRESET_SELECT) {

        presetIndex += arah;

        if (presetIndex < 0) presetIndex = JUMLAH_PRESET - 1;

        if (presetIndex >= JUMLAH_PRESET) presetIndex = 0;

        return;
    }

    if (uiState == UI_MANUAL_TEMP) {

        setpointC += 0.5f * arah;

        setpointC = batasiFloat(setpointC, SP_MIN, SP_MAX);

        perbaruiModeDariSetpoint();

        return;
    }

```



```

}

if (uiState == UI_MANUAL_TIME) {

    durasiJam += arah;

    if (durasiJam < DURASI_JAM_MIN) durasiJam = DURASI_JAM_MIN;

    if (durasiJam > DURASI_JAM_MAX) durasiJam = DURASI_JAM_MAX;

    return;

}

}

void prosesTombol() {

    byte evOK = bacaEventTombol(tombolOK);

    byte evUP = bacaEventTombol(tombolUP);

    byte evDOWN = bacaEventTombol(tombolDOWN);

    // Jika dua tombol terbaca bersamaan, abaikan untuk mencegah ghost/noise.

    byte jumlahEvent = 0;

    if (evOK == BTN_PRESS) jumlahEvent++;

    if (evUP == BTN_PRESS) jumlahEvent++;

    if (evDOWN == BTN_PRESS) jumlahEvent++;

    if (jumlahEvent != 1) {

        return;

    }

    bunyiTombol();

    if (evOK == BTN_PRESS) {

        prosesOKPendek();

        return;

    }

    if (evUP == BTN_PRESS) {

        prosesUpDown(+1);

        return;

    }

    if (evDOWN == BTN_PRESS) {

        prosesUpDown(-1);

```

```

    return;
}

}

// ===== LCD =====

void tampilLCD() {

    if (uiState == UI_MAIN_MENU) {

        lcdPrintPad(0, 0, "Pilih Mode Kopi");

        lcdPrintPad(0, 1, String(menuUtamaIndex == 0 ? ">" : " ") + " PRESET");

        lcdPrintPad(0, 2, String(menuUtamaIndex == 1 ? ">" : " ") + " MANUAL");

        lcdPrintPad(0, 3, "UP/DOWN OK=Pilih");

        return;

    }

    if (uiState == UI_PRESET_SELECT) {

        lcdPrintPad(0, 0, "Pilih Preset");

        lcdPrintPad(0, 1, String("> ") + presetList[presetIndex].nama);

        lcdPrintPad(0, 2, "UP/DOWN ganti");

        lcdPrintPad(0, 3, "OK lanjut");

        return;

    }

    if (uiState == UI_MANUAL_TEMP) {

        lcdPrintPad(0, 0, "Manual: Atur Suhu");

        lcdPrintPad(0, 1, "Suhu: " + String(setpointC, 1) + " C");

        lcdPrintPad(0, 2, "UP/DOWN ubah");

        lcdPrintPad(0, 3, "OK lanjut");

        return;

    }

    if (uiState == UI_MANUAL_TIME) {

        lcdPrintPad(0, 0, "Manual: Atur Waktu");

        lcdPrintPad(0, 1, "Waktu: " + String(durasiJam) + " Jam");

        lcdPrintPad(0, 2, "UP/DOWN ubah");

        lcdPrintPad(0, 3, "OK lanjut");

        return;

    }

    if (uiState == UI_CONFIRM_START) {

```

```

    lcdPrintPad(0, 0, "Mulai Cold Brew?");

    lcdPrintPad(0, 1, "SP: " + String(setpointC, 1) + " C");

    lcdPrintPad(0, 2, "Waktu: " + String(durasiJam) + " Jam");

    lcdPrintPad(0, 3, "OK Start");

    return;
}

if (uiState == UI_RUNNING) {
    if (timerProsesAktif) {
        lcdPrintPad(0, 0, "Sisa: " + formatDurasi(sisaDetik));
    } else {
        lcdPrintPad(0, 0, "Tunggu SP tercapai");
    }

    lcdPrintPad(0, 1, "Suhu: " + String(suhuValidTerakhir, 1) + " C");

    lcdPrintPad(0, 2, "SP: " + String(setpointC, 1) + "C PWM: " + String(pwmAktual));

    if (timerProsesAktif) {
        lcdPrintPad(0, 3, String(namaMode(modeAktif)) + " OK=STOP");
    } else {
        lcdPrintPad(0, 3, "Timer belum jalan");
    }

    return;
}

if (uiState == UI_FINISHED) {
    lcdPrintPad(0, 0, "Cold Brew Selesai");

    lcdPrintPad(0, 1, "Suhu: " + String(suhuValidTerakhir, 1) + " C");

    lcdPrintPad(0, 2, "PWM: 0");

    lcdPrintPad(0, 3, "OK ke menu awal");

    return;
}
}

// ===== PERINTAH SERIAL =====

void tampilkanStatus() {
    Serial.println("# ===== STATUS =====");

    Serial.print("# RUN=");

    Serial.println(pidAktif ? 1 : 0);
}

```

```

Serial.print("# UI=");

Serial.println((int)uiState);

Serial.print("# RTC=");

Serial.println(getRTCString());

Serial.print("# SP=");

Serial.println(setpointC, 2);

Serial.print("# DURASI_JAM=");

Serial.println(durasiJam);

Serial.print("# TIMER_AKTIF=");

Serial.println(timerProsesAktif ? 1 : 0);

Serial.print("# SETPOINT_TERCAPAI=");

Serial.println(setpointPernahTercapai ? 1 : 0);

Serial.print("# MODE=");

Serial.println(namaMode(modeAktif));

Serial.print("# SCALE=");

Serial.println(SCALE_OTOMATIS, 2);

Serial.print("# PMAX=");

Serial.println(PWM_MAX_TETAP);

Serial.print("# PWM=");

Serial.println(pwmAktual);

Serial.println("# =====");
}

void prosesPerintah(String perintah) {

    perintah.trim();

    perintah.toUpperCase();

    if (perintah == "START" || perintah == "RUN=1") {

        mulaiPIDKontrol();

        return;

    }

    if (perintah == "STOP" || perintah == "RUN=0") {

        stopPIDKontrol("SERIAL");

        uiState = UI_MAIN_MENU;

        lcd.clear();

        return;

    }
}

```

```

if (perintah.startsWith("SP=")) {

    float nilai = perintah.substring(3).toFloat();

    if (nilai >= SP_MIN && nilai <= SP_MAX) {

        setpointC = nilai;

        perbaruiModeDariSetpoint();

    }

    return;

}

if (perintah.startsWith("DUR=")) {

    int nilai = perintah.substring(4).toInt();

    if (nilai >= DURASI_JAM_MIN && nilai <= DURASI_JAM_MAX) {

        durasiJam = nilai;

    }

    return;

}

if (perintah == "STATUS") {

    tampilkanStatus();

    return;

}

}

void bacaPerintahSerial() {

    while (Serial.available() > 0) {

        char c = Serial.read();

        if (c == '\n' || c == '\r') {

            if (bufferSerial.length() > 0) {

                prosesPerintah(bufferSerial);

                bufferSerial = "";

            }

        } else {

            bufferSerial += c;

            if (bufferSerial.length() > 50) {

                bufferSerial = "";

            }

        }

    }

}

```

```

    }
}
}
}

// ===== CSV =====

void kirimCSV(float waktuS,
              float rawC,
              float suhuC,
              float error,
              float pTerm,
              float iTerm,
              float dTerm,
              bool dataHold,
              const char *status) {
    Serial.print(waktuS, 1);
    Serial.print(',');

    if (isnan(rawC)) Serial.print("nan");
    else Serial.print(rawC, 3);

    Serial.print(',');

    if (isnan(suhuC)) Serial.print("nan");
    else Serial.print(suhuC, 3);

    Serial.print(',');

    Serial.print(setpointC, 2);
    Serial.print(',');
    Serial.print(error, 3);
    Serial.print(',');
    Serial.print(kpAktif(), 6);
    Serial.print(',');
    Serial.print(kiAktif(), 6);
    Serial.print(',');
    Serial.print(kdAktif(), 6);
    Serial.print(',');
    Serial.print(pTerm, 3);

```

```

Serial.print(',');

Serial.print(iTerm, 3);

Serial.print(',');

Serial.print(dTerm, 3);

Serial.print(',');

Serial.print(pwmAktual);

Serial.print(',');

Serial.print(SCALE_OTOMATIS, 2);

Serial.print(',');

Serial.print(pidAktif ? 1 : 0);

Serial.print(',');

Serial.print(timerProsesAktif ? 1 : 0);

Serial.print(',');

Serial.print(totalSensorError);

Serial.print(',');

Serial.print(errorSensorBeruntun);

Serial.print(',');

Serial.print(dataHold ? 1 : 0);

Serial.print(',');

Serial.print(status);

Serial.print(',');

Serial.print(getRTCString());

Serial.print(',');

Serial.println(namaMode(modeAktif));
}

// ===== SETUP =====

void setup() {

  Serial.begin(9600);

  delay(1000);

  Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);

  lcd.init();

  lcd.backlight();

  lcd.clear();

  setupBuzzer();

  hentikanBuzzer();

```

```

tesBuzzerAwal();

if (rtc.begin()) {

    rtcTersedia = true;

    if (rtc.lostPower()) {

        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

    }

} else {

    rtcTersedia = false;

}

initTomboI(tombolOK);

initTomboI(tombolUP);

initTomboI(tombolDOWN);

pinMode(PIN_REN1, OUTPUT);

pinMode(PIN_LEN1, OUTPUT);

pinMode(PIN_REN2, OUTPUT);

pinMode(PIN_LEN2, OUTPUT);

digitalWrite(PIN_REN1, HIGH);

digitalWrite(PIN_LEN1, HIGH);

digitalWrite(PIN_REN2, HIGH);

digitalWrite(PIN_LEN2, HIGH);

setupPWM();

matikanPeltier();

sensors.begin();

sensors.setResolution(12);

sensors.setWaitForConversion(false);

sensors.requestTemperatures();

waktuMulaiSistem = millis();

waktuKontrol = millis();

perbaruiModeDariSetpoint();

```



```

lcdPrintPad(0, 0, "Cold Brew Cooler");

lcdPrintPad(0, 1, "ESP32 PID Control");

lcdPrintPad(0, 2, "Scale 0.20 PMAx255");

lcdPrintPad(0, 3, "Loading...");

delay(1500);

lcd.clear();


Serial.println("# COLD BREW COOLER ESP32");

Serial.println("# UI: PRESET / MANUAL");

Serial.println("# OK=pilih/lanjut/stop, UP/DOWN=ubah");

Serial.println("# TOMBOL: DOWN=GPIO12, UP=GPIO5, OK=GPIO13");

Serial.println("# LOGIKA: ACTIVE-HIGH, gunakan INPUT_PULLDOWN internal");

Serial.println("waktu_s,suhu_raw_C,suhu_C,setpoint_C,error_C,Kp,Ki,Kd,P,I,D,pwm,scale,pid_aktif,timer_aktif,total_sensor_error,error_ber
untun,data_hold,status,rtc_datetime,mode");
}

// ===== LOOP =====

void loop() {

  bacaPerintahSerial();

  updateBuzzer();

  prosesTombol();

  updateSisaWaktu();


  unsigned long sekarang = millis();

  if (sekarang - waktuLCDTerakhir >= LCD_UPDATE_MS) {

    waktuLCDTerakhir = sekarang;

    tampilLCD();

  }

  if (sekarang - waktuKontrol < SAMPLE_MS) {

    return;

  }

  waktuKontrol += SAMPLE_MS;

  float rawC = NAN;

  float suhuC = NAN;

  bool sensorValid = bacaSuhu(rawC, suhuC);

```

```
float waktuS = (millis() - waktuMulaiSistem) / 1000.0f;
```

```
if (!sensorValid) {
```

```
    totalSensorError++;
```

```
    errorSensorBeruntun++;
```

```
if (punyaSuhuValid &&
```

```
    errorSensorBeruntun <= MAX_SENSOR_HOLD) {
```

```
    pertahankanPWM();
```

```
float errorHold = suhuValidTerakhir - setpointC;
```

```
float pHold = kpAktif() * errorHold;
```

```
    kirimCSV(
```

```
        waktuS,
```

```
        suhuRawTerakhir,
```

```
        suhuValidTerakhir,
```

```
        errorHold,
```

```
        pHold,
```

```
        integralTerm,
```

```
        0.0f,
```

```
        true,
```

```
        "HOLD_SUHU_TERAKHIR"
```

```
    );
```

```
    return;
```

```
}
```

```
stopPIDKontrol("SENSOR ERROR");
```

```
    kirimCSV(
```

```
        waktuS,
```

```
        NAN,
```

```
        NAN,
```

```
        NAN,
```

```
        NAN,
```

```
        integralTerm,
```

```
        NAN,
```

```

        false,

        "SENSOR_ERROR_PID_OFF"

    );

    return;
}

errorSensorBeruntun = 0;
suhuRawTerakhir = rawC;
suhuValidTerakhir = suhuC;
punyaSuhuValid = true;

perbaruiModeDariSetpoint();

if (pidAktif &&
    !timerProsesAktif &&
    suhuMencapaiSetpoint(suhuC)) {
    mulaiTimerProses();
}

float batasOvershoot =
    (modeAktif == MODE_DINGIN)
    ? OVERSHOOT_DINGIN_C
    : OVERSHOOT_NORMAL_C;

float batasSuhuMode = setpointC - batasOvershoot;
float batasStop = (SUHU_MIN_ABSOLUT > batasSuhuMode)
    ? SUHU_MIN_ABSOLUT
    : batasSuhuMode;

if (suhuC <= batasStop) {
    stopPIDKontrol("PROTEKSI OVERSHOOT");
    uiState = UI_FINISHED;
    lcd.clear();

    kirimCSV(
        waktuS,
        rawC,
        suhuC,

```

```

    suhuC - setpointC,

    0.0f,

    0.0f,

    0.0f,

    false,

    "PROTEKSI_OVERSHOOT"

);

return;
}

float error = suhuC - setpointC;

float pTerm = 0.0f;

float dTerm = 0.0f;

if (pidAktif) {

    float Kp = kpAktif();

    float Ki = kiAktif();

    float Kd = kdAktif();

    pTerm = Kp * error;

    float dSuhu = 0.0f;

    if (!isnan(suhuSebelumnya)) {

        dSuhu = (suhuC - suhuSebelumnya) / DT;

    }

    derivativeFilter =

        ALPHA_D * derivativeFilter +

        (1.0f - ALPHA_D) * dSuhu;

    dTerm = -Kd * derivativeFilter;

    float integralKandidat =

        integralTerm + Ki * error * DT;

    integralKandidat =

        batasiFloat(integralKandidat, -255.0f, 255.0f);

```

```

float outputKandidat =
    pTerm + integralKandidat + dTerm;

bool saturasiAtas =
    outputKandidat > PWM_MAX_TETAP && error > 0.0f;

bool saturasiBawah =
    outputKandidat < PWM_MIN && error < 0.0f;

if (!(saturasiAtas || saturasiBawah)) {
    integralTerm = integralKandidat;
}

float outputPID =
    pTerm + integralTerm + dTerm;

int pwmTarget =
    (int)(batasiFloat(outputPID, PWM_MIN, PWM_MAX_TETAP) + 0.5f);

tulisPWM(pwmTarget);
} else {
    matikanPeltier();
}

suhuSebelumnya = suhuC;

const char *status =
    pidAktif ? "RUNNING" : "OFF";

if (pidAktif && abs(error) <= 0.20f) {
    status = "DEKAT_SETPOINT";
}

 kirimCSV(
    waktuS,
    rawC,
    suhuC,
    error,

```

```
pTerm,  
integralTerm,  
dTerm,  
false,  
status  
);  
}
```